数值方法与科学计算

Numerical Methods & Scientific Computing

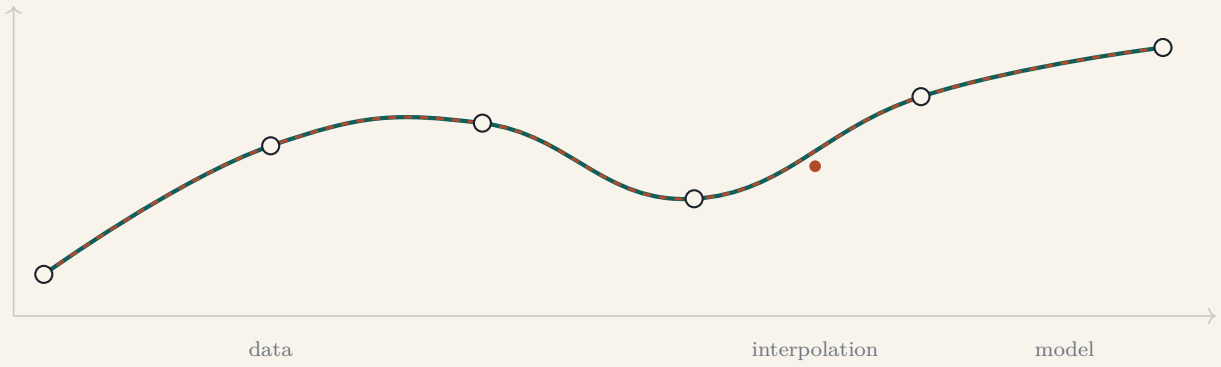
双语总复习讲义

A Bilingual Comprehensive Revision Guide

Ch. 9 Numbers & floating point · Ch. 11 Files & formats

Ch. 12 Visualization · Ch. 16 Least squares · Ch. 17 Interpolation

Tutorials 3–5 Python essentials, complexity, file I/O



Compiled 30 October 2026 · For personal study use

How to use this guide

使用说明：这份讲义怎么用

This guide consolidates the lecture material for **Chapters 9, 11, 12, 16 and 17** of PHYB57 together with **Tutorials 3–5**. It is written for revision rather than first learning: every topic is presented twice — an English exposition that mirrors the language and notation of the slides, followed by a Chinese companion that explains the intuition, the examinable 细节 and the traps.

> **中文精讲** Chinese companion
>
> 这份讲义把 **第 9、11、12、16、17 章**的课件与 **Tutorial 3–5** 合并成一条完整的复习线索：数字如何在计算机中表示 → 数据如何存盘 → 数据如何可视化 → 如何从带噪声的数据中拟合模型 → 如何让曲线精确穿过数据点。
>
> 每个主题都采用「英文讲解 + 中文精讲」的双层结构。英文部分贴近课件原文与术语，方便你对照幻灯片和考试题干；中文部分负责讲清直觉、推导细节、代码实现与易错点。时间紧时，可以只读中文精讲、公式框和每章末尾的考点清单；时间充裕时，建议英文与中文对照阅读，并用每章的“微型练习”自测。
>
> 全书最后附有：公式速查表、Python API 速查表、30 道期末风格练习题（含答案）以及“考前最后十分钟”清单。

Coverage map 覆盖对照

Source	Topics	Where it is covered
C9	Number systems, binary, IEEE 754, round-off	Chapters 3–5
C11	TXT, CSV, Pickle, JSON, HDF5	Chapters 6–10
C12	2D/3D plots, maps, animation	Chapters 11–15
C16 (1–2)	Least squares, linear algebra/calculus, nonlinear fits	Chapters 16–19
C17	Linear, cubic spline, Lagrange, Newton interpolation	Chapters 20–24
Tutorial 3	Conditionals, loops, comprehensions, functions, lambda	Chapter 1
Tutorial 4	Algorithmic complexity and Big-O	Chapter 2
Tutorial 5	Text files, NumPy I/O, 2D plotting	Chapters 6, 11

The five ideas that hold the course together

1. **Representation is approximation.** A finite number of bits cannot store most real numbers; every later numerical error starts here.
2. **Storage is a contract.** The file format you choose fixes what can be stored, who can read it, and how fast.
3. **A plot is an argument.** Axes, units, legends and colour are not decoration; they are part of the evidence.
4. **Fitting estimates, interpolation reproduces.** Least squares minimises a global mismatch; interpolation passes through every datum.
5. **Conditioning and stability decide whether the mathematics survives the computer.** The formula can be right and the answer still wrong.

A suggested revision sequence 建议复习顺序

Pass	Read	Do
1	Chapters 1–5	Python/complexity drills A–B; write the float and error formulas from memory
2	Chapters 6–10	File-format drills C; sketch an HDF5 tree and a CSV round trip
3	Chapters 11–15	Plotting drills D; reproduce the damped-oscillator figure with labels
4	Chapters 16–19	Least-squares drills E; derive the normal equations once without notes
5	Chapters 20–24	Interpolation drills F; fill the Newton table and count spline conditions
6	Formula sheet + drills	Timed mock: 30 questions in 90 minutes; review only the misses

Contents ·目录

1 Python Essentials: Control Flow and Functions . . . 2

1.1 Conditional statements . . . 2

1.2 Looping: `for` and `while` . . . 3

1.3 Comprehensions . . . 4

1.4 Functions, scope and `lambda` . . . 4

1.5 Worked tutorial exercises . . . 5

2 Algorithmic Complexity and Big-O Notation . . . 8

2.1 The definition, without the epsilons . . . 8

2.2 The standard orders . . . 8

2.3 Reading code for complexity . . . 9

2.4 Empirical timing and its limits . . . 11

3 Number Representation and Binary Arithmetic . . . 13

3.1 Decimal and base-N systems . . . 13

3.2 Binary numbers . . . 14

3.3 Why binary, and what it costs . . . 14

4 IEEE 754 Floating Point . . . 16

4.1 The 64-bit layout . . . 16

4.2 A worked decode . . . 17

4.3 Precision, spacing and machine epsilon . . . 17

4.4 Special values, overflow and underflow . . . 18

5 Round-Off Error . . . 19

5.1 Representation error . . . 20

5.2 Cancellation and the loss of significance . . . 20

5.3 Accumulation and repeated rounding . . . 21

5.4 How to live with round-off error . . . 22

6 Text Files: **`.txt`** . . . 24

6.1 Opening a file: modes and the `with` statement . . . 24

6.2 Reading: four ways . . . 25

6.3 Numerical arrays with NumPy . . . 25

6.4 Checklist and common errors . . . 26

7 CSV Files . . . 27
7.1 Structure and pitfalls . . . 27
7.2 Writing and reading with the `csv` module . . . 28
7.3 NumPy and pandas . . . 28
7.4 CSV versus TXT . . . 29
8 Pickle and JSON . . . 30
8.1 Pickle: Python's private memory . . . 30
8.2 JSON: the common language . . . 31
8.3 Choosing a format . . . 32
9 HDF5 and Hierarchical Data . . . 33
9.1 Groups, datasets and attributes . . . 33
9.2 Writing a file with `h5py` . . . 34
9.3 Reading and exploring . . . 35
9.4 Why HDF5 . . . 35
10 Choosing a Data Format . . . 36
10.1 A decision procedure . . . 36
10.2 Comparison and selection . . . 37
10.3 Worked scenario . . . 37
11 2D Plotting with Matplotlib . . . 40
11.1 The figure–axes model . . . 40
11.2 Format strings . . . 40
11.3 Context: titles, labels, legends, grid . . . 41
11.4 Worked example: damped oscillator . . . 42
12 3D Plotting . . . 44
12.1 Creating a 3D axes . . . 44
12.2 3D line and scatter plots . . . 44
12.3 Surfaces and `meshgrid` . . . 45
12.4 Reading a 3D plot critically . . . 46
13 Maps with Cartopy . . . 47
13.1 Coordinates and projections . . . 47
13.2 Cartopy basics . . . 47
13.3 Basemap, Cartopy and Folium . . . 48
14 Animations . . . 50
14.1 The frame model . . . 50

14.2 Metadata, writers and formats . . . 51
14.3 Designing an animation that teaches . . . 52

15 A Checklist for Scientific Figures . . . 52
15.1 The ten-point check . . . 53
15.2 Choosing the right chart . . . 53

16 Least Squares Regression . . . 56
16.1 Why not just fit every point? . . . 56
16.2 The estimation function . . . 57
16.3 The residual and the objective . . . 58
16.4 The spring example in full . . . 58

17 Deriving the Normal Equations . . . 60
17.1 The linear-algebra route . . . 60
17.2 The calculus route . . . 61
17.3 The normal equations in practice . . . 61

18 Least Squares in Python . . . 62
18.1 A complete linear fit . . . 63
18.2 Closed forms and `polyfit` . . . 64
18.3 Reading the output critically . . . 64

19 Nonlinear Least Squares . . . 66
19.1 Logarithmic linearisation . . . 66
19.2 Direct fitting with `curve_fit` . . . 67
19.3 Polynomial regression and overfitting . . . 67
19.4 Which method should you use? . . . 68

20 The Interpolation Problem . . . 70
20.1 Interpolation versus regression . . . 70
20.2 Existence and uniqueness . . . 70
20.3 Linear interpolation . . . 71
20.4 Error of linear interpolation . . . 72

21 Cubic Spline Interpolation . . . 74
21.1 Counting the unknowns . . . 74
21.2 The tridiagonal system . . . 75
21.3 Using SciPy . . . 75
21.4 Why splines beat high-degree polynomials . . . 76

22 Lagrange Polynomial Interpolation . . . 77
22.1 The basis polynomials . . . 77
22.2 The three-point example . . . 78
22.3 Using SciPy, and its limits . . . 78
23 Newton's Divided-Difference Form . . . 80
23.1 The Newton form . . . 80
23.2 Divided differences . . . 80
23.3 The three-point table . . . 81
23.4 Implementation . . . 81
23.5 Adding a point . . . 82
24 Comparing Interpolation Methods . . . 84
24.1 The comparison table . . . 84
24.2 The Runge phenomenon . . . 84
24.3 Choosing a method . . . 85
24.4 A worked comparison . . . 86
25 Formula Sheet . . . 88
26 Python API Cheat Sheet . . . 90
27 30 Exam-Style Questions . . . 93
28 The Last Ten Minutes . . . 97

PART

01

Foundations

Python 基础与算法复杂度

Tutorials 3–4. Control flow, functions, comprehensions, and the language of algorithmic cost.

01

1 Python Essentials: Control Flow and Functions

Python 基础：控制流、循环与函数

Tutorial 3 builds the small vocabulary that every later chapter assumes: decisions, repetition, containers produced in one line, and functions that return values. The exam rarely asks "what is a `for` loop?"; it asks you to read a short program and predict its output, or to write a function with a nested helper. Both require precise mental execution.

> **中文精讲** Chinese companion
> 这一章对应 Tutorial 3。考试不会孤立地问语法定义，而是给你一段 10–20 行的代码，问输出、问循环次数、问函数返回什么；或者要求你写一个带默认参数、嵌套函数、`lambda` 排序的小函数。因此本章的重点不是"背语法"，而是建立精确的执行模型：循环变量每一步取什么值、`break` 和 `continue` 分别跳到哪里、函数什么时候返回 `None`、闭包为什么能记住外层变量。

1.1 Conditional statements

条件语句：让程序分岔

Python executes statements from top to bottom unless a condition redirects the flow. A block is defined by a colon and indentation (conventionally four spaces). The three keywords form one mutually exclusive chain:

- `if`: evaluate the condition; if true, run the block.
- `elif`: test only when every previous condition was false.
- `else`: run when none of the preceding tests succeeded.

```
temp = -5
if temp <= 0:
    state = "solid"
elif temp < 100:
    state = "liquid"
else:
    state = "gas"
print(f"At {temp} C, water is a {state}.")
# At -5 C, water is a solid.
```

Conditions can be combined with `and`, `or` and `not`, and `if` statements can be nested. Python also treats many objects as truth values: `0`, `0.0`, `""`, `[]`, `{}` and `None` are false; almost everything else is true.

> Pitfall: **`elif`** is not the same as three separate **`if`**s
> With separate `if` statements, later tests run even after an earlier one matched, so `state` can be overwritten. With `elif`, exactly one branch of the chain executes. The exam loves this distinction because the code looks almost identical.

中文要点

- 冒号和缩进共同定义一个代码块；缩进不一致会直接报 `IndentationError`。
- `elif` 是“互斥的否则如果”：前面的条件一旦为真，后面的分支全部跳过。若写成三个独立的 `if`，`-5` 会先被判定为 solid，又因为 `-5 < 100` 被改写成 liquid，这就是典型陷阱。
- 空容器、空字符串、`None`、数字 0 在条件中都为假。判断“列表非空”可以直接写 `if lst:`。
- 嵌套 `if` 与用 `and` 合并条件在逻辑上常常等价，但嵌套结构能表达“先判断大类，再判断子类”的层次，可读性更好。

1.2 Looping: `for` and `while`

循环：遍历与条件驱动

A `for` loop iterates over a sequence. The loop variable is rebound to each element in turn; no manual index is needed. When the index matters, use `enumerate`, which yields `(index, value)` pairs.

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print(num, num**2)

total = 0
for i in range(1, 6):        # 1, 2, 3, 4, 5  (stop is excluded)
    total += i
print(total)                  # 15
```

`range(start, stop, step)` is a half-open sequence: it includes `start` but never `stop`. Two control keywords modify the flow: `break` leaves the loop immediately, while `continue` abandons the rest of the current iteration and jumps to the next one.

```
measurements = [5, 12, -1, 18, 999, 24]
for idx, val in enumerate(measurements):
    if val == -1:             # missing data
        continue
    if val == 999:            # error code
        break
    print(idx, val)
# prints (0,5), (1,12), (3,18); then stops at 999
```

A `while` loop repeats while a condition remains true. Use `for` when the number of iterations is known in advance, and `while` when the stopping point emerges from the computation (convergence, a threshold, user input). Every `while` loop must make progress towards falsifying its condition, or contain a `break`; otherwise it runs forever.

```
mass, half_life, years = 80.0, 5.0, 0.0
while mass >= 10.0:
    mass /= 2.0
    years += half_life
print(years, mass)            # 20.0 5.0
```

中文要点

- `range(1, 6)` 生成 1–5，右端不包含；`range(5)` 等价于 `range(0, 5)`；步长可以为负，例如 `range(5, 0, -1)`。
- `enumerate(seq)` 同时给出下标和元素，比 `for i in range(len(seq))` 更 Pythonic，也不容易越界。

- `break` 是“整个循环到此为止”；`continue` 是“这一次到此为止，进入下一次”。上面例子中 999 之后的 24 永远不会被处理。
- `while` 的三要素：初值、条件、更新。若忘记更新（例如注释掉 `years += half_life`），条件永远为真，程序会卡死。写数值迭代时，除了收敛条件，最好再加一个最大迭代次数作为保险。

1.3 Comprehensions

推导式：一行生成容器

A list comprehension packs a loop and an optional filter into one expression:

`[expression for item in iterable if condition].`

A dictionary comprehension replaces the expression with a `key: value` pair. The result is a new object; the original container is unchanged.

```
inventory = {"apple": 0.90, "bread": 1.00, "avocado": 2.50,
             "banana": 0.60, "almond milk": 3.40}

costly = [k for k in inventory if inventory[k] > 2]
discounted = {k: round(v * 0.8, 2) for k, v in inventory.items()}
print(costly)       # ['avocado', 'almond milk']
print(discounted)   # {'apple': 0.72, ...}
```

中文要点

推导式适合“对每个元素做同样的变换”或“筛掉不满足条件的元素”。它把三行循环压缩成一行，但不要为了短而嵌套三层——那时可读性比行数重要。字典推导式遍历 `inventory` 默认得到键；若要同时拿到键和值，用 `inventory.items()`。

1.4 Functions, scope and `lambda`

函数、作用域与匿名函数

A function packages a computation behind a name. It is defined with `def`, receives parameters, and sends a result back with `return`. The distinction between `return` and `print` is fundamental: `print` only displays text, whereas `return` hands the value back to the caller so it can be stored, tested or passed onward. A function without an explicit `return` returns `None`.

```
def kinetic_energy(mass, vel):
    """Return KE = 0.5*m*v^2 in joules."""
    return 0.5 * mass * vel**2

def projectile_range(v0, angle_deg, g=9.81):
    """Horizontal range; g has a default value."""
    theta = math.radians(2 * angle_deg)
    return round(v0**2 * math.sin(theta) / g, 2)

energy = kinetic_energy(2.0, 3.0)      # 9.0
r_earth = projectile_range(20.0, 45.0) # 40.77
r_mars = projectile_range(20.0, 45.0, g=3.71)  # 107.82
```

Default parameters make a function usable in the common case while still allowing override. Two scope rules matter: a nested function can read variables from the enclosing function (a closure), and an inner helper is invisible outside its parent unless it is returned.

```python
def make_power_function(power):
    def power_func(x):
        return x**power       # 'power' is remembered by the closure
    return power_func

square, cube = make_power_function(2), make_power_function(3)
print(square(5), cube(3))     # 25 27
```

A `lambda` is a one-expression anonymous function: `lambda arguments: expression`. It is convenient as a sorting key or a short callback, but it cannot contain statements, loops or multiple expressions. If the logic needs a name or more than one line, use `def`.

```python
students = [("Alice", 88), ("Bob", 92), ("Charlie", 78), ("Diana", 95)]
by_score = sorted(students, key=lambda item: item[1])
# [('Charlie', 78), ('Alice', 88), ('Bob', 92), ('Diana', 95)]
```

> Pitfall: mutable default arguments
> `def f(x, acc=[])` creates *one* list when the function is defined, and that same list is reused on every call. Use `acc=None` and create a fresh list inside the function. This is a classic written-exam question because the first call looks correct.

> **中文要点**
>
> - `return` 把结果交还给调用者，`print` 只负责显示。函数里只写 `print` 不写 `return`，调用结果是 `None`。
> - 默认参数在“函数定义时”求值一次，而不是每次调用时重新求值。因此可变默认值（列表、字典）会被所有调用共享。
> - 嵌套函数可以直接读取外层的变量（闭包）；外层函数返回内层函数时，内层函数仍然记得当时的变量，这就是 `make_power_function` 能制造“平方器/立方器”的原因。
> - `lambda` 只能写一个表达式，不能写赋值或循环。它最常见的考法是 `sorted(..., key=lambda ...)`：`key` 返回什么，就按什么排序。
> - 文档字符串 `"""..."""` 写在 `def` 下一行，用 `help(func)` 可以查看；这是代码质量的加分项，不是运行时必需。

1.5 Worked tutorial exercises

Tutorial 3 习题完整解答

1. Grade and honours

```python
score, attendance = 83, 92

if score >= 85:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "F"

honours = (grade == "A") and (attendance > 90)
print(f"Grade: {grade}; Honours: {'Yes' if honours else 'No'}")
# Grade: B; Honours: No
```

The order of the tests matters: because the first matching branch wins, each `elif` only needs its lower bound. Writing `if 75 <= score < 85` is also correct, but the chain above avoids repeating the upper bound.

2. Run times

```
times = [10.2, 14.5, 9.8, 12.0, 15.6, 8.4, 11.1]
total = 0
for t in times:
    total += t
average = total / len(times)          # 11.657...

for idx, t in enumerate(times):
    if t > 12.0:
        print(idx, t)                 # 1 14.5 ; 4 15.6

count = 0
for t in times:
    if t < 10.0:
        continue                      # skip short runs
    count += 1
print(total, round(average, 2), count)  # 81.6 11.66 5
```

3. Collatz sequence

```
n, steps = 26, 0
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    steps += 1
    print(n)
print("steps =", steps)     # sequence ends 1; steps = 10
```

Use integer division `//` so that `n` stays an integer; `n / 2` would produce a float and make the parity test unreliable for large values. The sequence for 26 has 10 steps: $26 \to 13 \to 40 \to 20 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$.

4. Particle-detector pipeline

```
import math

raw_events = [("p1", 12.4, True), ("p2", 8.1, False),
              ("p3", 25.6, True), ("p4", 4.3, True),
              ("p5", 19.8, False), ("p6", 15.0, True)]

def process_detector_data(events, threshold=10.0):
    def estimate_momentum(e):
        return math.sqrt(e[1]**2 - 0.5**2)   # m = 0.5 GeV

    valid = [e for e in events if e[2]]
    out = []
    for pid, energy, _ in valid:
        out.append({
            "id": pid,
            "energy": energy,
            "momentum": round(estimate_momentum((pid, energy, True)), 4),
            "category": "high-energy" if energy > threshold else "low-energy"
```

```
        })
    return out

processed = process_detector_data(raw_events)
ranked = sorted(processed, key=lambda d: d["momentum"], reverse=True)
for d in ranked:
    print(d)
```

The output is ordered by momentum: `p3` (25.60 GeV), `p6` (15.00), `p1` (12.40), `p4` (4.30, low-energy). Invalid events `p2` and `p5` never reach the momentum calculation. This exercise combines every Tutorial 3 idea at once: a default parameter, a nested helper, a comprehension with a filter, a loop, a conditional expression, and a `lambda` key.

> What to rehearse
>
> Be able to (i) trace a loop with `break`/`continue`, (ii) rewrite a loop as a comprehension and vice versa, (iii) explain `return` versus `print`, (iv) write a nested function that captures an outer variable, and (v) sort a list of dictionaries by one field with `lambda`. These five skills reappear in every later chapter.

本章考点清单

1. 给出带 `break`/`continue` 的循环，写出完整输出（注意被跳过的元素）。
2. 把 3–5 行的 `for` 循环改写成列表/字典推导式。
3. 说明为什么 `return` 和 `print` 不等价；函数缺少 `return` 时返回 `None`。
4. 解释闭包：内层函数为什么在外层函数结束后仍能访问 `power`。
5. 用 `lambda` 作为 `sorted` 的 `key`，并说明 `reverse=True` 的作用。
6. 可变默认参数陷阱：为什么 `def f(acc=[])` 是危险的。

2 Algorithmic Complexity and Big-O Notation

算法复杂度与 Big-O 记号

Tutorial 4 answers a practical question: if the input doubles, how much slower does the program become? A stopwatch is not a portable answer, because it depends on the machine, the language and the particular data. Complexity analysis replaces the stopwatch with a count: how does the number of basic operations grow as a function of the input size n?

> **中文精讲** Chinese companion
> 这一章只关心一个问题的“增长趋势”，不关心某台电脑上具体跑几秒。基本操作（加法、赋值、比较、索引）视为常数时间；输入规模 n 可以是列表长度、粒子数、数据点数。Big-O 描述 $n \to \infty$ 时的上界（最坏情况），写作 $O(f(n))$。考试常见题型：给一段循环代码判断复杂度；比较 $O(n)$、$O(n \log n)$、$O(n^2)$ 在 n 很大时的快慢；解释为什么二分查找是 $O(\log n)$。

2.1 The definition, without the epsilons

定义：只看主导项

We write $T(n) = O(f(n))$ if there exist constants $C > 0$ and n_0 such that

$$T(n) \le C\, f(n) \qquad \text{for all } n \ge n_0.$$

In practice this means: drop constant factors and lower-order terms, and keep the fastest-growing term. If an algorithm performs $3n^2 + 40n + 7$ basic operations, it is $O(n^2)$; the 3, the $40n$ and the 7 are irrelevant asymptotically. Big-O is an upper bound, so $O(n)$ is also a valid (if loose) description of an $O(1)$ algorithm; in this course we always quote the tight, conventional bound.

> Three rules that answer most exam questions
>
> 1. **Sequential code adds**: two consecutive loops of n each cost $n + n = O(n)$.
> 2. **Nested loops multiply**: a loop of n inside a loop of n costs $n \cdot n = O(n^2)$.
> 3. **Halving the problem gives a logarithm**: if each step removes a constant fraction of the remaining work, the count is $O(\log n)$.

> **中文要点**
> Big-O 是“上界 + 忽略常数 + 忽略低阶项”。例如 $5n^2 + 100n$ 记为 $O(n^2)$，不是 $O(5n^2)$。顺序结构相加，嵌套结构相乘，每次把问题规模减半则出现 $\log n$。注意：$O(1)$ 表示操作次数与 n 无关，不代表“很快”；一个常数极大的 $O(1)$ 仍可能比小 n 的 $O(n)$ 慢——Big-O 描述的是趋势，不是秒表。

2.2 The standard orders

常见增长阶

Order	Name	Typical source
$O(1)$	constant	indexing `a[i]`, arithmetic, dictionary lookup (average)
$O(\log n)$	logarithmic	binary search, bisection root finding, halving a range
$O(n)$	linear	one pass over a list, `sum`, `min`, `max`, linear search
$O(n \log n)$	linearithmic	merge sort, Timsort (`sorted`), FFT
$O(n^2)$	quadratic	nested loops, all pairwise interactions, bubble/insertion sort
$O(2^n)$	exponential	exhaustive subsets, naive recursive Fibonacci

Table 2.1 Common complexity classes, from fastest to slowest growth.

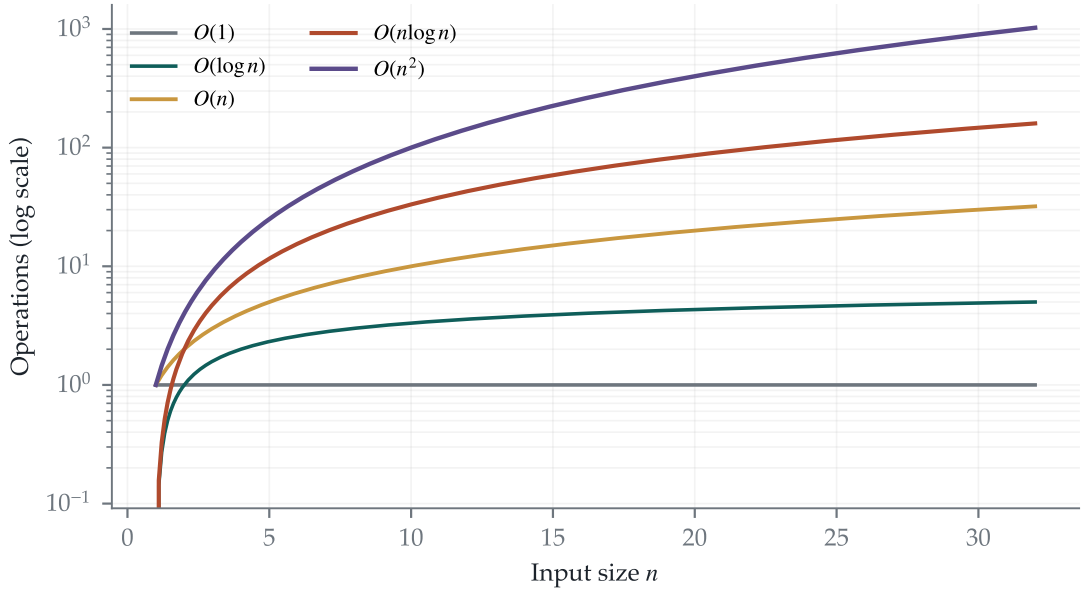


Figure 2.1 Growth of the standard complexity classes.
图 2.1 常见复杂度随输入规模 n 的增长；纵轴为对数坐标。

2.3 Reading code for complexity

从代码读出复杂度

Constant: $O(1)$. The number of operations does not depend on n. Accessing an element by index, arithmetic, and an average-case dictionary lookup are constant. A loop with a fixed number of iterations (say, exactly 10) is also $O(1)$, because 10 does not grow with n.

Linear: $O(n)$. One loop over the data, or a fixed number of separate loops, is linear. Finding a maximum is the canonical example: update a candidate once per element.

```
def find_max(numbers):
    current = numbers[0]
    for val in numbers:       # exactly n iterations
        if val > current:
            current = val
    return current            # O(n)
```

Quadratic: $O(n^2)$. Two nested loops that each range over the data. The physics example is the N-body problem: for n particles there are $n(n-1)/2 \sim \frac{1}{2}n^2$ unique pairs, and computing every pairwise force is quadratic. Note that the factor $\frac{1}{2}$ disappears in Big-O.

```
def count_pairs(particles):
    n, interactions = len(particles), 0
    for i in range(n):
        for j in range(i + 1, n):   # n(n-1)/2 iterations
            interactions += 1
    return interactions              # O(n^2)
```

Logarithmic: $O(\log n)$. Each step discards a constant fraction of the remaining candidates. Binary search halves a sorted interval; the bisection method halves a root bracket. In computer science, $\log$ without a base means $\log_2$, because the base only changes the constant factor.

```
def binary_search(a, target):
    low, high = 0, len(a) - 1
    while low <= high:            # at most log2(n) + 1 iterations
        mid = (low + high) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Linearithmic: $O(n \log n)$. Merge sort divides the list into halves ($\log n$ levels) and merges all n elements at each level, giving $n \log n$. Python's built-in `sorted` uses Timsort, also $O(n \log n)$.

```
def merge_sort(data):
    if len(data) <= 1:
        return data
    mid = len(data) // 2
    left = merge_sort(data[:mid])
    right = merge_sort(data[mid:])
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result
```

中文判读方法

- 看到"一层循环遍历 n 个元素"→ $O(n)$。
- 看到"两层循环，内层从 $i+1$ 到 n"→ 总次数 $n(n-1)/2$，仍是 $O(n^2)$，不要因为少了一半就写成 $O(n^2/2)$。
- 看到"每次 `low/high` 更新把区间减半"→ $O(\log n)$。10^6 个元素只需约 20 次比较。
- 看到递归"分成两半，再合并 n 个元素"→ $O(n \log n)$。
- 相邻的两段循环要相加而不是相乘；只有嵌套才相乘。
- 若循环次数是常数（如固定 100 次），即使里面做很多事，仍是 $O(1)$。

2.4 Empirical timing and its limits

实测时间：只能作为佐证

The tutorial measures `sum(list)` against a double loop for $n = 500, 1000, 2000$. Linear time stays at a few microseconds, while quadratic time grows roughly fourfold when n doubles (0.013 s → 0.033 s → 0.101 s). This is consistent with $O(n^2)$, but timing alone cannot prove a complexity class: caches, CPU frequency, Python overhead and the particular data all matter.

```
import time
for n in [500, 1000, 2000]:
    data = list(range(n))
    t0 = time.perf_counter()
    total = sum(data)                # O(n)
    t_lin = time.perf_counter() - t0
    t0 = time.perf_counter()
    count = 0
    for i in data:
        for j in data:
            count += 1               # O(n^2)
    t_quad = time.perf_counter() - t0
    print(n, t_lin, t_quad)
```

Pitfalls in complexity questions

- $2n + 5$ is $O(n)$, not $O(2n)$; constants are discarded.
- $O(n \log n)$ is not "almost n^2": at $n = 10^6$ it is about 2×10^7 operations versus 10^{12}.
- A single pass that calls a constant-time helper is still linear; a pass that calls a linear helper is quadratic.
- `in` on a list is $O(n)$; `in` on a set or dictionary is $O(1)$ on average. Choosing the right container changes the complexity.
- Best case, worst case and average case can differ. Binary search is $O(\log n)$ worst case; linear search is $O(1)$ best case but $O(n)$ worst case.

What to rehearse

Given a short function, identify the dominant term and justify it in one sentence. Be ready to compare $n = 10^6$ costs: $\log_2 10^6 \approx 20$, $n \log_2 n \approx 2 \times 10^7$, $n^2 = 10^{12}$. Also remember that NumPy vectorisation does not change the mathematical complexity, but it changes the constant factor by orders of magnitude because the loop runs in compiled C rather than in Python.

本章考点清单

1. 写出 Big-O 的定义，并解释"忽略常数与低阶项"。
2. 给一段嵌套循环，数出总迭代次数（如 $n(n-1)/2$）并化简。
3. 说明二分查找为什么是 $O(\log n)$，并算出 $n = 10^6$ 时的步数。
4. 说明归并排序的 $\log n$ 层与每层 n 次合并如何得到 $n \log n$。
5. 比较列表的 `in` 与集合的 `in` 的复杂度。
6. 解释为什么实测时间不能单独证明复杂度类别。

PART

02

Numbers

数字、浮点与误差

Chapter 9. How a finite machine represents the infinite real line, and what that costs.

3 Number Representation and Binary Arithmetic

数字表示与二进制算术

Chapter 9 begins with a deceptively simple fact: the number 147.3 that we write without thinking is already a polynomial in powers of ten. Once we see it that way, changing the base is only a change of coefficients. Computers use base 2 because a physical device can reliably distinguish two states, not ten.

> **中文精讲** Chinese companion
>
> 第 9 章从“我们熟悉的十进制其实是一种幂级数展开”出发，推广到任意 N 进制，再落到计算机使用的二进制。核心公式只有一个：
>
> $$(d_k d_{k-1}\cdots d_1 d_0 . f_1 f_2 \cdots)_N = \sum_{i=0}^{k} d_i N^i + \sum_{j\ge 1} f_j N^{-j}.$$
>
> 把 N 换成 10、3、2，得到的就是十进制、三进制、二进制。考试常考：进制转换、二进制加减乘、为什么计算机用二进制、定长存储的限制。

3.1 Decimal and base-N systems

十进制与任意进制

In base 10, the string 147.3 means

$$1\cdot 10^2 + 4\cdot 10^1 + 7\cdot 10^0 + 3\cdot 10^{-1} = 147.3.$$

Each digit is a coefficient between 0 and 9. In a base-N system there are exactly N allowed digits, $0, 1, \ldots, N-1$, and the same expansion holds with 10 replaced by N. For example,

$$121_3 = 1\cdot 3^2 + 2\cdot 3^1 + 1\cdot 3^0 = 9 + 6 + 1 = 16_{10}.$$

The subscript is essential when the base is not clear from context; 121 in base 3 and 121 in base 10 are different numbers.

	N^3	N^2	N^1	N^0
Base 10	1000	100	10	1
Base 2	8	4	2	1
Base 3	27	9	3	1

Converting an integer to base N. Repeatedly divide by N and record the remainders; the remainders read bottom-to-top give the digits. For 37 in base 2:

$$37 = 2\cdot 18 + 1,\quad 18 = 2\cdot 9 + 0,\quad 9 = 2\cdot 4 + 1,\quad 4 = 2\cdot 2 + 0,\quad 2 = 2\cdot 1 + 0,\quad 1 = 2\cdot 0 + 1,$$

so $37_{10} = 100101_2$. Checking the expansion recovers $32 + 4 + 1 = 37$.

中文要点

进制展开的“位权”从左到右依次是 $N^k, N^{k-1}, \ldots, N^0, N^{-1}, \ldots$。整数转 N 进制用“除 N 取余，倒序读”；小数部分则用“乘 N 取整，顺序读”。例如 $0.625 \times 2 = 1.25$ 取 1，$0.25 \times 2 = 0.5$ 取 0，$0.5 \times 2 = 1.0$ 取 1，所以 $0.625_{10} = 0.101_2$。注意：不是所有十进制小数都能用有限位二进制表示，这正是下一章浮点误差的根源。

3.2 Binary numbers

二进制：只含 0 和 1

Binary uses digits 0 and 1; each digit is a **bit**. The place values are $1, 2, 4, 8, 16, 32, \ldots$. Two conversions worth memorising:

$$11_{10} = 1011_2, \qquad 37_{10} = 100101_2.$$

A binary string of length b can represent 2^b distinct patterns. If the patterns are used for non-negative integers, the range is 0 to $2^b - 1$. A 32-bit integer therefore tops out at $2^{32} - 1 = 4\,294\,967\,295$.

Binary arithmetic. Addition and multiplication follow the same column rules as decimal, only the carry threshold is 2 instead of 10:

$$100101_2 + 10001_2 = 110110_2 = 54_{10},$$

$$100101_2 \times 10001_2 = 1001110101_2 = 629_{10}.$$

The second result can be checked by distributivity:

$$37 \times 17 = 37 \times (16 + 1) = 592 + 37 = 629.$$

中文要点

二进制加法：$1 + 1 = 10$（写 0 进 1），$1 + 1 + 1 = 11$（写 1 进 1）。乘法就是“移位相加”：乘以 2^k 相当于左移 k 位。上面 $37 \times 17 = 37 \times 16 + 37$，在二进制里就是 100101 左移 4 位再加自身。考试若要求写出竖式，记得对齐最低位。

3.3 Why binary, and what it costs

为什么用二进制，以及代价

Binary is convenient for hardware because logic gates naturally implement Boolean operations on two-valued signals: AND, OR, NOT, XOR. Addition, comparison and multiplication can be assembled from these gates, and the resulting circuits are fast and robust against noise. The price is that binary is verbose for humans (one decimal digit needs about 3.32 bits), and that numbers are stored in *fixed-width* words. A 32-bit register cannot hold 2^{32} or a negative number unless a signed representation is chosen; overflow silently wraps or raises, depending on the language.

Bit-width arithmetic

With b bits: 2^b patterns, unsigned range $0 \ldots 2^b - 1$. A signed two's-complement integer of b bits ranges from -2^{b-1} to $2^{b-1} - 1$. For $b = 32$: unsigned up to 4.29×10^9, signed from -2.147×10^9 to 2.147×10^9.

中文要点

计算机用二进制，不是因为它“短”，而是因为它“稳”：两种电平状态容易区分，逻辑门可以直接实现布尔运算。代价是 humans 读起来很长，而且寄存器位宽固定。Python 的 `int` 是任意精度、自动扩展的，所以你在 Python 里看不到 32 位溢出；但 C、Java 等语言的 `int` 会溢出，这正是课件强调“fixed bit lengths restrict the range”的原因。Python 的 `float` 则是固定 64 位的 IEEE 754 双精度，下一章详述。

What to rehearse

Convert 37, 17, 54, 629 to binary and back. Explain the base-3 example. State the unsigned range of a b-bit word and the number of patterns. Give two reasons for using binary in hardware and one limitation of fixed-width storage.

本章考点清单

1. 用位权展开计算 $(121)_3$、$(100101)_2$。
2. 用“除 2 取余”把十进制整数转成二进制，并用展开验算。
3. 完成二进制加法和乘法，再转回十进制核对。
4. 说明 b 位能表示多少个不同模式、无符号范围是多少。
5. 解释二进制对硬件的两个优点和一个缺点。

4 IEEE 754 Floating Point

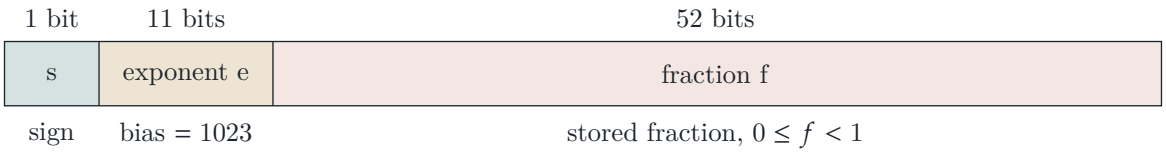
IEEE 754 浮点数：符号、指数与尾数

A floating-point number is scientific notation in base 2, with a fixed number of bits. The idea is to keep a wide dynamic range without keeping a wide mantissa: the exponent moves the binary point, while the fraction stores the significant digits. Python's `float` is a 64-bit IEEE 754 *double precision* value, and NumPy's default `float64` is the same type.

> **中文精讲** Chinese companion
> 浮点数 = 二进制科学计数法 + 固定位宽。它用"符号位 + 指数 + 尾数"三部分，把很大和很小的数都塞进 64 位里。理解这一章的关键是把三个符号分清：
>
> $$n = (-1)^s \times 2^{e-1023} \times (1 + f).$$
>
> 其中 s 是符号位（0 正 1 负），e 是 11 位指数域的整数值（不是真实指数），f 是 52 位尾数域所表示的小数（$0 \le f < 1$）。真实指数是 $e - 1023$，因为偏置（bias）是 1023。考试常考：给一串 64 位模式求十进制值；反过来给一个数写位模式；解释 ϵ、间距、次正规数、inf 与 NaN。

4.1 The 64-bit layout

64 位的三段式布局



For a **normal** number ($1 \le e \le 2046$):

$$\boxed{n = (-1)^s \times 2^{e-1023} \times (1 + f)}$$

The leading 1 is implicit and is not stored, which buys one extra bit of precision. For a **subnormal** number ($e = 0$, $f \neq 0$):

$$n = (-1)^s \times 2^{-1022} \times (0 + f).$$

The exponent field $e = 2047$ is reserved: if $f = 0$ the value is $\pm\infty$; if $f \neq 0$ it is **NaN** (not a number).

> **中文要点：三个符号别混**
>
> - 指数域 e 是一个 11 位无符号整数（0–2047），真实指数要减去 bias 1023。例如 $e = 1026$ 表示 2^3，不是 2^{1026}。
> - 尾数域 f 表示二进制小数 $0.f_1 f_2 \ldots f_{52}$。若存储位是 `1000...0`，则 $f = 0.5$，有效数字是 $1 + f = 1.5$。
> - 规格化数的有效数字总在 $[1, 2)$ 之间；次正规数则把隐含的 1 换成 0，用来表示比最小规格化数还小的值。
> - $e = 0, f = 0$ 是 ± 0；$e = 2047$ 是特殊值。不要用 $e = 2047$ 去做普通指数计算。

4.2 A worked decode

例题：把位模式翻译成十进制

Consider the 64-bit pattern

$$\underbrace{1}_{s}\ \underbrace{10000000010}_{e}\ \underbrace{1000\,0000\,\ldots\,0000}_{f}.$$

The fields give

$$s = 1, \qquad e = (10000000010)_2 = 1026, \qquad f = (0.1)_2 = 0.5.$$

Therefore

$$n = (-1)^1 \times 2^{1026-1023} \times (1 + 0.5) = -1 \times 8 \times 1.5 = -12.0.$$

The pattern is exactly the double-precision encoding of -12.0. Note the common slip: the slide writes "fraction $f = 1.5$"; 1.5 is the *significand* $1 + f$, while the stored fraction field is 0.5.

> **中文要点**
> 解题顺序永远是：拆三段 → 算 e 的十进制 → 算真实指数 $e - 1023$ → 把 f 的二进制位当成 $0.f$ → 代入公式。符号位单独处理，不要把它乘进指数。若题目给的是十六进制（如 `0x1.999999999999ap-4`），那是 Python 的 `float.hex()` 表示法：尾数 $1.999\ldots$、指数 -4，对应的就是 0.1。

4.3 Precision, spacing and machine epsilon

精度、间距与机器 epsilon

A double has 52 stored fraction bits plus the implicit leading 1, so about 53 significant binary bits, which corresponds to 15–17 significant decimal digits. The **machine epsilon** ε is the gap between 1 and the next representable double:

$$\varepsilon = 2^{-52} \approx 2.220446049250313 \times 10^{-16}.$$

The gap is not constant: it scales with the magnitude. Between x and the next float the spacing is roughly $|x|\,\varepsilon$. NumPy exposes this as `np.spacing(x)`.

x	`np.spacing(x)`
1	2.22×10^{-16}
10^3	1.14×10^{-13}
10^6	1.16×10^{-10}
10^{10}	1.91×10^{-6}
10^{16}	2.0

At 10^{16}, consecutive doubles are 2 apart, so adding 1 to 10^{16} changes nothing. This is not a bug; it is the meaning of finite precision.

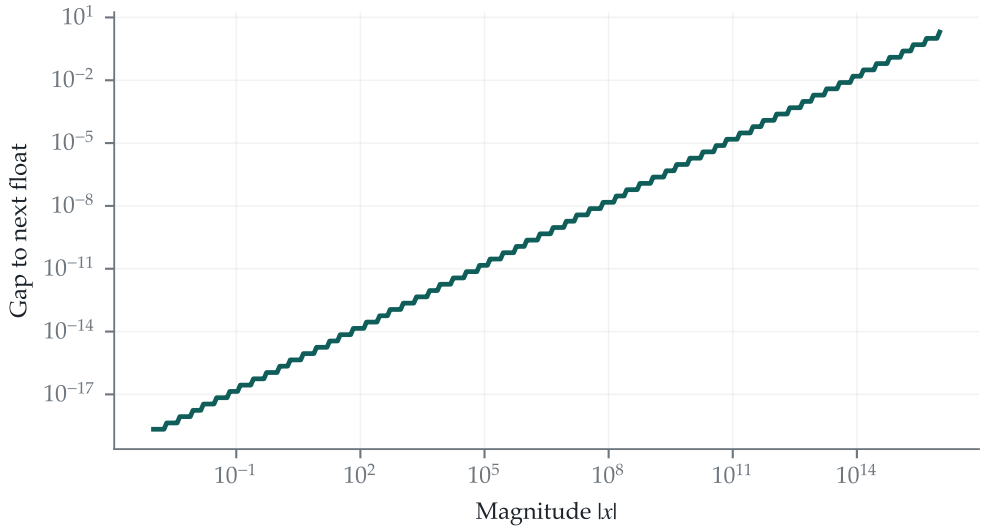


Figure 4.1 The gap between adjacent doubles grows linearly with magnitude on a log–log plot.
图 4.1 相邻浮点数的间距随数量级增大；双对数坐标下近似为直线。

中文要点

- 精度不是“小数点后 16 位”，而是“约 16 位有效数字”。10^{-20} 这样小的数仍然可以有 16 位有效数字，只是绝对间距很小。
- 机器 epsilon $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$ 是 1 附近的间距；在 10^{6} 附近间距约为 $10^{6}\varepsilon \approx 1.16 \times 10^{-10}$。
- $2^{53} = 9\,007\,199\,254\,740\,992 \approx 9.0 \times 10^{15}$ 以内的整数可以精确表示；超过后并非所有整数都能存下。
- 比较两个浮点数不要写 `a == b`，应写 `abs(a-b) < tol` 或 `np.isclose(a,b)`。

4.4 Special values, overflow and underflow

特殊值、上溢与下溢

Exponent	Fraction	Meaning	Example
0	0	signed zero	0.0, −0.0
0	nonzero	subnormal	5×10^{-324}
1 . . . 2046	any	normal number	1.0, −12.0
2047	0	±∞	`float('inf')`
2047	nonzero	NaN	`float('nan')`

Table 4.1 The five cases of the IEEE 754 double format.

The largest finite double is about 1.7977×10^{308}; the smallest positive normal is 2.225×10^{-308}; the smallest positive subnormal is 4.94×10^{-324}. When a result exceeds the largest finite value, **overflow** occurs; IEEE arithmetic returns ±∞ (NumPy warns, Python's `math.exp` may raise `OverflowError`). When a result is smaller than the smallest subnormal, it **underflows** to zero. Subnormals fill the gap between zero and the smallest normal number, at the cost of losing leading significant bits.

```
import numpy as np
np.finfo(float).eps      # 2.220446049250313e-16
np.finfo(float).max      # 1.7976931348623157e+308
np.finfo(float).tiny     # 2.2250738585072014e-308
np.nextafter(0, 1)       # 5e-324  (smallest subnormal)
np.spacing(1.0)          # 2.220446049250313e-16
```

```
np.array([1e308]) * 10   # inf  (with a runtime warning)
np.array([1e-320]) / 1e10  # 0.0 (underflow)
```

Do not test NaN with ==

By IEEE definition, every comparison with NaN is false, including `nan == nan`. Use `np.isnan(x)` or `math.isnan(x)`. Likewise, `inf - inf` and `0 * inf` produce NaN, not zero.

中文要点

- 上溢：结果超过约 1.8×10^{308}，得到 `inf`；继续运算可能得到 `inf`、`-inf` 或 `nan`。
- 下溢：结果小于约 5×10^{-324}，变成 0；介于最小正规数与最小次正规数之间的值会损失有效位。
- `inf - inf`、`0*inf`、`inf/inf` 都是 `nan`；`nan` 不等于任何值，包括它自己。
- 次正规数（subnormal）的指数域为 0，隐含位为 0，用来平滑地逼近 0；它们比正规数慢，但在物理计算中很少成为瓶颈。
- Python 的 `float('inf')`、`float('nan')` 可以直接构造；NumPy 还有 `np.inf`、`np.nan`。

What to rehearse

Decode a given 64-bit pattern (sign, biased exponent, fraction) into a decimal value. State the bias, the number of bits in each field, and the formula for normal and subnormal numbers. Explain why the spacing grows with $|x|$ and why $10^{16} + 1 = 10^{16}$ in double precision. List the bit patterns for zero, infinity and NaN.

本章考点清单

1. 画出 double 的 1+11+52 位布局，写出 bias 和三个字段的含义。
2. 用公式解码 −12.0 的位模式，注意 $f = 0.5$ 与 $1 + f = 1.5$ 的区别。
3. 写出 ε、最大有限数、最小正规数、最小次正规数的量级。
4. 解释 `np.spacing(1e16) == 2.0` 的含义。
5. 列出指数域为 0 和 2047 时的四种特殊情况。
6. 区分 overflow、underflow、subnormal 三个概念。

5 Round-Off Error

舍入误差：计算机算错的那一点点

Round-off error is the distance between a real number and the floating-point value actually stored. It is unavoidable: most decimal fractions have infinitely repeating binary expansions, and every arithmetic operation must round its exact result back into the 53-bit grid. The danger is not the size of one error but what happens when errors are amplified, accumulated, or hidden by cancellation.

中文精讲 Chinese companion

舍入误差来自“存不下”：十进制里的 0.1、π、1/3 在二进制中都是无限循环小数，只能截断到 52 位尾数。单次误差通常在 10^{-16} 量级，看起来无害；但在两种情况下会变成灾难：

1. **相减抵消**（catastrophic cancellation）：两个几乎相等的数相减，有效数字大量丢失，相对误差被放大。
2. **反复累加**：误差随运算次数累积，甚至被算法放大。

考试常考：判断 `0.1+0.2 == 0.3` 为什么是 `False`；解释 `4.9-4.845` 的输出；说明如何比较浮点数；区分 round-off error 与 truncation error。

5.1 Representation error

表示误差：从一开始就不精确

The decimal 0.1 is the infinite binary fraction $0.0001100110011\ldots_2$. Stored as a double, it becomes

$$0.1 \longrightarrow 0.1000000000000000055511151231257827\ldots$$

The stored value is the nearest double, not the real $1/10$. The same happens to π, $1/3$, and almost every decimal you type. This is **representation error**: the number is already wrong before any arithmetic begins.

```
>>> 0.1
0.1                      # Python displays the shortest round-trip form
>>> format(0.1, '.20f')
'0.10000000000000000555'
>>> 0.1 == 0.1000000000000000055511151231257827
True
```

中文要点

Python 的 `repr` 很聪明：它显示能唯一还原该浮点数的最短十进制串，所以 `0.1` 看起来就是 `0.1`。但用 `format(0.1, '.20f')` 把全部位数打印出来，就能看到真实存储值是 $0.10000000000000000555\ldots$。这解释了为什么“看起来相等”的两个数可能不相等。

5.2 Cancellation and the loss of significance

相减抵消：有效数字的塌陷

Subtracting nearly equal numbers is the most common way a small absolute error becomes a large relative error. In the lecture example,

$$4.9 - 4.845 = 0.055000000000000604 \neq 0.055.$$

Both operands are stored with an error of order 10^{-16}; their difference is only 0.055, so the relative error is amplified by roughly $1/0.055 \approx 18$. If the difference were 10^{-10}, the relative error would be amplified by 10^{10}.

```
>>> 4.9 - 4.845
0.055000000000000604
>>> 0.1 + 0.2 + 0.3
0.6000000000000001
>>> 0.1 + 0.2 + 0.3 == 0.6
False
>>> (0.1 + 0.2 + 0.3) - 0.6
1.1102230246251565e-16
```

The same mechanism makes the quadratic formula unreliable when $b^2 \gg 4ac$: the root $(-b +$

$\sqrt{b^2-4ac})/(2a)$ subtracts two nearly equal numbers. A stable rewrite uses

$$q = -\tfrac{1}{2}\left(b + \operatorname{sign}(b)\sqrt{b^2-4ac}\right), \qquad x_1 = \frac{q}{a}, \qquad x_2 = \frac{c}{q},$$

which avoids the cancellation.

> **中文要点**
> 相减本身没有错，错在“有效数字不够用”。4.9 和 4.845 各有约 16 位有效数字，相减后只剩约 14 位，而结果的首位是 5×10^{-2}，于是绝对误差 10^{-16} 对应的相对误差变成 10^{-14} 量级。判断是否会发生灾难性抵消，看两个操作数是否“几乎相等”；如果是，就改写公式（有理化、配凑、用稳定公式），而不是硬算。

5.3 Accumulation and repeated rounding

误差累积与重复舍入

One rounding is harmless; a million roundings may not be. Two distinct effects appear in the slides.

Accumulation. Add $1/3$ one hundred times, then subtract it one hundred times. Mathematically the result is exactly 1; numerically it is

$$1.0000000000000002.$$

Each addition rounds, and the positive errors do not cancel the negative ones exactly. The error grows roughly like $\sqrt{n}$ for random rounding, or like n in the worst case.

Repeated rounding. Rounding twice is not the same as rounding once. In decimal, $4.845 \to 4.85 \to 4.9$, while rounding directly to one decimal gives 4.8. Each rounding discards information that cannot be recovered. In Python, `round(4.845, 2)` returns `4.84`, because the stored value is slightly below 4.845 and Python uses round-half-to-even.

```python
# accumulation: add 1/3 a hundred times, then subtract it a hundred times
s = 1.0
for _ in range(100):
    s += 1/3
for _ in range(100):
    s -= 1/3
print(repr(s))          # 1.0000000000000002

print(round(4.845, 2))  # 4.84  (binary value is slightly below 4.845)
print(round(4.845, 1))  # 4.8
```

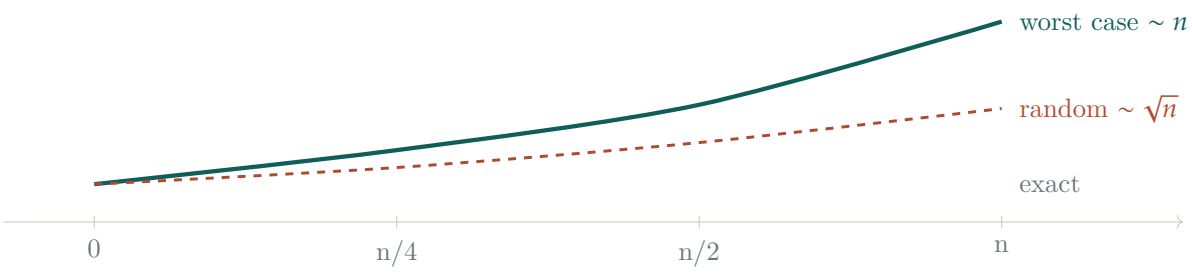


Figure 5.1 Schematic growth of accumulated round-off.
图 5.1 累积舍入误差的增长示意；随机舍入通常按 $\sqrt{n}$ 增长，最坏情况可接近 n。

中文要点

- 连续加 10^6 个小数，朴素求和的误差上界与 $n\varepsilon\sum|x_i|$ 同阶；NumPy 的 `np.sum` 使用成对求和（pairwise summation），误差更小。
- Kahan 补偿求和用一个额外的补偿变量记录“被舍掉的低位”，把误差从 $O(n\varepsilon)$ 降到 $O(\varepsilon)$，是数值计算中的经典技巧。
- 重复舍入不可逆：$4.845 \to 4.85 \to 4.9$ 与直接 $4.845 \to 4.8$ 不同。金融计算应使用 `decimal.Decimal` 并明确舍入规则。
- Python 的 `round` 采用 banker's rounding（四舍六入五取偶），且受二进制表示影响，不要用它来做精确的十进制四舍五入。

5.4 How to live with round-off error

与误差共处：实用策略

1. **Never compare floats exactly.** Use `abs(a-b) <= atol + rtol*abs(b)` or `np.isclose`, with a tolerance appropriate to the problem.
2. **Avoid subtracting nearly equal numbers.** Rearrange formulas algebraically; use stable quadratic roots, log-sum-exp, or trigonometric identities.
3. **Sum intelligently.** Use `np.sum` (pairwise), `math.fsum` (exact-ish), or Kahan summation for long accumulations.
4. **Scale the problem.** Work in dimensionless units so that quantities are of order 1; this reduces both overflow and cancellation.
5. **Use the right type.** `decimal.Decimal` for decimal arithmetic, `fractions.Fraction` for exact rationals, `np.longdouble` only when the platform really provides extra precision.
6. **Check the condition number.** A well-conditioned problem is insensitive to small input perturbations; an ill-conditioned one amplifies them regardless of the algorithm.

```python
import numpy as np, math
a, b = 0.1 + 0.2 + 0.3, 0.6
print(abs(a - b) < 1e-12)          # True
print(np.isclose(a, b, rtol=1e-12))# True
print(math.isclose(a, b, rel_tol=1e-12))  # True
print(math.fsum([0.1]*10))         # 1.0 exactly rounded
```

Round-off is not truncation error

Round-off error comes from finite precision. **Truncation error** comes from replacing an infinite mathematical process by a finite one (a Taylor series cut after n terms, a finite step h in a derivative, a numerical integral with n panels). Reducing h reduces truncation error but may *increase* round-off error, because differences of close values lose digits. The total error often has a U-shape: too large h is inaccurate, too small h is noisy.

What to rehearse

Explain, with the binary expansion of 0.1, why `0.1+0.2 != 0.3`. Compute the relative error in $4.9 - 4.845$. Show how to compare floats correctly. Give three concrete ways to reduce cancellation or accumulation, and distinguish round-off from truncation error in one sentence.

PART

03

Data

数据文件与输入输出

Chapter 11 and Tutorial 5. From plain text to hierarchical scientific data.

6 Text Files: `.txt`

文本文件：最朴素、最通用的数据容器

A text file stores characters, organised into lines. It is the lowest common denominator of scientific data exchange: every language, every operating system and every text editor can read it. The cost is size and parsing effort. Chapter 11 and Tutorial 5 treat `.txt` as the first rung of a ladder that climbs through CSV, Pickle, JSON and HDF5.

> **中文精讲** Chinese companion
> 文本文件是所有数据格式的“共同语言”。它的优点是通用、可读、可用 `grep` 搜索、可用 Git 做版本比较；缺点是体积大、读写慢、没有类型信息，浮点数还会因为十进制转换而损失精度。考试重点：`open` 的六种模式、`with` 的作用、四种读取方式的区别、`np.savetxt/loadtxt` 的参数、以及“写完必须关闭/刷新”的原因。

6.1 Opening a file: modes and the `with` statement

打开文件：模式与上下文管理器

The built-in `open(filename, mode)` returns a file object. The mode determines whether the file must exist, whether it is truncated, and where writes go.

Mode	Name	Behaviour
`'r'`	read	open an existing file; `FileNotFoundError` if absent (default)
`'w'`	write	create a new file, or **erase** an existing one
`'a'`	append	create if absent; writes go to the end, old content kept
`'r+'`	read/write	file must exist; both reading and writing
`'w+'`	write/read	create or erase; then read and write
`'a+'`	append/read	create if absent; read anywhere, write at end

Table 6.1 The six text modes used in the course. Add `'b'` for binary, `'t'` is the default text mode.

Always close a file when you are done: `f.close()` flushes buffered data to disk and releases the operating-system handle. The safer idiom is a `with` block, which closes the file even if an exception occurs inside the block.

```
f = open("sensor_log.txt", "w")
for i in range(5):
    f.write(f"Sample {i}: reading = {i*2.5:.2f} mV\n")
f.close()

with open("run_summary.txt", "w") as f_out:
    f_out.write("Status: RUN COMPLETE\n")
    f_out.write("Exit Code: 0\n")
# f_out is closed automatically here, even after an error
```

Pitfall: `'w'` destroys the old file

Opening an existing file with `'w'` truncates it immediately, before you write anything. If you want to add a line, use `'a'`. If you want to read and then rewrite, read first, close, then open with `'w'` — or write to a temporary file and replace the original.

中文要点

- `'r'` 只读且要求文件存在；`'w'` 会清空原文件；`'a'` 保留原内容并在末尾追加；带 `+` 的模式可读写，但文件指针位置不同，初学阶段不建议混用。
- `f.close()` 不只是"礼貌"：Python 的写操作有缓冲区，不关闭或不 `flush`，程序异常退出时数据可能丢失。
- `with open(...) as f:` 是标准写法；离开缩进块时自动关闭，即使块内抛出异常。
- 文本模式默认使用平台编码（macOS/Linux 多为 UTF-8）。跨平台交换数据时，显式写 `encoding='utf-8'` 更安全。

6.2 Reading: four ways

读取的四种方式

- `f.read()` — read the entire remaining file into one string. Simple, but memory-hungry for large files.
- `f.readline()` — read one line up to and including `\n`. Repeated calls walk through the file.
- `f.readlines()` — read all remaining lines into a list of strings. Convenient for small files.
- `for line in f:` — iterate line by line, holding only one line in memory. This is the recommended way for large files.

Lines keep their trailing newline. Use `.strip()` to remove leading and trailing whitespace, or `.rstrip('\n')` to remove only the newline. To turn a line of numbers into floats, split first: `float(line.split()[0])`.

```
with open("sensor_log.txt", "r") as f:
    content = f.read()                # one big string

with open("sensor_log.txt", "r") as f:
    lines = f.readlines()             # list of lines, each with \n
    print(len(lines), lines[0].strip())

with open("sensor_log.txt", "r") as f:
    for idx, line in enumerate(f):    # streaming, memory friendly
        print(idx, line.strip())
```

中文要点

`read()` 一次读完，适合小文件；`readlines()` 得到一个列表，适合需要随机访问行的场景；直接 `for line in f` 是逐行流式读取，内存占用最小。注意 `read()` 之后文件指针在末尾，再调用 `read()` 会得到空字符串；要重新读必须 `seek(0)` 或重新打开。每行末尾的 `\n` 会保留，所以打印时常见多余空行，用 `.strip()` 处理。

6.3 Numerical arrays with NumPy

用 NumPy 读写数值表

For a table of numbers, manual parsing is tedious and slow. NumPy provides `np.savetxt` and `np.loadtxt`. The former writes a 1-D or 2-D array as plain text; the latter reads columns into an array and skips comment lines beginning with `#`. Both accept a `delimiter`, a `fmt`, and a `header`.

```
import numpy as np

t = np.linspace(0, 4, 5)
x = 0.5 * 9.81 * t**2
v = 9.81 * t
data = np.column_stack((t, x, v))

np.savetxt("trajectory.txt", data, fmt="%.2f",
           header="Time(s) Position(m) Velocity(m/s)")

loaded = np.loadtxt("trajectory.txt")        # header skipped automatically
tt, xx, vv = np.loadtxt("trajectory.txt", unpack=True)
print(loaded.shape)                          # (5, 3)
```

The file looks like this:

```
# Time(s) Position(m) Velocity(m/s)
0.00 0.00 0.00
1.00 4.91 9.81
2.00 19.62 19.62
3.00 44.15 29.43
4.00 78.48 39.24
```

If a file contains missing values or inconsistent columns, `np.loadtxt` fails; `np.genfromtxt` is the tolerant alternative, returning `nan` for missing entries and handling named columns. The argument `unpack=True` transposes the result so that each column becomes a separate variable.

> **中文要点**
>
> - `np.savetxt` 的 `header` 会自动在行首加 `#`，`np.loadtxt` 默认跳过 `#` 开头的行，所以能无缝往返。
> - `fmt='%.2f'` 决定写出的精度；默认 `'%.18e'` 会写出很多位，文件大且难读。
> - `delimiter=','` 可以写 CSV；默认是空格。
> - `unpack=True` 等价于对读入的二维数组做转置再拆包，常用于“一列一个变量”。
> - `np.savetxt` 只能处理规整的数值数组；混合类型、缺失值、带表头的复杂表，应使用 pandas 或 CSV 模块。

6.4 Checklist and common errors

检查清单与常见错误

1. Did you open with the intended mode? `'w'` overwrites silently.
2. Did you close the file, or use `with`?
3. Are you parsing the line before converting to `float`? `float("1.0 mV")` raises `ValueError`.
4. Does every line have the same number of columns? `loadtxt` requires a rectangular table.
5. Is the decimal separator a point? A comma decimal (common in some locales) will confuse `loadtxt`.
6. Are you mixing text and binary? Never open a `.txt` in `'rb'` and expect strings.

What to rehearse

Write a five-line program that creates a file, appends a line, then reads it line by line and prints the average of the numeric column. Know the exact difference between `'w'`, `'a'` and `'r+'`. Be able to explain why `with` is preferred over manual `close()`, and what `unpack=True` does.

本章考点清单

1. 写出六种打开模式，并说明哪一种会清空文件。
2. 比较 `read`、`readline`、`readlines` 和直接迭代的内存行为。
3. 用 `np.savetxt` 写带表头的二维数组，再用 `loadtxt` 读回并拆列。
4. 解释 `header` 前的 `#` 与 `loadtxt` 的注释规则。
5. 说明不关闭文件可能导致的数据丢失。
6. 说出 `loadtxt` 失败时的两个替代方案（`genfromtxt`、pandas）。

7 CSV Files

CSV 文件：表格数据的通用格式

CSV stands for **comma-separated values**. Each line is a record; fields are separated by commas; the first line is often a header. It is the lingua franca of spreadsheets and experimental tables: simple enough to inspect in a text editor, structured enough for pandas, Excel, R and MATLAB.

中文精讲 Chinese companion

CSV 的本质是“用分隔符排成列的纯文本”。它比 `.txt` 多了一层表格约定：一行一条记录，一列一个变量，通常有表头。优点是 Excel 可直接打开、跨语言、可读；缺点是没有类型信息、没有公式、对逗号/换行/引号需要转义，浮点精度也会在十进制转换中损失。考试常考：CSV 与 TXT 的区别、`csv` 模块的读写、NumPy 的 `delimiter` 参数、pandas 的 `read_csv` 基本用法。

7.1 Structure and pitfalls

结构与陷阱

A minimal CSV file:

```
time,position,velocity
0.00,0.00,0.00
1.00,4.91,9.81
2.00,19.62,19.62
```

Three details cause most of the pain. First, a field that contains a comma, a quote or a newline must be quoted; the `csv` module handles this automatically. Second, the decimal separator is a point in the standard format, but some locales write 3,14; a file produced by such a system will not parse as floats. Third, Excel may silently convert long digit strings to scientific notation or dates; never use Excel as the final storage for precise numbers.

中文要点

CSV 不是“把数组用逗号存起来”这么简单：字段中若含逗号（例如地址“Toronto, ON”），必须用双引号包住；字段中若含双引号，要写成两个双引号。手写解析器很容易在这些边界情况出错，所以优先使用 `csv` 模块或 pandas。另一个常见坑是表头：`np.loadtxt` 默认不跳过表头，必须加 `skiprows=1`；而 `np.genfromtxt(names=True)` 可以把表头当作字段名。

7.2 Writing and reading with the `csv` module

用标准库读写 CSV

```
import csv

rows = [["time", "position", "velocity"],
        [0.0, 0.0, 0.0],
        [1.0, 4.91, 9.81],
        [2.0, 19.62, 19.62]]

with open("trajectory.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(rows)          # writes all rows

with open("trajectory.csv", "r", newline="") as f:
    reader = csv.reader(f)
    header = next(reader)           # consume the header
    data = [[float(x) for x in row] for row in reader]
print(header, data[1])
```

Use `newline=""` when opening CSV files: without it, Windows may insert blank lines between records, and the `csv` module's own newline handling can be defeated. For files with a header and named columns, `csv.DictReader` yields dictionaries, which are often more readable than positional lists.

```
with open("trajectory.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["time"], row["position"])
```

中文要点

写 CSV 时用 `csv.writer`，它会自动处理引号与转义；不要自己用 `",".join()` 拼接。读的时候 `csv.reader` 返回的是字符串，需要手动 `float()`；`DictReader` 则把表头映射为键，适合列数多、顺序可能变化的表。`next(reader)` 用来跳过表头；如果表头有多行，就多调用几次。

7.3 NumPy and pandas

NumPy 与 pandas

For a purely numerical table, NumPy is enough:

```
import numpy as np
data = np.loadtxt("trajectory.csv", delimiter=",", skiprows=1)
np.savetxt("out.csv", data, delimiter=",", fmt="%.3f",
           header="time,position,velocity", comments="")
```

The `comments=""` argument prevents `savetxt` from prefixing the header with `#`, which would break a strict CSV reader.

pandas is the better tool when the table has named columns, mixed types, missing values, or needs

filtering and grouping:

```
import pandas as pd
df = pd.read_csv("trajectory.csv")
print(df.head())
print(df.describe())
print(df[df["velocity"] > 10])
df["kinetic"] = 0.5 * df["velocity"]**2
df.to_csv("processed.csv", index=False)
```

A pandas `DataFrame` is a labelled 2-D table: `df["column"]` selects a column, Boolean masks filter rows, and `groupby` summarises categories. Missing values become `NaN` by default; `dropna()` or `fillna()` decides what to do with them.

中文要点

- NumPy 适合“纯数字、规整、列数固定”的表；pandas 适合“有列名、混合类型、缺失值、需要筛选/分组”的表。
- `np.loadtxt` 读 CSV 必须显式写 `delimiter=','` 和 `skiprows=1`；否则第一行表头会导致 `ValueError`。
- `np.savetxt` 默认给表头加 `#`；若下游是严格的 CSV 解析器，加 `comments=''`。
- pandas 的 `read_csv` 会自动识别表头与类型，但“自动”也可能把 `00123` 读成整数 123，把日期读成 `Timestamp`；需要时用 `dtype=str` 或 `parse_dates` 明确指定。
- 大文件不要一次 `read_csv` 全读入；用 `chunksize=...` 分块处理。

7.4 CSV versus TXT

CSV 与 TXT 的选择

	TXT (whitespace)	CSV (comma)
Readability	good for numbers	good for tables, Excel
Separator	spaces/tabs	comma (or semicolon)
Headers	convention, often #	first row by convention
Ambiguity	whitespace inside fields	quoting rules needed
Best for	quick numeric arrays	experimental tables, sharing

Table 7.1 Two plain-text formats, same idea, different conventions.

What to rehearse

Write a CSV with a header using `csv.writer`, read it back with `csv.reader`, and convert one column to floats. Know the three NumPy arguments `delimiter`, `skiprows`, `comments`. Give one task where pandas is clearly better than NumPy, and one where plain text is better than both.

本章考点清单

1. 解释 CSV 的三个约定：行 = 记录、逗号 = 分隔、首行常为表头。
2. 说明为什么写 CSV 要用 `newline=''` 和 `csv.writer`。
3. 用 NumPy 读写带表头的 CSV，正确设置 `delimiter` 与 `skiprows`。
4. 说出 pandas 相对 NumPy 的两个优势（列名、缺失值/筛选）。
5. 列举 CSV 的三个坑：字段内逗号、十进制逗号、Excel 自动改格式。

8 Pickle and JSON

Pickle 与 JSON：对象的序列化

Both formats store Python objects rather than just numbers, but they make opposite trade-offs. Pickle is Python-specific and can preserve almost any object, including custom classes and NumPy arrays; JSON is language-independent and deliberately limited to a small set of types. Choosing between them is a question of audience and lifetime, not of taste.

> **中文精讲** Chinese companion
> 序列化（serialization）是把内存中的对象变成字节流，反序列化（deserialization）是反向还原。Pickle 是 Python 的“私有记忆”：能把字典、列表、元组甚至自定义类原样存下，但只有 Python 能读，而且不同 Python 版本可能不兼容。JSON 是“公共语言”：只支持字典、列表、字符串、数字、布尔和 `null`，但几乎所有编程语言都能读写，适合 API 和跨平台交换。考试常考：两者的适用场景、`dump/load` 与 `dumps/loads` 的区别、Pickle 的安全风险、JSON 不能直接存 NumPy 数组。

8.1 Pickle: Python's private memory

Pickle：Python 对象的二进制快照

Serialization converts an in-memory object (a dictionary, a list, a trained model) into a byte stream; **deserialization** reconstructs the object. Pickle writes binary data, so the file must be opened in `'wb'` or `'rb'` mode, never `'w'` or `'r'`.

```
import pickle

data = {"temperature": [20.1, 20.3, 19.8],
        "sensor": "Berkeley",
        "calibrated": True}

with open("data.pkl", "wb") as f:      # w = write, b = binary
    pickle.dump(data, f)

with open("data.pkl", "rb") as f:      # r = read, b = binary
    restored = pickle.load(f)
print(restored["sensor"])               # Berkeley
```

The pair `dump/load` works with file objects; `dumps/loads` work with `bytes` in memory. Pickle preserves Python types, including tuples, sets, NumPy arrays and instances of classes defined in your own modules. That power is also the problem: unpickling can execute arbitrary code embedded in the file, so **never load a pickle from an untrusted source**. It is also not a long-term archival format: a pickle written by Python 2 may fail in Python 3, and a class whose definition has moved may no longer be importable.

```
with open("old_py2.pkl", "rb") as f:
    obj = pickle.load(f, encoding="latin1")   # compatibility with Python 2
```

> Security and portability
> Pickle is not encrypted, not signed and not safe against malicious files. Use it only for data you produced yourself, on the same machine or a trusted environment. For long-term storage or cross-

language exchange, prefer JSON, CSV, HDF5 or Parquet. If a pickle fails with `UnicodeDecodeError`, try `encoding='latin1'`; if it fails with `ModuleNotFoundError`, the class definition is missing, not the data.

中文要点

- `pickle.dump(obj, f)` 把对象写入已打开的文件；`pickle.load(f)` 读回。文件必须用二进制模式。
- `pickle.dumps(obj)` 返回 `bytes`，`pickle.loads(b)` 从字节还原；适合网络传输或存进数据库。
- Pickle 保存的是“Python 对象的构造方式”，不是通用数据；反序列化时需要能找到原来的类，所以跨版本/跨项目容易失败。
- 安全：不要 `pickle.load` 任何不可信来源的文件。反序列化可以触发任意代码执行，这是设计特性，不是漏洞。
- 兼容性：Python 2 的 pickle 在 Python 3 中读取时可能需要 `encoding='latin1'`；但并非所有对象都能恢复。

8.2 JSON: the common language

JSON：跨语言的键值格式

JSON (JavaScript Object Notation) stores data as nested key–value pairs. Its type system is small: object (dictionary), array (list), string, number, boolean and `null`. That limitation is a feature: every language agrees on what the file means.

```
import json

config = {
    "experiment": "spring calibration",
    "samples": 120,
    "temperatures": [20.1, 20.3, 19.8],
    "calibrated": True,
    "notes": None
}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

with open("config.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["temperatures"][0])        # 20.1
```

JSON is text, so it is human-readable, diff-friendly and safe to send over HTTP. It is also compact compared with an equivalent pickle, and it is the default payload of most web APIs. The price is type loss: tuples become lists, dictionary keys must be strings, and NumPy arrays are not serializable by default. Convert them explicitly:

```
import numpy as np, json
arr = np.array([1.0, 2.0, 3.0])
payload = {"x": arr.tolist(), "shape": list(arr.shape)}
with open("array.json", "w") as f:
    json.dump(payload, f)
```

中文要点

- JSON 的六个类型：object、array、string、number、boolean、null。Python 的

`dict/list/str/int/float/bool/None` 可以一一对应。

- `json.dump(obj, f)` 写入文件，`json.dumps(obj)` 返回字符串；`load/loads` 是反向操作。
- `indent=2` 让文件带缩进、便于人读；`sort_keys=True` 让键有序、便于版本比较；`ensure_ascii=False` 保留中文，不转成 `\uXXXX`。
- JSON 没有元组、集合、日期、复数、NumPy 数组；存之前要转成列表/字符串。读回来也只是列表，不再是原来的类型。
- JSON 的键必须是字符串；数字只有一种 `number` 类型，整数和浮点在外观上可能相同。

8.3 Choosing a format

如何选择：一张表说清

	Pickle	JSON	When to use
Type	binary	text	—
Language	Python only	almost all	cross-language → JSON
Types	almost anything	six basic types	custom objects → Pickle
Readability	none	good	human review → JSON
Size	compact	larger	large numeric arrays → HDF5
Security	unsafe if untrusted	safe to parse	untrusted input → JSON
Long-term	version-sensitive	stable	archive → JSON/CSV/HDF5

Table 8.1 Pickle versus JSON; the last column is the decision rule, not a property.

Rule of thumb

Use JSON when the data leaves your program; use Pickle when the data is a temporary Python-only snapshot; use CSV for rectangular tables; use HDF5 for large arrays with structure. If you would not email the file to a collaborator, do not call it an archive.

What to rehearse

Write the four lines that save and load a dictionary with Pickle. Explain why the mode is `'wb'` and `'rb'`. State two security/portability warnings. Write the JSON equivalent, and explain how to store a NumPy array in JSON. Give a scenario in which Pickle is the right choice and one in which it is the wrong choice.

本章考点清单

1. 定义序列化与反序列化，分别对应 Pickle 的哪两个函数。
2. 说明 `dump/load` 与 `dumps/loads` 的区别（文件 vs 字节）。
3. 列出 Pickle 的两个限制：仅 Python、版本/类定义依赖。
4. 说明为什么不能反序列化不可信来源的 Pickle。
5. 列出 JSON 的六种类型，并解释为什么 NumPy 数组要 `.tolist()`。
6. 给定场景选择 Pickle、JSON、CSV 或 HDF5，并给出理由。

9 HDF5 and Hierarchical Data

HDF5：为大规模科学数据设计的层次化容器

HDF5 stands for **Hierarchical Data Format, version 5**. It is a binary container that behaves like a filesystem inside a file: **groups** are folders, **datasets** are array-valued files, and **attributes** are small metadata notes attached to either. It was designed for data too large or too structured for CSV, and it supports compression, chunked I/O and parallel access.

> **中文精讲** Chinese companion
> HDF5 可以想成一个“文件里的文件系统”：组（group）像文件夹，数据集（dataset）像里面的 NumPy 数组，属性（attribute）像贴在组或数据集上的便签（单位、采样率、起始时间、位置）。它没有 CSV 的行列限制，也没有 Pickle 的 Python 绑定，适合 TB 级数据、多传感器、多实验的归档。考试重点：三个概念的区别、`h5py` 的读写语法、路径式访问、属性读取，以及 HDF5 与 CSV/JSON/Pickle 的取舍。

9.1 Groups, datasets and attributes

三个核心概念

Object	Analogy	Meaning
Group	folder	a container for datasets and other groups; the root is `/`
Dataset	file / array	a homogeneous, typed, possibly huge array (NumPy-like)
Attribute	sticky note	small metadata attached to a group or dataset (scalar or short array)

Table 9.1 The HDF5 object model used in the course.

A path such as `/acc/1/data` reads left to right: the group `acc` contains the subgroup `1`, which contains the dataset `data`. The same dataset can carry attributes `dt`, `start_time` and `location`. This is exactly the structure needed when several instruments sample at different rates: the hierarchy names the instrument and channel, while the attributes record how to interpret the numbers.

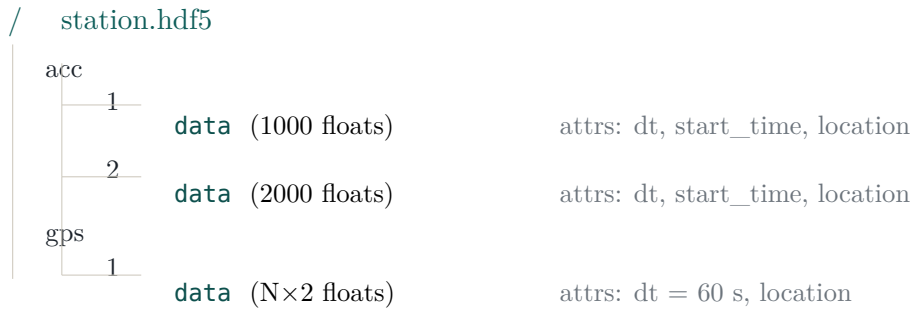


Figure 9.1 A station file with two accelerometers and one GPS channel.
图 9.1 一个 HDF5 文件的层次结构：两个加速度计与一个 GPS 通道。

中文要点

组可以嵌套，路径用 / 分隔；数据集是同类型的多维数组，支持切片读取（不必一次读入全部）；属性是“关于数据的数据”，只适合小量标量/短数组，不适合存大表。HDF5 的文件结构自描述：拿到文件就能看到组名、数据集形状、类型和属性，不需要额外的说明文档。

9.2 Writing a file with **h5py**

用 h5py 写文件

The lecture example monitors two accelerometers (Berkeley and Oakland) at different sampling rates and one GPS station in San Francisco sampling every 60 s. The code creates the hierarchy, writes each array, and attaches metadata.

```
import numpy as np
import h5py

# --- synthetic sensor data ---
acc_1 = np.random.random(1000)
location_1, start_time_1, dt_1 = "Berkeley", 1542000276, 0.04

acc_2 = np.random.random(2000)
location_2, start_time_2, dt_2 = "Oakland", 1542000276, 0.02

gps = np.random.random((1000, 2))       # (lon, lat)
location_3, dt_3 = "San Francisco", 60.0

with h5py.File("station.hdf5", "w") as hf:
    hf["/acc/1/data"] = acc_1
    hf["/acc/1/data"].attrs["dt"] = dt_1
    hf["/acc/1/data"].attrs["start_time"] = start_time_1
    hf["/acc/1/data"].attrs["location"] = location_1

    hf["/acc/2/data"] = acc_2
    hf["/acc/2/data"].attrs.update(
        {"dt": dt_2, "start_time": start_time_2, "location": location_2})

    hf["/gps/1/data"] = gps
    hf["/gps/1/data"].attrs.update({"dt": dt_3, "location": location_3})
```

Assigning an array to a path creates the intermediate groups automatically. For large datasets, create the dataset explicitly so you can choose chunking and compression:

```
with h5py.File("big.hdf5", "w") as hf:
    dset = hf.create_dataset("/experiment/run1/trace", shape=(10000, 512),
                             dtype="float32", chunks=(100, 512),
                             compression="gzip", compression_opts=4)
    for i in range(0, 10000, 100):
        dset[i:i+100] = np.random.random((100, 512))
```

Chunking stores the array in blocks, so a slice reads only the blocks it needs; compression reduces disk size at some CPU cost. Neither changes the logical array: `dset.shape` is still $(10000, 512)$.

中文要点

- `hf['/acc/1/data'] = arr` 会自动创建 `acc` 和 `1` 两个组；路径必须唯一。
- 大量小数据集会产生大量元数据，降低性能；相反，一个巨大的二维数组应配合 chunking 和压缩。

- `attrs` 支持标量、字符串、短数组；不要用它存长列表。
- 写模式 `'w'` 会覆盖已有文件；追加/修改用 `'a'` 或 `'r+'`。
- 文件必须关闭（`with` 最稳），否则缓冲区可能未落盘，其他进程也可能读不到完整内容。

9.3 Reading and exploring

读取与浏览结构

```python
import h5py
with h5py.File("station.hdf5", "r") as hf:
    print(list(hf.keys()))                  # ['acc', 'gps']
    print(list(hf["acc"].keys()))           # ['1', '2']
    data = hf["/acc/1/data"][:]             # load the whole array
    first = hf["/acc/1/data"][:10]          # load only 10 samples
    dt = hf["/acc/1/data"].attrs["dt"]      # 0.04
    print(data.shape, dt)

    def show(name, obj):
        if isinstance(obj, h5py.Dataset):
            print(name, obj.shape, obj.dtype)
    hf.visititems(show)
```

`keys()` lists the children of a group; `visititems` walks the whole tree recursively. Slicing a dataset reads only the requested part, which is why HDF5 scales to files larger than RAM. `hf['/path'][()]` returns a scalar, while `[:]` copies an array into memory.

中文要点

`hf.keys()` 只列当前组的直接子节点；要看整棵树用 `visititems`。`hf['/acc/1/data'][:]` 会把整个数据集复制进内存，数据大时只取需要的切片。属性用 `.attrs['name']` 读取，返回的是 NumPy 标量或数组。若忘记关闭文件，macOS/Linux 上可能残留文件锁，导致下次写入失败。

9.4 Why HDF5

为什么用 HDF5

- **No practical size limit.** It is designed for datasets larger than memory; slices and chunks are read on demand.
- **Structure.** Groups encode the experiment, instrument and channel; attributes record units and sampling.
- **Efficiency.** Binary storage, chunking, compression and parallel I/O avoid the parse cost of text.
- **Portability.** The format is open and supported by Python, MATLAB, R, C, Java and Julia; it is not tied to Python like Pickle.

The cost is complexity: you cannot open an HDF5 file in a text editor, a corrupted file can be hard to recover, and concurrent writers need careful locking. For a small table, CSV is simpler and more transparent.

	CSV	JSON	Pickle	HDF5
Readable	yes	yes	no	no
Types	numbers/text	6 basic	Python objects	typed arrays
Hierarchy	no	nested	nested	groups + datasets
Large arrays	poor	poor	moderate	excellent
Cross-language	yes	yes	no	yes
Metadata	header only	keys	embedded	attributes

Table 9.2 The four storage formats of Chapter 11 at a glance.

What to rehearse

Draw the tree for `/acc/1/data` and `/gps/1/data`. Write the four lines that create a file, store an array at a path, attach `dt`, and close. Explain the difference between a group, a dataset and an attribute. Give two reasons to choose HDF5 over CSV and one reason not to.

本章考点清单

1. 区分 group、dataset、attribute，并各举一个例子。
2. 用 `h5py.File(..., 'w')` 创建层次结构，写入数组与属性。
3. 用 `keys()`、`visititems()`、切片读取浏览一个大文件。
4. 解释 chunking 与 compression 的作用，以及它们不改变逻辑形状。
5. 说明 HDF5 相对 CSV/Pickle 的两个优势和两个代价。
6. 为什么 HDF5 适合“多传感器、不同采样率”的数据？

10 Choosing a Data Format

数据格式的取舍：一张决策图

The five formats in Chapter 11 are not interchangeable; each encodes a different answer to three questions: *Who will read the file? How large is the data? How long must it live?* A good choice makes the next step easy; a bad one forces a conversion script that someone will maintain forever.

中文精讲 Chinese companion

格式选择没有“最好”，只有“匹配”。先问三个问题：

1. 文件要给谁读？只有自己的 Python 脚本，还是 Excel、MATLAB、R、同事、网页前端？
2. 数据多大？几行数字，几百 MB 的表，还是超过内存的数组？
3. 要保存多久？一次会话的临时缓存，还是一个项目的归档？

答案几乎自动指向某个格式：临时 Python 对象 → Pickle；跨语言小数据 → JSON；规则表格 → CSV；带结构的数值大数据 → HDF5；纯数值快速交换 → TXT。

10.1 A decision procedure

决策流程

The decision ladder

1. **Will anyone else — or another language — read it?**
   Yes, and the data are small or nested: **JSON**. Yes, and the data form a table: **CSV**. No: continue.
2. **Is it a rectangular table of numbers?**
   Yes: **CSV** (or TXT if it is a quick numeric dump). No: continue.
3. **Are there large numeric arrays with structure (channels, runs, metadata)?**
   Yes: **HDF5**. No: **Pickle** is acceptable for a temporary Python-only cache.

中文要点

注意决策图的顺序：先问“跨语言/跨人”，再问“是否规则表格”，最后才问“是否大规模数值”。很多人一上来就用 Pickle，因为最省事，但它在跨语言和长期保存两个维度上都失败。反过来，JSON 虽然通用，但存 100 万个浮点数会非常臃肿；这时 HDF5 才是正确选择。

10.2 Comparison and selection

五种格式的总表

Format	Type	Human	Cross-lang	Structure	Typical use
TXT	text	yes	yes	flat	quick numeric arrays, logs
CSV	text	yes	yes	table	experimental tables, Excel
JSON	text	yes	yes	nested	config, API, small records
Pickle	binary	no	no	arbitrary Python	temporary cache, models
HDF5	binary	no	yes	hierarchy	large arrays, multi-sensor

Table 10.1 A compact selection table; “structure” means native support, not what you can fake with naming conventions.

Three rules that survive the exam

1. **Exchange → text; archive → open binary.** JSON/CSV for exchange, HDF5 for large archives; Pickle never leaves the machine.
2. **Preserve the metadata.** Units, sampling rate, start time, calibration and missing-value codes belong with the data, not in a separate README that will be lost.
3. **Test the round trip.** Write a small sample, read it back, compare shapes, dtypes and values. A format that cannot round-trip is not a storage choice; it is a data loss.

10.3 Worked scenario

综合例题：为项目选格式

A lab records three accelerometers at 100 Hz for six months, plus a 60 s GPS track, and shares results with a collaborator who uses MATLAB. The raw data are several hundred GB; the final paper needs

a small table of fitted parameters.

- **Raw signals:** HDF5. Hundreds of GB, many channels, different sampling rates, needs slicing and compression; MATLAB can read HDF5.
- **Experiment log:** JSON or CSV. Small, human-readable, easy to diff.
- **Paper table:** CSV. A dozen rows, collaborators may open it in Excel.
- **Temporary cache of Python objects:** Pickle, on the same machine, never shared.

Mixing formats is normal. The mistake is using one format for every job: a 300 GB pickle is unreadable outside Python, and a 300 GB CSV is slow to parse and easy to corrupt.

> What to rehearse
> Given a scenario (size, readers, lifetime, structure), name the best format and justify it in two sentences. Be ready to say why Pickle is a poor archive and why CSV is a poor choice for a 10 GB array. Also know that HDF5 and JSON can both represent nested structure, but HDF5 is binary and array-oriented while JSON is text and record-oriented.

本章考点清单

1. 用三个问题（读者、大小、寿命）建立格式选择框架。
2. 记住五种格式的类型、可读性、跨语言性与结构能力。
3. 为一个“多传感器 + 大数组 + MATLAB 合作者”的项目选型并说明理由。
4. 解释“交换用文本、归档用开放二进制、Pickle 不出本机”的三条规则。
5. 说明 round-trip 测试的必要性。

PART

04

Visualization

科学绘图、地图与动画

Chapter 12 and Tutorial 5. Making data visible without misleading the reader.

11 2D Plotting with Matplotlib

二维绘图：让数据自己说话

Matplotlib is the foundational plotting library of the scientific Python stack. Its `pyplot` interface mirrors MATLAB: create a figure, draw on it, label it, and show or save it. A good plot is not decoration; it is an argument about the data, and every axis, unit, label and colour is part of that argument.

> **中文精讲** Chinese companion
> 二维绘图是 C12 与 Tutorial 5 的交汇点。必须掌握的最小集合：`plot(x, y)`、格式字符串（颜色/标记/线型）、标题与轴标签、图例、网格、`figsize`、`savefig`。考试常考：给出一段绘图代码问图像长什么样；指出图中缺少的元素（单位、图例、标题）；解释 `'r--'` 的含义；说明为什么 `plt.show()` 之后再 `savefig` 会得到空白图。

11.1 The figure–axes model

Figure 与 Axes：画布与坐标系

A **Figure** is the whole canvas; an **Axes** is a coordinate system inside it (usually one per subplot). `plt.plot` draws on the current axes; the object-oriented API makes the structure explicit:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot([0, 1, 2, 3], [0, 1, 4, 9], "o-", color="teal", label="x^2")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("A parabola")
ax.legend()
ax.grid(True)
fig.savefig("parabola.png", dpi=300)
plt.show()
```

For quick exploration, `plt.plot(x, y)` followed by `plt.show()` is enough. For anything reusable, prefer `fig, ax = plt.subplots()` so that each axis is a named object. In a Jupyter notebook, the magic `%matplotlib notebook` enables panning and zooming inside the output cell, while `%matplotlib inline` produces a static image; these are IPython commands, not Python functions.

> **中文要点**
> Figure 是整张画布，Axes 是画布上的一个坐标系；一个 Figure 可以放多个 Axes（子图）。`plt.plot` 作用于“当前 Axes”，适合交互式快速画图；`fig, ax = plt.subplots()` 返回显式对象，适合需要精细控制或保存的图。考试若问“为什么图上没有图例”，先检查 `plot` 里有没有 `label=...`，再检查有没有调用 `legend()`。

11.2 Format strings

格式字符串：颜色、标记、线型

The third argument of `plot` is a compact format string:

`'[color][marker][line]'`.

For example, `'ro'` means red circles with no connecting line, `'g--'` a dashed green line, and `'k.-'` a black dash-dot line with points.

Colour		Marker	Line style
`'r'` red	`'b'` blue	`'o'` circle	`'-'` solid
`'g'` green	`'k'` black	`'*'` star	`'--'` dashed
`'c'` cyan	`'m'` magenta	`'x'` cross	`':'` dotted
`'y'` yellow	`'w'` white	`'.'` point	`'-. '` dash-dot

Equivalent keyword arguments are clearer when a plot has many options:

```
ax.plot(x, y, color="red", marker="o", linestyle="--", linewidth=2,
        markersize=6, label="measurement")
```

> **中文要点**
> 格式字符串的顺序是“颜色 + 标记 + 线型”，例如 `'r*'` 是红色星形点（无连线），`'b-'` 是蓝色实线，`'g--'` 是绿色虚线。颜色字母 `b` 是 blue，`k` 是 black，别把 `b` 当成 brown。更推荐显式关键字（`color=`、`linestyle=`），可读性更好，也不容易记错顺序。

11.3 Context: titles, labels, legends, grid

上下文：标题、轴标签、图例、网格

A plot without units is not a scientific plot. The minimum annotation set is: a descriptive title, an x-axis label with units, a y-axis label with units, a legend when more than one dataset is shown, and a grid if the reader needs to read values.

```
x = np.linspace(-5, 5, 25)
plt.figure(figsize=(9, 5))
plt.plot(x, x**2, "ko", label="Quadratic")
plt.plot(x, x**3, "r*", label="Cubic")
plt.title("Polynomials from -5 to 5")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend(loc="upper left")
plt.savefig("polynomials.png", dpi=300, bbox_inches="tight")
plt.show()
```

Useful knobs: `figsize=(width,height)` in inches; `dpi` for resolution; `bbox_inches='tight'` to avoid clipping labels; `plt.style.use('ggplot')` or `'seaborn-v0_8'` for a global look. Style names changed across Matplotlib versions; `'seaborn-poster'` from older slides may raise an error in current releases, so prefer the versioned name or set the individual properties.

> **中文要点**
>
> - 标题说“是什么”，轴标签说“量 + 单位”，图例说“哪条线是谁”。三者缺一，图就不能独立阅读。
> - `label=` 只是给这条线起名，必须再调用 `legend()` 才会显示。
> - `savefig` 要在 `show` 之前调用；某些后端在 `show` 之后会清空画布，保存出来是空白。
> - `dpi=300` 适合印刷；网页用 100–150 即可。`bbox_inches='tight'` 防止标签被裁掉。
> - 样式名随版本变化：老代码的 `'seaborn-poster'` 在新版 Matplotlib 中已改名，考试若写代码，建议直接用颜色/线型参数，避免依赖样式名。

11.4 Worked example: damped oscillator

完整例题：阻尼振子对比

The Tutorial 5 exercise compares two damped oscillators,

$$y(t) = A e^{-\gamma t}\cos(\omega t),$$

with $A = 2.0$ m, $\omega = 3.0$ rad/s, $\gamma_1 = 0.3\ \mathrm{s}^{-1}$ and $\gamma_2 = 1.0\ \mathrm{s}^{-1}$, over $t \in [0, 10]$ s.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 200)
A, omega = 2.0, 3.0
y1 = A * np.exp(-0.3 * t) * np.cos(omega * t)
y2 = A * np.exp(-1.0 * t) * np.cos(omega * t)

data = np.column_stack((t, y1, y2))
np.savetxt("oscillators.txt", data, fmt="%.2f",
           header="Time(s) y1(m) y2(m)")

plt.figure(figsize=(10, 5))
plt.plot(t, y1, "b-", linewidth=2,
         label=r"Lightly damped ($\gamma=0.3$)")
plt.plot(t, y2, "r--", linewidth=2,
         label=r"Heavily damped ($\gamma=1.0$)")
plt.title("Damped Harmonic Oscillator Response", fontsize=14)
plt.xlabel("Time (s)", fontsize=12)
plt.ylabel("Position y(t) (m)", fontsize=12)
plt.legend(loc="upper right", fontsize=11)
plt.grid(True, linestyle="--", alpha=0.7)
plt.show()
```

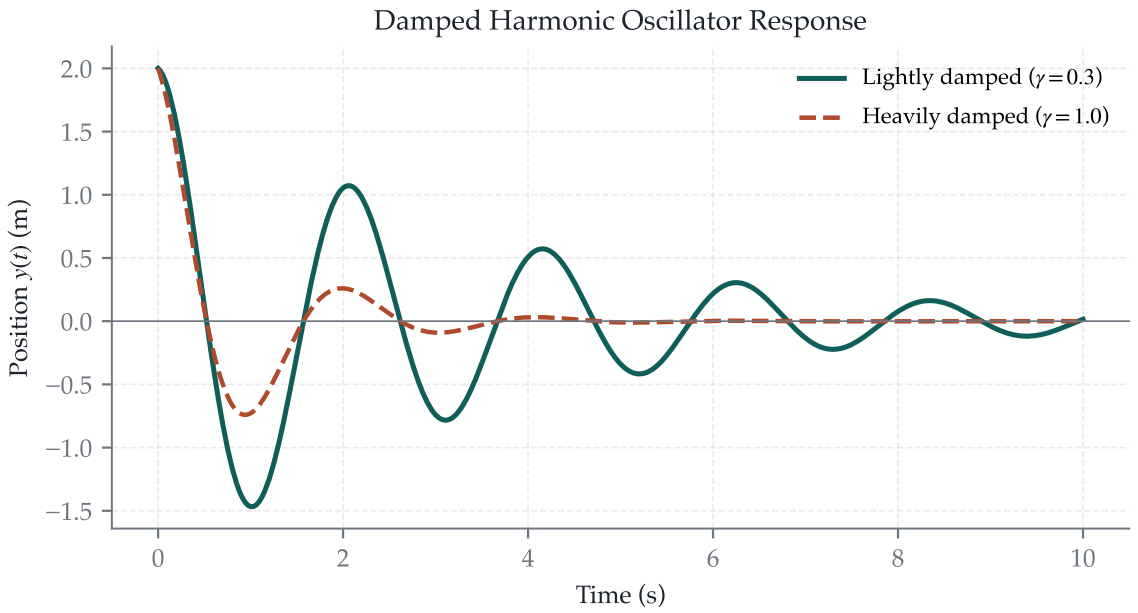


Figure 11.1 The two oscillator trajectories: light damping decays slowly, heavy damping kills the oscillation within a few periods.

图 11.1 两种阻尼的位移–时间曲线：轻阻尼衰减慢，重阻尼在几个周期内趋于零。

中文要点

这道题覆盖了完整工作流：生成时间数组 → 计算两条曲线 → 用 `column_stack` 合成表格并 `savetxt` → 建图 → 画线并加 `label` → 标题/轴标签/图例/网格 → 显示。注意 `r'...'` 前缀：图例里有 LaTeX 的 γ，必须用 raw string，否则 `\g` 会被当成转义。物理上，γ 越大，包络 $e^{-\gamma t}$ 衰减越快，振荡次数越少。

What to rehearse

Given `plt.plot(x, y, 'r--')`, describe the result. List the five commands that make a plot self-explanatory. Explain why `label=` alone does not create a legend. Write the three lines that save a figure at 300 dpi without clipping. Sketch the damped-oscillator curves and explain which parameter controls the envelope and which controls the oscillation frequency.

本章考点清单

1. 解释 Figure 与 Axes 的关系；写出两种建图方式。
2. 读懂格式字符串 `'ro'`、`'g--'`、`'k.-'`。
3. 列出标题、轴标签、图例、网格、`figsize`、`dpi` 的作用。
4. 说明 `savefig` 与 `show` 的顺序。
5. 画出阻尼振子并解释 γ 对包络的影响。
6. 指出一张“坏图”缺少哪些元素。

12 3D Plotting

三维绘图：线、散点与曲面

Three-dimensional plots are the natural language of functions of two variables, $z = f(x, y)$, and of trajectories in space. Matplotlib's `mplot3d` toolkit provides the same grammar as 2D plotting, with an extra axis. The key new idea is the **meshgrid**: a pair of 2-D coordinate arrays that cover a rectangular grid in the xy-plane.

> **中文精讲** Chinese companion
> 三维图并不比二维图难，只是多了一个 z 轴和一个"视角"参数。必须掌握三种图：`plot3D` 画参数曲线、`scatter` 画三维散点、`plot_surface` 画二元函数曲面。曲面的关键是 `np.meshgrid(x, y)`：它把一维的 x 和 y 变成两个同样形状的二维数组 X, Y，再逐点计算 $Z = f(X, Y)$。考试常考：解释 meshgrid 的作用、为什么 `Z` 的 shape 与 `X` 相同、`projection='3d'` 写在哪里、颜色映射（colormap）与 colorbar 的作用。

12.1 Creating a 3D axes

建立三维坐标系

There are two equivalent idioms:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D   # registers the 3d projection

fig = plt.figure(figsize=(7, 5))
ax = fig.add_subplot(projection="3d")     # preferred, modern style
# ax = plt.axes(projection="3d")          # also works
```

The `projection='3d'` argument is what turns an ordinary axes into a 3-D one. Without it, `plot3D` and `plot_surface` are unavailable.

> **中文要点**
> `from mpl_toolkits.mplot3d import Axes3D` 在新版 Matplotlib 中主要是"注册投影"，即使不显式导入，`projection='3d'` 通常也能工作；但课件保留这行导入，考试写出来不会错。关键是创建 axes 时指定投影，而不是在 `plot` 里指定。

12.2 3D line and scatter plots

三维曲线与散点

A 3-D line plot takes three coordinate arrays of equal length. The lecture example uses a parametric curve (x, y, t):

```
import numpy as np
t = np.linspace(0, 20, 1000)
x = np.sin(t)
y = np.cos(t)
ax.plot3D(x, y, t, "teal", linewidth=2)
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("t")
```

A 3-D scatter plot takes three arrays and optional colour/size arguments:

```
x = np.random.random(50)
y = np.random.random(50)
z = np.random.random(50)
ax.scatter(x, y, z, c="r", s=50)
```

The argument `c` can be a single colour or a sequence mapped through a colormap; `s` is the marker size in points squared.

> **中文要点**
> 三维线的三个数组必须等长，分别对应 x, y, z；三维散点的 `c` 既可以是一个颜色字符串，也可以是一组数值（配合 `cmap` 使用）。若 `c` 是数值数组，必须同时给出 `cmap`，并用 `fig.colorbar(...)` 说明颜色代表什么。散点图不要用太多点，否则遮挡严重；可以调小 `s` 或降低透明度 `alpha`。

12.3 Surfaces and `meshgrid`

曲面与网格

To plot $z = \sin x \cos y$, we need the value of z at every grid point. `np.meshgrid` builds the grid:

```
x = np.arange(-5, 5.1, 0.2)
y = np.arange(-5, 5.1, 0.2)
X, Y = np.meshgrid(x, y)      # X, Y both have shape (len(y), len(x))
Z = np.sin(X) * np.cos(Y)

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
surf = ax.plot_surface(X, Y, Z, cmap="cividis", linewidth=0,
                       antialiased=True)
fig.colorbar(surf, shrink=0.6, pad=0.1)
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
ax.view_init(elev=30, azim=-125)   # camera elevation and azimuth
```

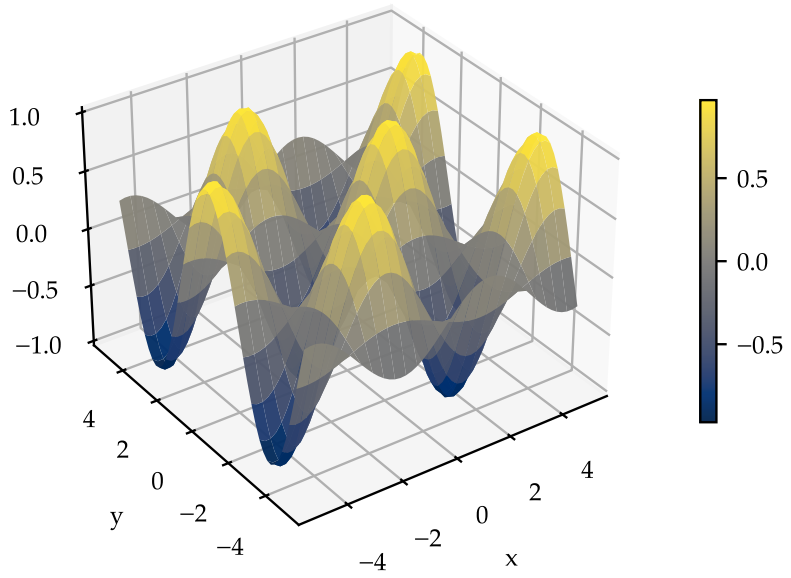


Figure 12.1 A surface plot of $z = \sin x \cos y$ with the `cividis` colormap.
图 12.1 $z = \sin x \cos y$ 的曲面图，颜色映射为 cividis。

`X` and `Y` have shape `(len(y), len(x))`: each row of `X` is a copy of `x`, and each column of `Y` is a copy of `y`. Consequently `Z` has the same shape, and `Z[i,j]` corresponds to the point (x_j, y_i). This row/column convention is the most common source of confusion.

中文要点

- `meshgrid` 的输出形状是 `(len(y), len(x))`，不是 `(len(x), len(y))`；因为第一个维度对应行（y），第二个维度对应列（x）。
- 若把 `Z` 转置或把 `X,Y` 顺序写反，曲面会旋转 90° 或出现奇怪的条纹。
- `plot_surface` 的 `cmap` 把高度映射成颜色；`fig.colorbar(surf)` 给出颜色标尺，否则读者不知道黄色代表什么。
- `view_init(elev, azim)` 控制相机：`elev` 是仰角，`azim` 是方位角。换一个视角，遮挡关系会完全不同。
- 网格太密会让图变慢、文件变大；`0.2` 的步长在 $[-5, 5]$ 上已经足够平滑。

12.4 Reading a 3D plot critically

批判性地读三维图

3-D plots are visually attractive but easy to mislead. Perspective distorts lengths; a surface can hide points behind it; colour can imply a precision that the data do not have. Use a 3-D plot when the third dimension is genuinely a coordinate, not when you are trying to make a 2-D result look impressive. For quantitative reading, a 2-D contour plot or a heat map is often better.

What to rehearse

Explain what `np.meshgrid` returns and why `Z.shape == X.shape`. Write the four lines that create a 3-D axes, plot a surface with a colormap, add a colorbar, and set the labels. Given a surface that looks rotated or striped, diagnose the likely meshgrid/transpose error. State one advantage and one disadvantage of 3-D plots.

本章考点清单

1. 写出创建 3D axes 的两种方式。
2. 区分 `plot3D`、`scatter`、`plot_surface` 的输入。
3. 解释 `meshgrid` 的输出形状与索引约定。
4. 说明 `cmap` 与 `colorbar` 的配合。
5. 用 `view_init` 调整视角，并说明其两个参数。
6. 指出三维图在定量阅读上的两个风险。

13 Maps with Cartopy

地图可视化：经纬度、投影与地理要素

A map is a 2-D plot with a geographical projection. The data are positions on a sphere, but the figure is flat, so every map distorts something: area, angle, distance or shape. Cartopy provides the projections, the coastlines and the coordinate transforms that make the distortion explicit and controllable.

> **中文精讲** Chinese companion
>
> 地图 = 二维坐标图 + 投影。经度（longitude）范围 −180° 到 180°，向东为正；纬度（latitude）范围 −90° 到 90°，向北为正。经纬度是球面坐标，画到平面上必须选一个投影，常见的有 Plate Carrée（等距圆柱）、Mercator（墨卡托）和 Robinson。课件里的工具链：Basemap 已弃用 → Cartopy 接替 → Folium 做交互式网页地图。考试常考：经纬度范围、`set_extent` 的参数顺序、`transform=ccrs.PlateCarree()` 的作用、`coastlines` 与 `add_feature` 的区别。

13.1 Coordinates and projections

经纬度与投影

Longitude measures east–west position, from $-180°$ (west) through $0°$ (Greenwich) to $180°$ (east). Latitude measures north–south position, from $-90°$ (South Pole) through $0°$ (equator) to $90°$ (North Pole). A location is written (lon, lat), in that order — the opposite of the usual (x, y) convention in some GIS files, so always check.

A projection maps (λ, φ) on the sphere to (x, y) on the page. **Plate Carrée** simply uses longitude and latitude as x and y: easy to read, but areas near the poles are stretched. **Mercator** preserves angles (useful for navigation) but exaggerates high-latitude areas. **Robinson** compromises, keeping the whole world visually balanced.

> **中文要点**
>
> 经纬度的顺序是先经度后纬度，和 (x, y) 的直觉不同。`set_extent([lon_min, lon_max, lat_min, lat_max])` 的顺序也是“先经度后纬度、先小后大”。如果图和预期上下颠倒，多半是把纬度的两个数写反了；如果左右颠倒，多半是经度写反了。投影没有“正确”的，只有“适合用途”的：导航用 Mercator，全球概览用 Robinson，快速绘图用 Plate Carrée。

13.2 Cartopy basics

Cartopy 基本用法

Cartopy replaced Basemap (deprecated after 2016). It integrates with Matplotlib: you create an axes with a Cartopy projection, then use the ordinary plotting commands.

```
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature

fig = plt.figure(figsize=(9, 7))
ax = plt.axes(projection=ccrs.PlateCarree())
```

```
ax.set_extent([-123.5, -121.5, 37.0, 38.5],
              crs=ccrs.PlateCarree())      # lon_min, lon_max, lat_min, lat_max

ax.add_feature(cfeature.LAND, facecolor="#f4f1ea")
ax.add_feature(cfeature.OCEAN, facecolor="#dce9e7")
ax.coastlines(resolution="10m", linewidth=0.6)
ax.add_feature(cfeature.BORDERS, linewidth=0.5)
ax.add_feature(cfeature.LAKES, edgecolor="gray")
ax.add_feature(cfeature.RIVERS, edgecolor="gray")

gl = ax.gridlines(draw_labels=True, linestyle=":")
gl.top_labels = False
gl.right_labels = False

ax.plot([-122.42], [37.77], "ro", markersize=8,
        transform=ccrs.PlateCarree())      # San Francisco
ax.set_title("San Francisco Bay Area")
plt.show()
```

The `transform=ccrs.PlateCarree()` argument tells Cartopy that the data coordinates are longitude/latitude, so they can be projected onto whatever axes projection you chose. Without it, a point may land in the wrong place when the map projection is not Plate Carrée. For a quick background, `ax.stock_img()` adds a low-resolution Natural Earth image; higher-resolution coastlines come from `resolution='10m'`.

中文要点

- `plt.axes(projection=ccrs.PlateCarree())` 创建地图坐标系；之后的 `plot`、`scatter` 都画在这张地图上。
- `set_extent` 只显示指定窗口，参数顺序是 $[lon_{min}, lon_{max}, lat_{min}, lat_{max}]$。
- `coastlines(resolution='10m')` 画海岸线，分辨率有 110m、50m、10m；10m 最细，也最慢。
- `add_feature(cfeature.LAND)` 等填充陆地、海洋、湖泊、河流、国界；`cfeature` 是 Cartopy 的内置地理要素集合。
- `gridlines(draw_labels=True)` 画经纬网并标注；`top_labels=False` 去掉上边重复的标签。
- 数据点必须带 `transform=ccrs.PlateCarree()`，否则在非等距投影下位置会错。

13.3 Basemap, Cartopy and Folium

三种地图工具的分工

Tool	Status	Strength	Use when
Basemap	deprecated (post-2016)	legacy code only	maintain old notebooks
Cartopy	active	projections, features, Matplotlib integration	publication-quality static maps
Folium	active	interactive Leaflet maps in HTML	exploration, web dashboards

Table 13.1 Map tools in the course; Folium is optional but useful for interactive output.

中文要点

Basemap 是“上一代”工具，2016 年后不再维护；新代码不要从它开始。Cartopy 与 Matplotlib 无缝集成，适合论文插图：投影、海岸线、国界、河流、经纬网都能精细控制。Folium 生成的是 HTML/JavaScript 地图，可以缩放、点击、弹窗，适合探索和网页展示，但不适合印刷。三者不是互斥的：可以用 Cartopy 出静态图，用 Folium 做交互版本。

What to rehearse

State the ranges and order of longitude and latitude. Write the three lines that create a Plate Carrée axes, set an extent, and add coastlines. Explain why `transform=ccrs.PlateCarree()` is needed for data points. Name one advantage of Cartopy over Basemap and one advantage of Folium over Cartopy.

本章考点清单

1. 写出经度、纬度的范围，并说明 (lon, lat) 的顺序。
2. 解释 `set_extent` 四个参数的顺序。
3. 用 Cartopy 创建地图、加海岸线、加陆地/海洋、加经纬网。
4. 说明 `transform` 在投影中的作用。
5. 比较 Basemap、Cartopy、Folium 的定位。
6. 指出 Plate Carrée 与 Mercator 各自的失真特点。

14 Animations

动画：把时间维度画出来

An animation is a sequence of still frames shown rapidly enough to create the illusion of motion. In scientific computing it is not entertainment: a parameter sweep, a wave propagating, or a solver converging is often understood in five seconds of motion than in fifty static plots.

> **中文精讲** Chinese companion
>
> 动画的本质是“一帧一帧地改图”。Matplotlib 的 `animation` 模块提供框架：`Animation` 是基类，`FuncAnimation` 是最常用的具体类，它反复调用一个 `update(frame)` 函数，每次只改动画对象的数据，然后重绘。三个必备部件：
>
> 1. `init()`：画静态背景（坐标轴、曲线、标签）；
> 2. `update(frame)`：根据帧号更新会动的对象（如红点位置），并返回它；
> 3. `writers['ffmpeg']` 或 `ani.save(...)`：把帧编码成 mp4/gif。
>
> 考试常考：解释 `frames`、`interval`、`blit` 的含义；说明 `init_func` 与 `update` 的分工；为什么保存 mp4 需要 ffmpeg；如何让动画在 Jupyter 中直接显示。

14.1 The frame model

帧模型：背景不动，对象在动

The efficient pattern is to draw the background once, keep a handle to the moving artist, and update only its data. For a point moving along a sine wave, the background is the blue curve; the moving object is a red circle whose position is set from the current frame index.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

n = 1000
x = np.linspace(0, 6 * np.pi, n)
y = np.sin(x)

fig, ax = plt.subplots(figsize=(8, 4))
ax.set_xlim(0, 6 * np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel("x"); ax.set_ylabel("sin(x)")

ax.plot(x, y, "b-", lw=1)                       # static background
red_circle, = ax.plot([], [], "ro", ms=8)       # empty moving artist

def init():
    red_circle.set_data([], [])
    return (red_circle,)

def update(frame):
    red_circle.set_data([x[frame]], [y[frame]])
    return (red_circle,)

ani = animation.FuncAnimation(
```

```
    fig, update, frames=n, init_func=init,
    interval=15, blit=True)

Writer = animation.writers["ffmpeg"]
writer = Writer(fps=15, metadata={"artist": "PHYB57"}, bitrate=1800)
ani.save("sine.mp4", writer=writer, dpi=100)
plt.show()
```

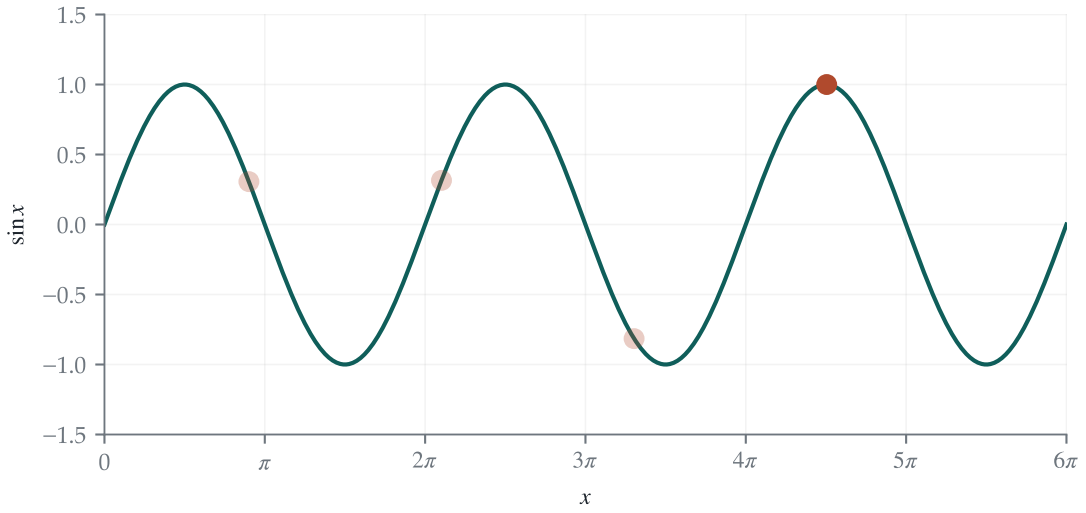


Figure 14.1 Four frames of the sine-wave animation: the solid dot is the current frame, the faint dots are earlier positions.

图 14.1 正弦波动画的四个帧：实心点是当前帧，浅色点是此前位置。

中文要点

- `frames=n` 表示调用 `update(0)` 到 `update(n-1)` 共 n 帧；也可以传一个列表或生成器。
- `interval=15` 是帧间隔，单位毫秒；15 ms 约等于 67 fps，实际播放速度还受编码器和播放器影响。
- `blit=True` 只重绘发生变化的像素，速度快，但要求 `update` 返回所有被修改的 artist；若返回不完整，画面会残留旧内容。
- `init_func` 只调用一次，用来清空/初始化；`update` 每帧调用。
- `ani.save` 需要 ffmpeg 可执行文件；没有 ffmpeg 时可用 `PillowWriter` 存 gif，或在 Jupyter 中用 `HTML(ani.to_jshtml())` 直接播放。

14.2 Metadata, writers and formats

元数据、写入器与格式

The writer controls the encoding. `animation.writers['ffmpeg']` returns a class; instantiate it with `fps`, `bitrate` and a metadata dictionary (title, artist, comment). The `ani.save` call then writes the frames through that writer.

```
print(animation.writers.list())          # available writers on this machine
writer = animation.FFMpegWriter(fps=30, bitrate=2400)
ani.save("wave.mp4", writer=writer, dpi=150)

from matplotlib.animation import PillowWriter
ani.save("wave.gif", writer=PillowWriter(fps=15))
```

For notebooks, `HTML(ani.to_jshtml())` embeds a JavaScript player without writing a file. For

publication, mp4 (H.264) is smaller and smoother than gif; for a quick check, gif is enough.

中文要点

`fps` 是每秒帧数，`bitrate` 是码率；码率越高越清晰，文件也越大。mp4 适合演讲和网页嵌入，gif 适合聊天工具但颜色和体积都吃亏。若 `ani.save` 报 `FileNotFoundError: ffmpeg`，说明系统没有安装 ffmpeg，安装后重启内核即可；或者改用 `PillowWriter`。动画的“流畅”取决于帧率与每帧的绘制成本，不是越多帧越好。

14.3 Designing an animation that teaches

让动画真正解释问题

- **Keep the background fixed.** A moving axis makes the viewer lose the reference frame.
- **Show the parameter.** If the animation varies γ, print γ on the frame; otherwise the motion is unexplained.
- **Limit the duration.** Ten to twenty seconds is usually enough; a loop that never ends is ignored.
- **Provide a static fallback.** A paper figure or a few key frames should convey the same point when the video cannot be played.
- **Check the first and last frames.** They are the ones people screenshot.

What to rehearse

Name the three components of a `FuncAnimation` (figure, update function, frames) and the role of `init_func`. Explain `interval`, `blit`, `fps`. Write the minimal code that animates a point along a curve and saves it as mp4. State what ffmpeg is for and what to do if it is missing.

本章考点清单

1. 定义动画，并说明为什么它比静态图更适合动态系统。
2. 写出 `FuncAnimation` 的四个关键参数。
3. 区分 `init_func` 与 `update`，说明 `blit` 的代价与收益。
4. 用 writer 保存 mp4，并解释 fps、bitrate、dpi 的作用。
5. 说明没有 ffmpeg 时的两个替代方案。
6. 列出三条“教学动画”的设计原则。

15 A Checklist for Scientific Figures

科学图表的检查清单

A figure is finished not when it looks pretty but when a reader can answer the intended question without asking you anything. The checklist below is deliberately short; it catches most of the defects that appear in student reports and exam answers.

中文精讲 Chinese companion

这一章不是新的 API，而是一套“审图”标准。考试可能给你一张图，让你指出问题；也可能要求你画图并说明为什么这样画。记住一个原则：图是为读者服务的，不是为代码服务的。每一个视觉元素都应该有信息上的理由；没有理由的网格、阴影、3D 效果、彩虹色，都是噪声。

15.1 The ten-point check

十项检查

1. **Question.** Does the figure answer one clear question? If it tries to answer five, split it.
2. **Axes.** Are both axes labelled with the quantity *and* its unit? Is the scale (linear/log) stated when it is not obvious?
3. **Range.** Do the limits show the relevant variation without hiding outliers or exaggerating noise?
4. **Legend.** Is every curve identified? Is the legend necessary, or could direct labels be clearer?
5. **Colour.** Does the palette remain distinguishable in greyscale and for colour-blind readers? Avoid rainbow maps for ordered quantities; use a perceptually uniform map such as `viridis` or `cividis`.
6. **Data–ink ratio.** Remove gridlines, borders, shadows and 3-D effects that carry no information.
7. **Uncertainty.** Are error bars, confidence bands or the number of samples shown where they matter?
8. **Resolution.** Is the export at least 300 dpi for print, with fonts legible at final size?
9. **Consistency.** Same quantity, same colour and same unit across all panels; same significant figures.
10. **Caption.** Can the caption be understood without the main text? It should state what is plotted, the conditions, and the take-away.

中文要点

这十条可以压缩成三句话：**坐标轴说清楚“是什么、什么单位”；图例/颜色说清楚“哪条线是谁”；标题/图注说清楚“所以呢”**。另外两条最容易被忽略：误差棒（没有不确定性的测量值几乎总是不完整的）和色盲友好（红绿对比是最常见的坏选择）。灰度打印测试是一个很快的检查：把图转成黑白，如果两条线分不清，颜色就没有承担起信息功能。

15.2 Choosing the right chart

选对图型

Question	Chart	Why
How does y change with x?	line / scatter	preserves order and spacing
How are values distributed?	histogram / box plot	shows spread, skew, outliers
How do categories compare?	bar chart	length is easy to compare
How do two variables relate?	scatter + fit	reveals correlation and residuals
How does a function of two variables look?	surface / contour	shows topography; contour is better for exact reading
How does a quantity evolve in time?	line / animation	time on the horizontal axis

Table 15.1 Match the chart to the question, not to the available function.

Five recurring defects

- A line plot connecting categorical points, implying an order that does not exist.
- A truncated y-axis that turns a 2% difference into a dramatic cliff (sometimes justified, but must be labelled).
- A 3-D pie chart, or any pie chart with more than five slices.
- A rainbow colormap on a quantity with no natural hue ordering.
- A legend that repeats the axis label, or a title that says nothing ("Results").

What to rehearse

Be ready to critique a figure in three sentences: what it shows, what is missing, what you would change. Know which chart type matches a given question, and why a perceptually uniform colormap is preferred. In a plotting question, marks are usually lost for missing units and legends rather than for the wrong colour.

本章考点清单

1. 用"问题–坐标轴–图例–图注"四问检查一张图。
2. 说明误差棒与样本量的作用。
3. 解释为什么有序数据不适合彩虹色，推荐 `viridis/cividis`。
4. 为"分布、比较、相关、二元函数、时间演化"各选一种图。
5. 指出至少三个常见坏图特征。

PART

05

Fitting

最小二乘回归

Chapter 16. The best model is the one whose residuals are orthogonal to everything it can explain.

16 Least Squares Regression

最小二乘回归：在噪声中找一条最合理的线

Regression starts from a different premise than interpolation: the data are noisy, so the curve should *not* pass through every point. Instead, we choose a model with a few parameters and select the parameters that make the overall mismatch as small as possible. The least-squares criterion measures mismatch by the sum of squared residuals, which is convenient analytically and statistically.

> **中文精讲** Chinese companion
> 第 16 章回答一个问题：已知一组带噪声的 (x_i, y_i)，如何估计模型参数？课件用弹簧实验切入：胡克定律 $F = kx$，但测得的 F 有噪声，直接取某一点算 k 不可靠。最小二乘的做法是：先假定模型 $\hat{y}(x) = \sum_j \alpha_j f_j(x)$，再让所有点的残差平方和
>
> $$E = \sum_{i=1}^{m} \big(y_i - \hat{y}(x_i)\big)^2$$
>
> 最小。注意“平方”有两个作用：正负残差都变成正数，且大残差被惩罚得更重。考试重点：写出模型与误差、说明基函数必须线性无关、把问题写成矩阵形式 $E = \|Y - A\alpha\|^2$、解释为什么最小二乘不是“穿过所有点”。

16.1 Why not just fit every point?

为什么不直接穿过所有点

If the data were exact, interpolation would be the natural choice: a curve through every point, no compromise. Real measurements contain noise, and a curve that follows every fluctuation is fitting the noise, not the physics. A model with fewer parameters than data points acts as a filter: it captures the trend and leaves the random part in the residuals.

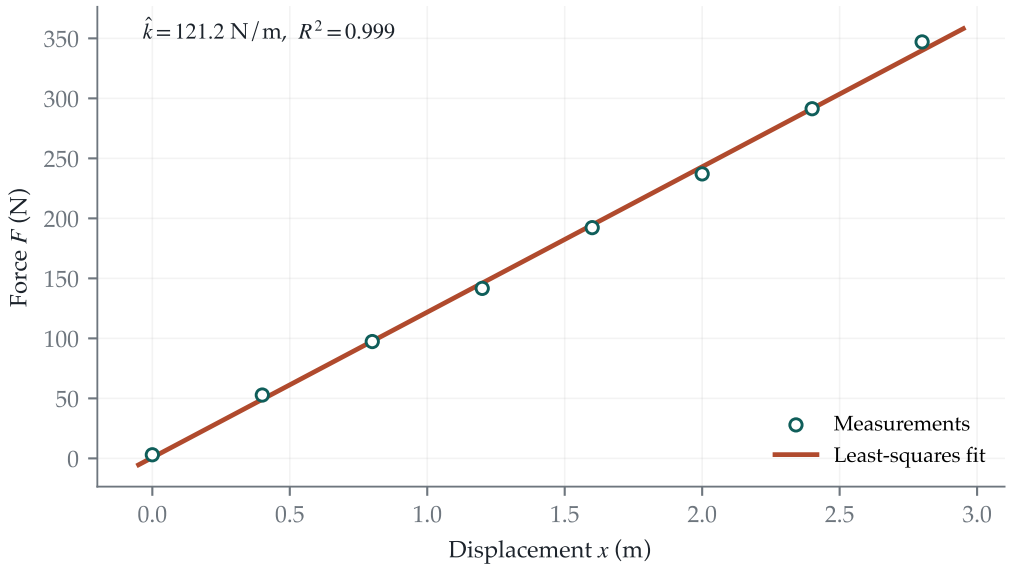


Figure 16.1 A spring calibration: measurements (circles), least-squares line (solid), and residuals (thin vertical lines).

图 16.1 弹簧标定：测量点、最小二乘拟合直线，以及每个点的残差（细竖线）。

> **中文要点**
> 插值说“我相信每个数据点”，回归说“我相信模型，数据点有误差”。当测量误差不可忽略时，强行穿过所有点会让曲线剧烈摆动，预测新点时反而更差。最小二乘用“全局误差最小”换取“局部不完全贴合”，这是一种偏差–方差权衡：模型越复杂，拟合越贴合但越容易过拟合。

16.2 The estimation function

估计函数：基函数的线性组合

The general model is

$$\hat{y}(x) = \sum_{j=1}^{n} \alpha_j f_j(x),$$

where α_j are unknown parameters and $f_j(x)$ are **basis functions**. The basis functions must be linearly independent: no one of them can be written as a linear combination of the others. Otherwise the parameters are not identifiable — different parameter sets give the same curve.

Model	Basis functions	Parameters
straight line	$f_1 = 1,\ f_2 = x$	intercept, slope
quadratic	$f_1 = 1,\ f_2 = x,\ f_3 = x^2$	three coefficients
polynomial degree d	$1, x, \ldots, x^d$	$d + 1$ coefficients
spring $F = kx$	$f_1 = x$	k only
exponential $y = ae^{bx}$	$f_1 = e^{bx}$ (nonlinear in b)	a, b

For the spring, the physics already tells us the intercept should be zero, so the model is $\hat{F}(x) = kx$: one basis function $f_1(x) = x$, one parameter $\alpha_1 = k$. If the sensor has an offset, we add $f_2(x) = 1$ and estimate both k and the offset.

中文要点

“线性最小二乘”的“线性”指的是对参数线性，不是对 x 线性。$f_j(x)$ 可以是 x^2、$\sin x$、e^x，只要模型是 α_j 的线性组合，就能用同一套矩阵公式求解。反之，$y = ae^{bx}$ 对 b 是非线性的，不能直接套线性公式，需要取对数或数值优化（第 19 章）。

16.3 The residual and the objective

残差与目标函数

For each data point, the residual is

$$e_i = y_i - \hat{y}(x_i) = y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i).$$

Stacking the residuals into a column vector $e = (e_1, \dots, e_m)^\top$, the objective is

$$E = \sum_{i=1}^{m} e_i^2 = \|e\|_2^2 = \|Y - A\alpha\|_2^2,$$

where $Y = (y_1, \dots, y_m)^\top$ and A is the $m \times n$ **design matrix**

$$A = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix}.$$

Each row corresponds to a data point; each column to a basis function. The model predictions are $\hat{Y} = A\alpha$.

中文要点

矩阵 A 的形状是 $m \times n$：m 个数据点、n 个基函数。第 i 行是“第 i 个点的所有基函数值”，第 j 列是“第 j 个基函数在所有点的值”。写代码时最容易错的是把 A 转置；记住 `A.shape == (len(x), n_basis)`，`A @ alpha` 必须得到长度 m 的预测向量。

16.4 The spring example in full

弹簧例子的完整写法

Suppose we measure force F_i at displacements x_i. The model is $F = kx$, so

$$A = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix}, \qquad \alpha = (k), \qquad E = \sum_i (F_i - kx_i)^2.$$

Setting $dE/dk = 0$ gives the one-line solution

$$\hat{k} = \frac{\sum_i x_i F_i}{\sum_i x_i^2}.$$

This is the least-squares slope through the origin. If we include an intercept, the design matrix gains a column of ones and the solution becomes the usual two-parameter line fit.

中文要点

过原点直线的最小二乘斜率是 $\hat{k} = \sum x_i F_i / \sum x_i^2$，不是 $\sum F_i / \sum x_i$，也不是各点斜率 F_i/x_i 的平均。前者是“让平方误差最小”的解，后两者是别的准则。这个小题经常出现在考试中，要求你从 $dE/dk = 0$ 推一遍。

What to rehearse

Write the model, the residual, the objective and the design matrix for a straight line and for the spring. Explain why the basis functions must be linearly independent. State in one sentence the difference between regression and interpolation. Be able to derive $\hat{k} = \sum x_i F_i / \sum x_i^2$.

本章考点清单

1. 写出最小二乘的模型、残差、平方误差。
2. 说明“线性”是对参数线性，而非对 x 线性。
3. 构造设计矩阵 A，并检查其形状。
4. 解释基函数线性无关的必要性。
5. 推导过原点弹簧的 $\hat{k}$。
6. 用一句话区分回归与插值。

17 Deriving the Normal Equations

正规方程：代数与微积分两条路

The least-squares problem can be solved in two apparently different ways. Linear algebra asks for the point in the column space of A closest to Y; calculus differentiates the squared error with respect to each parameter and sets the derivatives to zero. Both lead to the same **normal equations** $A^\top A\alpha = A^\top Y$. Seeing why they agree is more valuable than memorising the final formula.

> **中文精讲** Chinese companion
>
> 这一节是第 16 章的理论核心。两条推导路线：
>
> 1. **线性代数**：$A\alpha$ 只能落在 A 的列空间里，要使它离 Y 最近，误差向量必须与列空间垂直，即 $A^\top(Y - A\alpha) = 0$。
> 2. **多变量微积分**：把 E 对每个 α_k 求偏导并令其为零，得到 n 个方程，整理成矩阵就是 $A^\top A\alpha = A^\top Y$。
>
> 两条路得到同一个方程，不是巧合：前者说“残差与所有基函数正交”，后者说“在所有方向上的斜率为零”，这是同一个驻点条件的两种语言。考试常考：完整推导、解释 $A^\top A$ 可逆的条件、说明为什么用 `lstsq` 而不是显式求逆。

17.1 The linear-algebra route

线性代数路线：投影

Let $Y \in \mathbb{R}^m$ be the vector of observations and $A \in \mathbb{R}^{m\times n}$ the design matrix. The set of all possible predictions is the column space

$$C(A) = \{A\alpha : \alpha \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

Minimising $\|Y - A\alpha\|^2$ means finding the point $A\hat{\alpha}$ in this subspace that is closest to Y. The closest point is the orthogonal projection of Y onto $C(A)$. Therefore the residual

$$r = Y - A\hat{\alpha}$$

must be orthogonal to every column of A:

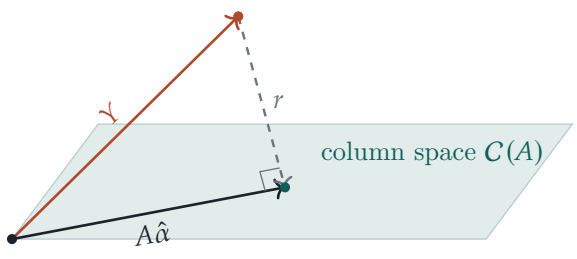
$$A^\top r = 0 \quad\Longleftrightarrow\quad A^\top(Y - A\hat{\alpha}) = 0.$$

Rearranging,

$$\boxed{A^\top A\,\hat{\alpha} = A^\top Y}$$

and, if $A^\top A$ is invertible,

$$\boxed{\hat{\alpha} = (A^\top A)^{-1}A^\top Y.}$$



The residual is perpendicular to the plane; the fit is the shadow of Y on the column space.

> **中文要点**
> 把 A 的每一列想成一个“可解释的方向”。$A\alpha$ 是所有列的线性组合，只能在一个平面/子空间里移动。Y 通常不在这个子空间里，最近的点就是垂直投影；垂线方向就是残差。因此“残差与每个基函数正交”是最小二乘的几何本质，也是 $A^\top r = 0$ 的来源。这个视角还能解释：如果 A 的列线性相关，投影平面会“塌陷”，参数不唯一，但投影点仍然唯一。

17.2 The calculus route

微积分路线：偏导为零

Write the error explicitly:

$$E(\alpha_1, \ldots, \alpha_n) = \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right)^2 .$$

For each parameter α_k, differentiate with respect to α_k. Only the term $-\alpha_k f_k(x_i)$ inside the bracket depends on α_k, so

$$\frac{\partial E}{\partial \alpha_k} = 2 \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right) \big(-f_k(x_i) \big) = 0.$$

Cancelling the -2:

$$\sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right) f_k(x_i) = 0, \qquad k = 1, \ldots, n.$$

This is exactly the statement that the residual is orthogonal to the k-th basis column. Stacking the n equations gives

$$A^\top A \alpha = A^\top Y.$$

The matrix $A^\top A$ is $n \times n$, symmetric and positive semidefinite; it is positive definite (hence invertible) exactly when the columns of A are linearly independent.

> **中文要点**
> 求导时注意：括号内对 α_k 的导数是 $-f_k(x_i)$，不是 -1；因为第 k 个参数乘的是基函数 f_k。把 -2 约掉后，每个 k 给出一个方程，共 n 个方程、n 个未知数。$A^\top A$ 对称半正定；若列满秩则正定可逆，若列相关则奇异，需要用伪逆或 `lstsq` 得到最小范数解。

17.3 The normal equations in practice

正规方程的实践含义

- **Do not invert explicitly.** Forming $(A^\top A)^{-1}$ squares the condition number; solving the normal equations directly is better, and QR/SVD is better still.
- **Use `np.linalg.lstsq`.** It solves the problem via a stable factorisation (typically SVD or QR) and returns the rank, singular values and residual sum of squares.
- **Check the rank.** If `rank < n`, the basis functions are dependent (or a column is all zeros), and the parameters are not unique.

- **Scale the columns.** A Vandermonde matrix with large x has columns of wildly different magnitudes; centring and scaling x improves conditioning.

```
import numpy as np
A = np.column_stack([x, np.ones_like(x)])
coef, residuals, rank, sv = np.linalg.lstsq(A, y, rcond=None)
print(coef)          # [slope, intercept]
print(rank, sv)      # rank and singular values
```

Why not just use the formula?

$(A^\top A)^{-1}A^\top Y$ is mathematically correct but numerically fragile. If A is ill-conditioned, $A^\top A$ has condition number $\kappa(A)^2$, so a condition number of 10^8 becomes 10^{16} — at the edge of double precision. `lstsq` avoids this by not forming $A^\top A$ explicitly.

中文要点

- 公式 $(A^\top A)^{-1}A^\top Y$ 用于推导和理解，不用于生产代码；显式求逆会把条件数平方，放大舍入误差。
- `np.linalg.lstsq(A, y, rcond=None)` 返回四个值：系数、残差平方和（仅当秩亏时为空）、秩、奇异值。考试若要求“用 NumPy 求解”，写这一行即可。
- 若 `rank < n`，说明基函数线性相关（例如同时放了 x 和 $2x$），参数不唯一；此时 `lstsq` 返回最小范数解。
- 多项式拟合前把 x 中心化/缩放（$x \mapsto (x - \bar{x})/s$）能显著改善数值条件，这是实践中的常用技巧。

What to rehearse

Derive $A^\top A\alpha = A^\top Y$ both ways, without looking. Explain the geometric meaning of $A^\top r = 0$. State the condition for invertibility. Explain why `lstsq` is preferred over the explicit inverse. Given A and Y, write the normal equations and identify the dimensions.

本章考点清单

1. 写出最小二乘的投影解释，并画出残差垂直的示意图。
2. 从 $\partial E/\partial \alpha_k = 0$ 推到正规方程。
3. 说明 $A^\top A$ 的尺寸、对称性、正定条件。
4. 解释秩亏时会发生什么，如何处理。
5. 说明为什么生产代码用 `lstsq` 而非显式求逆。
6. 解释中心化/缩放对条件数的改善。

18 Least Squares in Python

用 Python 做最小二乘：从数据到系数

The theory gives a formula; the practice gives three functions. `np.linalg.lstsq` solves linear least squares for an arbitrary design matrix, `np.linalg.pinv` computes a pseudo-inverse when the matrix is rank-deficient, and `scipy.optimize.curve_fit` handles nonlinear models by iterative optimisation. This chapter walks through the workflow on a spring calibration and interprets the

output.

> **中文精讲** Chinese companion
>
> Python 里做线性最小二乘的标准流程只有四步：
>
> 1. 准备数据 x, y（一维数组，长度相同）；
> 2. 构造设计矩阵 A（每一列一个基函数）；
> 3. 调用 `np.linalg.lstsq(A, y, rcond=None)`；
> 4. 用系数预测、画图、检查残差。
>
> 考试常考：写出这四步、解释 `lstsq` 的四个返回值、说明 `np.polyfit` 与 `lstsq` 的关系、计算 R^2、判断拟合是否合理。

18.1 A complete linear fit

完整的直线拟合

Suppose a spring obeys $F = kx + b$ with true $k = 120$ N/m and a small sensor offset, but the measurements carry noise. We fit both parameters, even though the physics suggests $b = 0$, to see whether the offset is significant.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
x = np.array([0.0, 0.4, 0.8, 1.2, 1.6, 2.0, 2.4, 2.8])
F = 120.0 * x + 3.0 + rng.normal(0, 6.0, size=x.size)

A = np.column_stack([x, np.ones_like(x)])      # columns: x and 1
coef, residuals, rank, sv = np.linalg.lstsq(A, F, rcond=None)
k_hat, b_hat = coef
print(k_hat, b_hat)          # 121.154... 0.701...
print(rank, sv)              # 2 [5.34 1.37]

F_hat = A @ coef
ss_res = np.sum((F - F_hat)**2)
ss_tot = np.sum((F - F.mean())**2)
r2 = 1 - ss_res / ss_tot
rmse = np.sqrt(ss_res / len(F))
print(r2, rmse)              # 0.99869... 4.02...
```

The fitted slope is 121.15 N/m, close to the true 120; the intercept 0.70 N is small compared with the noise level. The coefficient of determination

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

is 0.9987: 99.87% of the variation in F is explained by the linear model. R^2 is dimensionless and at most 1 for a model with an intercept; it can be negative for a badly chosen model evaluated on new data.

> **中文要点**
>
> - `np.column_stack([x, np.ones_like(x)])` 生成两列：第一列 x，第二列全 1。顺序决定系数顺序：先斜率、后截距。
> - `lstsq` 返回 `(coef, residuals, rank, sv)`；`residuals` 是残差平方和（不是残差向量），且当 $m \le n$ 或秩亏时为空数组。

- R^2 越接近 1 并不自动意味着模型正确；它只衡量对当前数据的解释程度。加入高次项总能提高 R^2，所以要看残差结构和预测能力。
- RMSE 与 y 同单位，便于物理解释；R^2 无量纲，便于跨数据集比较。

18.2 Closed forms and `polyfit`

闭式解与 polyfit

For a straight line $y = mx + b$, the normal equations have the explicit solution

$$m = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}, \qquad b = \frac{\sum y_i - m\sum x_i}{n}.$$

For a polynomial of degree d, the design matrix is the Vandermonde matrix $[1, x, x^2, \ldots, x^d]$. NumPy wraps this in one call:

```
m, b = np.polyfit(x, F, 1)          # highest power first
p = np.polyfit(x, F, 3)             # cubic coefficients
F_fit = np.polyval(p, x)
```

`np.polyfit` returns coefficients from highest to lowest power, the opposite of the column order of a Vandermonde matrix. It is convenient but can be ill-conditioned for high degree or badly scaled x; for serious work, centre and scale x, or use `lstsq` with an explicit basis.

中文要点

闭式解只适用于直线；次数一高，公式就变成解 $A^\top A$ 的线性系统，手算不现实。`np.polyfit(x,y,d)` 返回的系数顺序是“从高次到低次”，例如二次拟合返回 $[a, b, c]$ 对应 $ax^2 + bx + c$；而设计矩阵的列顺序通常是“从低次到高次”。这个顺序差异是考试和代码中的常见错误。高次多项式在 x 很大或很分散时条件数很差，建议先中心化/缩放。

18.3 Reading the output critically

批判性地读拟合结果

- **Residual plot.** Plot $y_i - \hat{y}_i$ against $\hat{y}_i$ or x_i. A pattern (curvature, funnel, clusters) means the model is missing structure; random scatter supports the fit.
- **Parameter uncertainty.** For a linear model with independent Gaussian noise, $\mathrm{Cov}(\hat{\alpha}) = s^2(A^\top A)^{-1}$ with $s^2 = \mathrm{SS}_{\mathrm{res}}/(m-n)$. The diagonal square roots are standard errors.
- **Extrapolation.** A fit is supported only over the range of x used. Extending a spring calibration beyond the measured displacements is a physical assumption, not a mathematical guarantee.
- **Leverage.** A single point far from the others can dominate the fit. Always inspect the data, not only the summary numbers.

```
resid = F - F_hat
plt.axhline(0, color="gray", lw=0.8)
plt.plot(F_hat, resid, "o")
plt.xlabel("Fitted force (N)"); plt.ylabel("Residual (N)")
```

R^2 is not a scoreboard

Adding a basis function can never decrease R^2 on the training data; it can always be made exactly 1 by using as many parameters as data points. Prefer adjusted R^2, cross-validation, or a residual analysis. A model with $R^2 = 0.99$ can still be wrong outside the observed range.

What to rehearse

Write the four-step workflow. Explain each return value of `lstsq`. Compute R^2 from $\mathrm{SS}_{\mathrm{res}}$ and $\mathrm{SS}_{\mathrm{tot}}$. State the coefficient order of `polyfit` versus a Vandermonde matrix. Give two reasons a high R^2 does not prove a good model.

本章考点清单

1. 写出数据 → 设计矩阵 → `lstsq` → 预测/画图的四步。
2. 解释 `coef, residuals, rank, sv` 的含义。
3. 写出直线拟合的闭式解，并与 `polyfit` 对照。
4. 计算 R^2、RMSE，并解释它们的区别。
5. 说明残差图能揭示什么。
6. 解释为什么高 R^2 不等于好模型。

19 Nonlinear Least Squares

非线性最小二乘：取对数还是直接拟合

Many physical laws are not linear in their parameters: exponentials, powers, saturating curves, Gaussians. There are two strategies. The first *linearises* the model with a transformation (usually a logarithm) and then applies the linear machinery. The second fits the nonlinear model directly by iterative optimisation. The first is fast and transparent but changes the error criterion; the second is flexible but needs good starting values.

> **中文精讲** Chinese companion
> “非线性”有两种含义：对 x 非线性（如 x^2、$\sin x$）和对参数非线性（如 e^{bx} 中的 b）。前者仍是线性最小二乘，因为模型对 α_j 是线性的；后者才需要本章的方法。两条路线：
> - **对数线性化**：$y = ae^{bx} \Rightarrow \ln y = \ln a + bx$，把曲线变成直线；$y = bx^m \Rightarrow \ln y = m \ln x + \ln b$。
> - **直接非线性拟合**：用 `scipy.optimize.curve_fit` 最小化 $\sum (y_i - f(x_i;\theta))^2$。
>
> 考试常考：写出两种变换、说明变换后的误差准则变了、给出 `curve_fit` 的基本调用、解释初值 `p0` 的作用、讨论过拟合。

19.1 Logarithmic linearisation

对数线性化：把曲线拉直

Exponential model. For

$$y = ae^{bx}, \qquad a > 0,$$

taking the natural logarithm gives

$$\ln y = \ln a + bx.$$

This is a straight line in the variables $(x, \ln y)$: the slope is b, the intercept is $\ln a$. Fit a line to the transformed data, then recover $\hat{a} = e^{\widehat{\ln a}}$.

Power model. For

$$y = bx^m, \qquad x > 0,\ b > 0,$$

taking logs gives

$$\ln y = m \ln x + \ln b.$$

Now the straight line is in $(\ln x, \ln y)$: the slope is m, the intercept is $\ln b$.

```
import numpy as np

# exponential: y = a * exp(b*x)
coef = np.polyfit(x, np.log(y), 1)     # slope b, intercept ln(a)
b_hat, ln_a = coef
a_hat = np.exp(ln_a)

# power: y = b * x**m
coef = np.polyfit(np.log(x), np.log(y), 1)
m_hat, ln_b = coef
```

```
b_hat = np.exp(ln_b)
```

> **中文要点**
> 取对数前必须保证 $y > 0$（指数/幂函数模型），幂函数还要求 $x > 0$。变换后的直线拟合最小化的是 $\sum(\ln y_i - \ln \hat{y}_i)^2$，不是原来的 $\sum(y_i - \hat{y}_i)^2$；这等价于给小的 y 更大的相对权重。如果测量误差是“加性”的（σ 与 y 无关），变换后的估计会有偏；如果误差是“乘性”的（相对误差恒定），对数变换反而更合适。

19.2 Direct fitting with `curve_fit`

直接拟合：curve_fit

`scipy.optimize.curve_fit` minimises the sum of squared residuals for an arbitrary callable $f(x;\theta)$. It needs an initial guess `p0`, because nonlinear least squares is a local, iterative problem.

```
import numpy as np
from scipy.optimize import curve_fit

def model(x, a, b):
    return a * np.exp(b * x)

popt, pcov = curve_fit(model, x, y, p0=[1.0, 0.1])
a_hat, b_hat = popt
perr = np.sqrt(np.diag(pcov))          # 1-sigma parameter errors
print(a_hat, b_hat, perr)

x_fit = np.linspace(x.min(), x.max(), 200)
y_fit = model(x_fit, *popt)
```

The covariance matrix `pcov` estimates parameter uncertainty under the assumption of independent, identically distributed Gaussian noise. It is not a confidence interval, and it says nothing about systematic error.

> **中文要点**
>
> - `p0` 是初值，顺序必须与 `model` 的参数顺序一致；初值太远可能不收敛或收敛到局部极小。
> - `curve_fit` 默认使用 Levenberg–Marquardt（无界）或 Trust Region Reflective（有 `bounds`）；可以用 `bounds=([a_min,b_min],[a_max,b_max])` 加物理约束。
> - `pcov` 的对角线开方是参数的标准误；若 `pcov` 出现 `inf`，说明拟合没有约束好或数据不足。
> - 直接拟合最小化的是原始 y 尺度的平方误差，和取对数后的结果一般不同；选择哪一种取决于噪声模型，而不是哪个 R^2 更高。

19.3 Polynomial regression and overfitting

多项式回归与过拟合

A polynomial of degree d,

$$\hat{y}(x) = \alpha_0 + \alpha_1 x + \cdots + \alpha_d x^d,$$

is linear in its coefficients, so it is still a linear least-squares problem with the Vandermonde design matrix. Increasing d makes the curve more flexible: it can reduce training error all the way to zero, but it may oscillate wildly between the data points.

```
for degree in [1, 3, 5, 9]:
    p = np.polyfit(x, y, degree)
    y_hat = np.polyval(p, x)
```

```
    rmse = np.sqrt(np.mean((y - y_hat)**2))
    print(degree, rmse)
```

This is the same phenomenon as the Runge example in interpolation: a high-degree polynomial fitted to equidistant points can overshoot near the ends. The cure is not a higher degree but a better model: fewer parameters, a spline, regularisation, or more data.

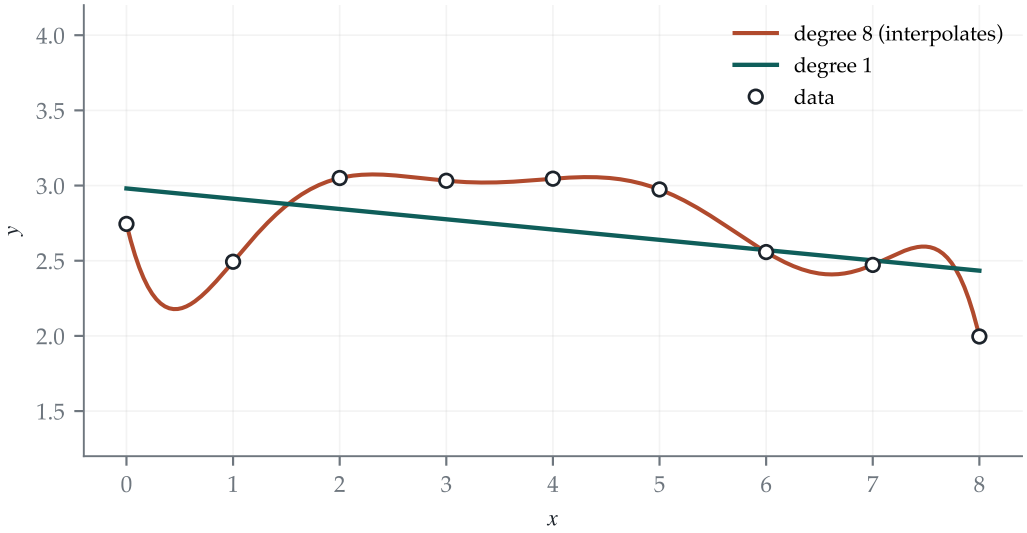


Figure 19.1 A degree-8 polynomial passes through all nine points but oscillates between them; the straight line captures the trend with less variance.
图 19.1 8 次多项式穿过全部 9 个点，却在点间剧烈振荡；直线反而更稳健。

> Overfitting checklist
> If the training RMSE keeps falling while the curve oscillates, if a small change in one data point moves the fit dramatically, or if predictions just outside the data range explode, you are overfitting. Prefer the simplest model consistent with the physics; validate on held-out data; use `p0` and `bounds` to encode known constraints.

19.4 Which method should you use?

方法选择

Situation	Method	Reason
$y = ae^{bx}$, multiplicative noise	log-linear	constant relative error becomes additive in logs
$y = ae^{bx}$, additive noise	`curve_fit`	preserves the y-scale error criterion
$y = bx^m$	log–log line	slope is the exponent, intercept is $\ln b$
smooth unknown curve	spline / low-degree polynomial	flexibility without global oscillation
parameters with physical bounds	`curve_fit` + `bounds`	constrains the search space

Table 19.1 Choose the fitting method from the noise model and the physics, not from convenience.

PART

06

Interpolation

插值方法

Chapter 17. A curve that must pass through every point — and the price of that promise.

20 The Interpolation Problem

插值问题：让曲线穿过每一个数据点

Interpolation estimates values *between* known data points by requiring the estimate to reproduce the known values exactly. It is the opposite compromise from regression: instead of smoothing away noise, it trusts every datum. That makes interpolation ideal for smooth, reliable data — a calibrated instrument, a tabulated function, a trajectory sampled densely — and dangerous for noisy measurements.

> **中文精讲** Chinese companion
> 插值的定义只有一句话：找一个函数 $\hat{f}$，使它对每个数据点都满足
>
> $$\hat{f}(x_i) = y_i \qquad (i = 0, 1, \ldots, n).$$
>
> 然后用它估计区间内任意 x 处的值。注意“区间内”四个字：插值只负责 $x_{\min} \le x \le x_{\max}$；超出这个范围叫外推（extrapolation），误差通常不可控。考试常考：插值与回归的区别、线性插值公式与手算、$n+1$ 个点为什么能唯一确定一个 n 次多项式、几种插值方法的优缺点。

20.1 Interpolation versus regression

插值与回归：两种哲学

	Interpolation	**Regression**
Passes through data?	yes, every point	generally no
Assumes a model?	only smoothness / form	a parametric model
Data quality	assumes reliable, smooth	tolerates noise
Parameters vs data	as many constraints as data	fewer parameters than data
Typical use	tables, calibrated curves	noisy experiments, prediction
Danger	noise is reproduced	model bias / underfitting

Table 20.1 Interpolation reproduces; regression estimates.

> **中文要点**
> 一句话区分：**回归相信模型，插值相信数据**。如果数据来自光滑的物理过程（例如小车随时间的位置），插值合理；如果每个点都有明显噪声，插值会把噪声也画进曲线，预测新点时反而更差。考试若问“为什么不用插值做实验数据拟合”，答案就是：它强迫曲线穿过每个带误差的点，导致过拟合与振荡。

20.2 Existence and uniqueness

存在性与唯一性

Given $n+1$ data points with distinct x_i, there is exactly one polynomial of degree at most n that passes through all of them. The proof is a linear-algebra count: write

$$p(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

The interpolation conditions $p(x_i) = y_i$ form the linear system

$$\begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

The matrix is the **Vandermonde matrix**; its determinant is $\prod_{i<j}(x_j - x_i)$, nonzero exactly when the nodes are distinct. Hence the coefficients, and therefore the polynomial, are unique.

> **中文要点**
> “$n+1$ 个点唯一确定一个次数不超过 n 的多项式”是插值理论的基石。直观上，$n+1$ 个约束对应 $n+1$ 个系数，正好一一对应；范德蒙行列式非零保证了解唯一。注意“次数不超过 n”而不是“恰好 n”：若数据恰好落在低次曲线上，高次项系数会是 0。节点重复（$x_i = x_j$）会让行列式为零，问题不适定。

20.3 Linear interpolation

线性插值：两点之间连直线

For $x_i < x < x_{i+1}$, linear interpolation assumes the curve is the straight line joining the two neighbouring points. By similar triangles,

$$\boxed{\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}}$$

Equivalently, with $t = (x - x_i)/(x_{i+1} - x_i) \in (0, 1)$,

$$\hat{f}(x) = (1 - t)y_i + t\, y_{i+1},$$

a weighted average of the two endpoint values.

```python
import numpy as np
x = np.array([0.0, 1.0, 2.0])
y = np.array([1.0, 3.0, 2.0])
print(np.interp(1.5, x, y))          # 2.5

from scipy.interpolate import interp1d
f = interp1d(x, y)                   # linear by default
print(f(1.5))                        # 2.5
```

For the lecture data $x = [0, 1, 2]$, $y = [1, 3, 2]$, the point $x = 1.5$ lies between $x_1 = 1$ and $x_2 = 2$:

$$\hat{f}(1.5) = 3 + \frac{(2 - 3)(1.5 - 1)}{2 - 1} = 3 - 0.5 = 2.5.$$

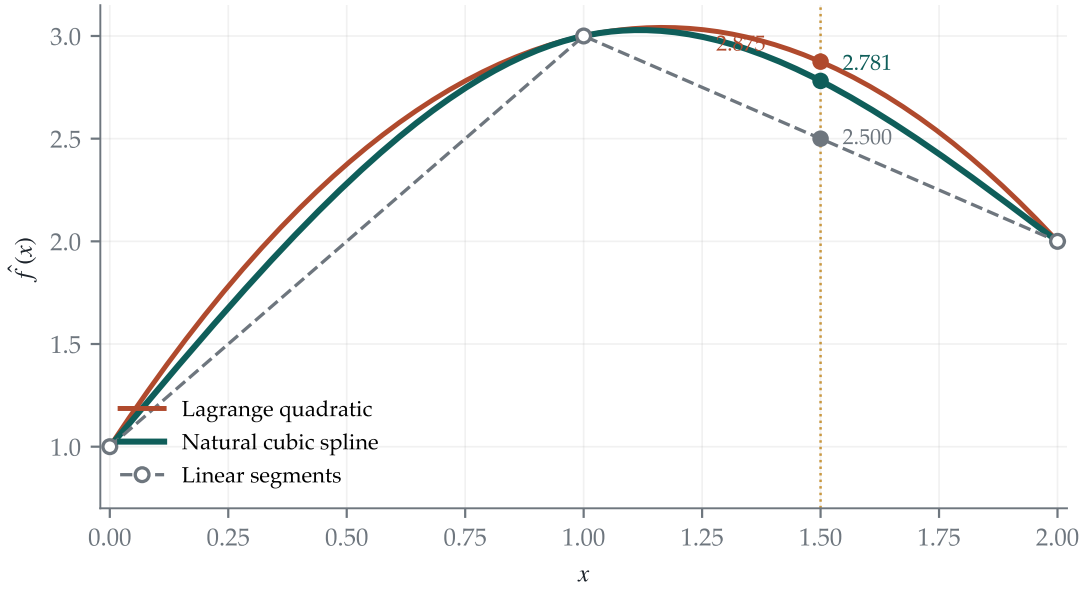


Figure 20.1 The same three points, three interpolants: piecewise linear (dashed), natural cubic spline (solid), and the Lagrange quadratic (thin). At $x = 1.5$ they give 2.500, 2.781 and 2.875 respectively.

图 20.1 同一组三点的三种插值：分段线性（虚线）、自然三次样条（粗实线）、Lagrange 二次多项式（细线）。在 $x = 1.5$ 处分别为 2.500、2.781、2.875。

中文要点

线性插值等价于“在相邻两点之间按比例加权”：x 离谁近，谁的权重就大。它简单、不会过冲，但导数在节点处不连续，曲线看起来有折角；如果真实函数是光滑的，线性插值的误差是 $O(h^2)$，其中 h 是节点间距。`np.interp` 默认就是线性插值，且对超出范围的值会返回端点值（可用 `left/right` 改变），这一点与“外推”不同。

20.4 Error of linear interpolation

线性插值的误差

If f is twice continuously differentiable on $[x_i, x_{i+1}]$, then for some ξ in that interval,

$$f(x) - \hat{f}(x) = \frac{f''(\xi)}{2}(x - x_i)(x - x_{i+1}).$$

Since $|(x - x_i)(x - x_{i+1})| \le h^2/4$ with $h = x_{i+1} - x_i$,

$$|f(x) - \hat{f}(x)| \le \frac{h^2}{8} \max_{[x_i, x_{i+1}]} |f''(x)|.$$

Halving the spacing reduces the bound by a factor of four. The bound also shows why linear interpolation is poor where the curvature is large.

中文要点

误差公式的结构很重要：它由“二阶导数（曲率）”和“节点间距的平方”共同决定。函数越弯、点越稀，误差越大；把点距减半，误差上界降为四分之一。若 $f'' = 0$（直线），线性插值精确；若 f'' 很大（尖峰、快速振荡），需要更密的点或更高阶的方法。注意这是上界，实际误差可能小得多。

Do not extrapolate

Outside $[x_0, x_n]$, the linear formula continues the last segment indefinitely, and higher-order polynomials can grow without bound. The error formula no longer applies, and a small change in the data can produce a large change in the prediction. If extrapolation is unavoidable, use a physically motivated model (regression), not an interpolant.

What to rehearse

State the interpolation conditions and the uniqueness theorem. Compute a linear interpolation by hand for the three-point dataset. Write the error bound and explain the roles of h and f''. Explain why interpolation should not be used on noisy data or outside the data range.

本章考点清单

1. 写出插值条件 $\hat{f}(x_i) = y_i$，并解释“区间内”限制。
2. 用表格比较插值与回归。
3. 说明 $n+1$ 个点唯一确定 n 次多项式的理由（范德蒙行列式）。
4. 手算线性插值 $x = 1.5$ 得 2.5。
5. 写出线性插值误差上界，解释 $h^2/8$ 与 f''。
6. 说明外推为什么危险。

21 Cubic Spline Interpolation

三次样条：用分段三次曲线换来光滑

A single high-degree polynomial through many points tends to oscillate. A cubic spline takes the opposite approach: use many low-degree pieces, and glue them together so smoothly that the joins are invisible. Each piece is a cubic; the pieces share the same value at every node by construction, and we impose continuity of the first and second derivatives at every interior node.

> **中文精讲** Chinese companion
> 三次样条的核心思想是“分段 + 光滑拼接”。在每两个相邻节点之间用一个三次多项式
>
> $$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3,$$
>
> 它必须穿过两端的点，并且在内部节点处与左右两段共享一阶、二阶导数。这样曲线看起来是一条光滑曲线，而不是折线。代价是未知系数很多，需要解一个线性方程组；好处是不会像高次多项式那样在端点附近剧烈振荡。考试常考：数未知数与方程数、说明为什么还需要两个边界条件、自然样条的定义、`CubicSpline` 的调用。

21.1 Counting the unknowns

数一数：未知数与条件数

Suppose there are n data points $x_0 < x_1 < \cdots < x_{n-1}$, hence $N = n - 1$ intervals. Each interval has four coefficients (a_i, b_i, c_i, d_i), so there are

$$4N = 4(n - 1)$$

unknowns. The conditions are:

Condition	Count	Meaning
Interpolation at both ends of each interval	$2N$	$S_i(x_i) = y_i,\ S_i(x_{i+1}) = y_{i+1}$
Continuity of S' at interior nodes	$N - 1$	no corner
Continuity of S'' at interior nodes	$N - 1$	no kink in curvature
Total	$4N - 2$	two conditions short

Continuity of S itself is automatic: both neighbouring pieces are required to equal the same y_i at the shared node, so there is no separate row for it.

Two more conditions are needed. The **natural spline** sets the second derivative to zero at both endpoints:

$$S''(x_0) = 0, \qquad S''(x_{n-1}) = 0.$$

Other choices include the **clamped** spline (specified first derivatives at the ends), the **not-a-knot** condition (third derivative continuous across the first and last interior node), and periodic conditions. Each choice produces a different but equally smooth interpolant.

中文要点

数条件时不要重复：每个区间两端的插值条件共 $2N$ 个；内部节点有 $N-1$ 个，每个节点要求 S' 与 S'' 连续，贡献 $2(N-1)$ 个；总计 $4N-2$，比未知数少 2，所以必须补两个边界条件。函数值 S 的连续是自动的，因为相邻两段都等于同一个 y_i，不要再单独数一次。自然样条令两端二阶导数为 0，是最常用的默认；它不是“唯一正确”的，只是“端点最平直”的一种约定。若物理上知道端点斜率，用 clamped 更合适。

21.2 The tridiagonal system

三对角方程组：实际怎么解

Write $h_i = x_{i+1} - x_i$ and $c_i = S_i''(x_i)/2$. The interpolation and derivative-continuity conditions reduce to a tridiagonal system for the c_i's. With natural boundaries $c_0 = c_{n-1} = 0$, each interior node $i = 1, \dots, n-2$ satisfies

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_i c_{i+1} = 3\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right).$$

Once the c_i are known, the remaining coefficients follow from

$$a_i = y_i, \qquad b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2c_i + c_{i+1})}{3}, \qquad d_i = \frac{c_{i+1} - c_i}{3h_i}.$$

The matrix is tridiagonal, so the system is solved in $O(n)$ operations rather than $O(n^3)$.

中文要点

实际实现不会去解 $4N$ 个未知数的稠密系统，而是先消元到只含 c_i 的三对角系统。三对角矩阵可以用 Thomas 算法在 $O(n)$ 时间内求解，这就是样条适合大量数据的原因。手算一般只考“数条件”和“自然边界”，不会要求解完整系统；但要知道 c_i 与二阶导数的关系 $c_i = S''(x_i)/2$。

21.3 Using SciPy

用 SciPy 实现

```python
import numpy as np
from scipy.interpolate import CubicSpline
import matplotlib.pyplot as plt

x = np.array([0.0, 1.0, 2.0])
y = np.array([1.0, 3.0, 2.0])

cs = CubicSpline(x, y, bc_type="natural")
print(cs(1.5))          # 2.78125
print(cs(1.5, 1))       # first derivative: -1.1875
print(cs.c)             # coefficients, shape (4, n-1)

x_new = np.linspace(0, 2, 200)
plt.plot(x_new, cs(x_new), "b-", label="natural cubic spline")
plt.plot(x, y, "ro", label="data")
plt.legend(); plt.show()
```

For the three-point example, the natural spline is

$$S_0(x) = 1 + 2.75x - 0.75x^3, \qquad S_1(x) = 3 + 0.5(x-1) - 2.25(x-1)^2 + 0.75(x-1)^3.$$

Both pieces meet at $x = 1$ with value 3, slope 0.5 and second derivative -4.5. At $x = 1.5$, the second piece gives

$$3 + 0.25 - 0.5625 + 0.09375 = 2.78125.$$

The Lagrange quadratic through the same points gives 2.875, and linear interpolation gives 2.5; the spline is the smoothest of the three.

中文要点

- `CubicSpline(x, y, bc_type='natural')` 返回一个可调用对象；`cs(x_new)` 直接在新区间上求值，不需要手动写分段。
- `cs(x, 1)` 求一阶导，`cs(x, 2)` 求二阶导；`cs.c` 的 shape 是 `(4, n-1)`，四行分别对应三次、二次、一次、常数项。
- 其他 `bc_type`：`'clamped'`（端点斜率）、`'not-a-knot'`（默认）、`'periodic'`（周期）；也可传 `((1, slope_left), (1, slope_right))`。
- 样条在数据范围内通常很稳；超出范围时 SciPy 默认按端点三次多项式外推，仍可能快速发散，不要依赖。

21.4 Why splines beat high-degree polynomials

为什么样条比高次多项式稳

A degree-n polynomial is global: changing one data point moves the entire curve. A spline is local: changing one point affects only the neighbouring intervals, and the curve remains smooth. This locality controls the **Runge phenomenon**, the violent oscillation of high-degree interpolants near the ends of an equidistant grid.

Spline vs polynomial

Use a spline when the data are smooth and you need a visually smooth curve through many points. Use a single low-degree polynomial when you need a simple global formula or derivatives in closed form. Never use a degree-20 polynomial just because you have 21 points.

What to rehearse

Count the unknowns and conditions for n points; explain why two boundary conditions are required. State the natural-spline conditions. Write the `CubicSpline` call and evaluate at a point. Explain in one sentence why splines avoid the Runge phenomenon.

本章考点清单

1. 写出分段三次的一般形式与 $4(n-1)$ 个未知数。
2. 数出 $4N - 2$ 个条件，并说明缺两个边界条件。
3. 定义自然样条 $S''(x_0) = S''(x_{n-1}) = 0$。
4. 用 `CubicSpline` 插值并在 $x = 1.5$ 求值（2.78125）。
5. 解释三对角系统为什么只需 $O(n)$。
6. 说明样条的局部性如何抑制 Runge 振荡。

22 Lagrange Polynomial Interpolation

Lagrange 插值：用一组“开关”拼出多项式

Lagrange interpolation gives a closed formula for the unique polynomial through $n+1$ points. The formula is not the most efficient way to compute, but it makes the structure of interpolation visible: every data point owns a basis polynomial that equals 1 at that point and 0 at all the others.

> **中文精讲** Chinese companion
> Lagrange 的想法很漂亮：先为每个节点 x_i 造一个“开关函数”$P_i(x)$，它在自己的节点取 1、在别的节点取 0；然后把 P_i 乘以对应的 y_i 再加起来：
>
> $$L(x) = \sum_{i=0}^{n} y_i P_i(x).$$
>
> 因为每个 P_i 在 x_i 处为 1、在 x_j 处为 0，所以 $L(x_j) = y_j$ 自动成立。考试常考：写出基函数、验证 $P_i(x_j) = \delta_{ij}$、对三点数据写出 $L(x)$ 并求值、说明 Lagrange 与 Newton 是同一个多项式。

22.1 The basis polynomials

基函数：每个点一个开关

For nodes $x_0, \dots, x_n$, the i-th Lagrange basis polynomial is

$$\boxed{P_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}}$$

It is a product of n linear factors, hence a polynomial of degree n. At $x = x_i$, every factor is 1; at $x = x_j$ with $j \neq i$, one factor is 0. Therefore

$$P_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

The interpolant is

$$L(x) = \sum_{i=0}^{n} y_i P_i(x),$$

which has degree at most n and satisfies $L(x_j) = y_j$ for every node. By the uniqueness theorem, it is *the* interpolating polynomial — the same one Newton's formula produces.

> **中文要点**
> 基函数的构造可以记成“跳过自己的节点，其余因子相乘”。分母 $x_i - x_j$ 是常数，保证 $P_i(x_i) = 1$。P_i 的次数是 n（$n+1$ 个节点时），所以 L 的次数不超过 n。如果某个 $y_i = 0$，对应的项就消失，但基函数本身仍然存在。节点顺序不影响最终的 L，只影响求和顺序。

22.2 The three-point example

三点例题：把公式写开

For $x = [0, 1, 2]$ and $y = [1, 3, 2]$, the three basis polynomials are

$$P_0(x) = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \tfrac{1}{2}x^2 - \tfrac{3}{2}x + 1,$$

$$P_1(x) = \frac{(x-0)(x-2)}{(1-0)(1-2)} = -x(x-2) = 2x - x^2,$$

$$P_2(x) = \frac{(x-0)(x-1)}{(2-0)(2-1)} = \tfrac{1}{2}x^2 - \tfrac{1}{2}x.$$

Each takes the value 1 at its own node and 0 at the other two. Multiplying by the data values and adding,

$$L(x) = 1 \cdot P_0(x) + 3 \cdot P_1(x) + 2 \cdot P_2(x) = -\tfrac{3}{2}x^2 + \tfrac{7}{2}x + 1.$$

Check: $L(0) = 1$, $L(1) = 3$, $L(2) = 2$. At $x = 1.5$,

$$L(1.5) = -\tfrac{3}{2}(2.25) + \tfrac{7}{2}(1.5) + 1 = -3.375 + 5.25 + 1 = 2.875.$$

This is higher than the linear value 2.5 and the natural-spline value 2.78125.

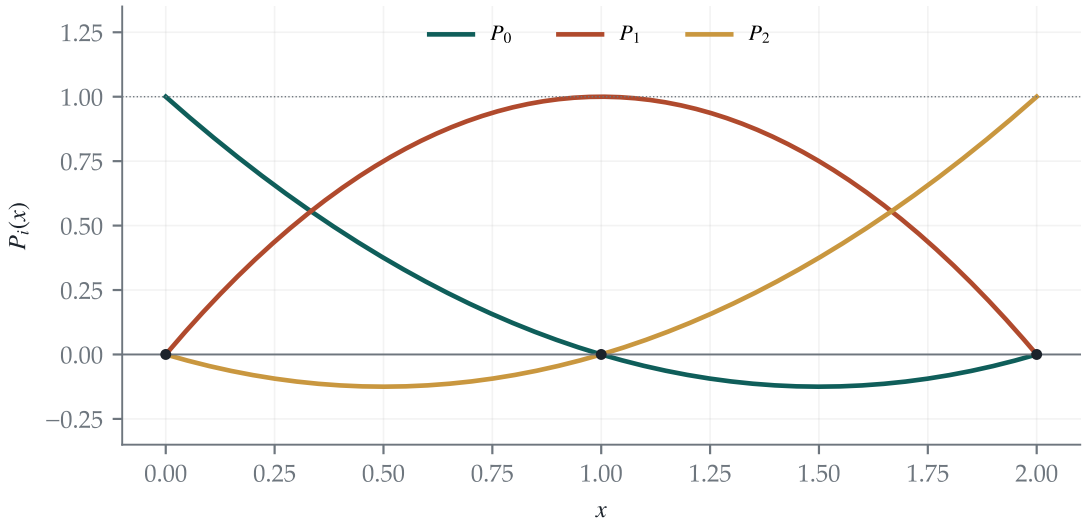


Figure 22.1 The three Lagrange basis polynomials. Each is 1 at its own node and 0 at the other two.
图 22.1 三个 Lagrange 基函数：在各自节点为 1，在其余节点为 0。

> **中文要点**
> 手算时先写因式形式，再展开，最后用三个节点验算。注意课件中对应 $x_0 = 0$ 的第一个基函数（课件标为 P_1）省略了常数项 +1，写成 $-1.5x + 0.5x^2$；正确的 $P_0(0) = 1$ 必须包含 +1，否则 $L(0)$ 就不是 1。考试写因式形式最安全，既不容易漏项，也能直接看出零点。

22.3 Using SciPy, and its limits

SciPy 实现与局限

```
import numpy as np
from scipy.interpolate import lagrange
```

```
x = np.array([0.0, 1.0, 2.0])
y = np.array([1.0, 3.0, 2.0])

poly = lagrange(x, y)       # numpy.poly1d, highest power first
print(poly)                 # -1.5 x^2 + 3.5 x + 1
print(poly(1.5))            # 2.875
```

Newer SciPy versions deprecate `lagrange` in favour of `BarycentricInterpolator`, which evaluates the same polynomial in a numerically stable way:

```
from scipy.interpolate import BarycentricInterpolator
f = BarycentricInterpolator(x, y)
print(f(1.5))               # 2.875
```

The naive Lagrange evaluation costs $O(n^2)$ operations per point: each of the $n+1$ basis polynomials is a product of n factors. Adding a new data point requires rebuilding every basis polynomial, unlike Newton's form. For large n, the barycentric form is preferred; for $n \gtrsim 20$ with equidistant nodes, any single high-degree polynomial is a bad idea because of the Runge phenomenon.

中文要点

- `lagrange(x,y)` 返回 `numpy.poly1d`，系数按从高次到低次排列；`poly(1.5)` 直接求值。
- 新版本 SciPy 已弃用 `lagrange`，推荐 `BarycentricInterpolator`；两者在数学上是同一个插值多项式。
- 朴素 Lagrange 每算一个点要 $O(n^2)$ 次运算；重心形式先把权重算好，之后每点 $O(n)$。
- 加点时 Lagrange 要全部重算；Newton 的差商形式只需增加一项，这是它的主要优势。
- 高次等距节点会 Runge 振荡；不要因为"公式能算"就用到 20 个点以上。

Interpolation is not approximation

Lagrange gives the exact interpolating polynomial, but "exact at the nodes" does not mean "close in between". Between equidistant nodes, a high-degree polynomial can deviate wildly; near the ends, the deviation can be larger than the data range. The fix is to use splines, Chebyshev nodes, or a lower-degree model — not to add more equidistant points.

What to rehearse

Write $P_i(x)$ from memory and prove $P_i(x_j) = \delta_{ij}$. For the three-point dataset, derive $L(x) = -\frac{3}{2}x^2 + \frac{7}{2}x + 1$ and evaluate at 1.5. Compare the Lagrange, spline and linear values. State two computational disadvantages of Lagrange and two alternatives.

本章考点清单

1. 写出 Lagrange 基函数与插值多项式。
2. 验证 $P_i(x_j) = \delta_{ij}$。
3. 对三点数据展开 $L(x)$，求 $L(1.5) = 2.875$。
4. 说明 Lagrange 与 Newton 的唯一性关系。
5. 说出 $O(n^2)$ 的计算代价与加点需重算的缺点。
6. 解释高次等距节点的 Runge 风险。

23 Newton's Divided-Difference Form

Newton 插值：可以逐点扩展的多项式

Lagrange's formula is elegant but expensive to update. Newton writes the same interpolating polynomial in a nested form whose coefficients are computed recursively from **divided differences**. Adding a new data point then costs only one new diagonal of the table, not a complete rebuild.

> **中文精讲** Chinese companion
>
> Newton 插值回答的是"如果我后来又测了一个点，能不能不算前面所有系数？"答案是可以。它把多项式写成
>
> $$N(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)\cdots(x - x_{n-1}),$$
>
> 其中 $a_k = f[x_0, \ldots, x_k]$ 是 k 阶差商。差商表逐列计算，每增加一个点只在表的右下角多算一列。考试常考：写出差商定义、填三点差商表、写出 Newton 多项式、说明它与 Lagrange 是同一个多项式、解释"加点方便"的原因。

23.1 The Newton form

Newton 形式：嵌套的基

For $n + 1$ nodes $x_0, \ldots, x_n$, the interpolant is

$$\boxed{N(x) = a_0 + \sum_{k=1}^{n} a_k \prod_{j=0}^{k-1} (x - x_j)}$$

with

$$a_k = f[x_0, x_1, \ldots, x_k].$$

The first term is a constant, the second is linear, the third is quadratic, and so on. The nested (Horner-like) evaluation is efficient:

$$N(x) = a_0 + (x - x_0)\Big(a_1 + (x - x_1)\big(a_2 + \cdots\big)\Big),$$

which costs $O(n)$ operations once the coefficients are known.

> **中文要点**
>
> Newton 形式与 Lagrange 形式是同一个多项式的两种写法（唯一性保证）。它的基函数是"累积乘积"：第 k 项含有 $(x - x_0)(x - x_1)\cdots(x - x_{k-1})$，因此每加一个新点，只需在末尾增加一项，前面的项完全不变。这就是"加点方便"的来源。注意 $a_0 = y_0$，不是 y_1。

23.2 Divided differences

差商：递归的斜率

The zeroth-order divided difference is just the function value:

$$f[x_i] = y_i.$$

Higher orders are defined recursively:

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

For $k = 1$, this is the ordinary slope between two points:

$$f[x_i, x_{i+1}] = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}.$$

For $k = 2$,

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}.$$

The denominator is always the difference between the outermost nodes, not the neighbouring spacing.

> **中文要点**
> 差商可以理解为“divided difference = 斜率的变化率”。一阶差商是两点斜率；二阶差商是两个一阶差商的差，再除以两端节点的距离。分母永远是 $x_{i+k} - x_i$，不是 $x_{i+1} - x_i$，这是手算最容易错的地方。节点不必等距，差商公式对任意不同节点都成立。

23.3 The three-point table

三点差商表：完整算一遍

For the data $(0, 1), (1, 3), (2, 2)$:

x_i	$y_i = f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_0, x_1, x_2]$
0	1		
1	3	$\frac{3-1}{1-0} = 2$	
2	2	$\frac{2-3}{2-1} = -1$	$\frac{-1-2}{2-0} = -1.5$

The coefficients are the first entry of each column: $a_0 = 1$, $a_1 = 2$, $a_2 = -1.5$. Hence

$$N(x) = 1 + 2x - 1.5x(x-1) = 1 + 3.5x - 1.5x^2,$$

which matches the Lagrange polynomial. At $x = 1.5$:

$$N(1.5) = 1 + 3 - 1.5(1.5)(0.5) = 4 - 1.125 = 2.875.$$

> **中文要点**
> 填表时沿对角线取系数：第一列第一个是 a_0，第二列第一个是 a_1，第三列第一个是 a_2。不要把整列都当成系数。验算方法：代回三个节点，应分别得到 1、3、2；再与 Lagrange 展开式比较，二次项系数都是 -1.5。

23.4 Implementation

Python 实现：差商表 + 嵌套求值

```python
import numpy as np

def divided_diff(x, y):
    """Return the divided-difference table (lower triangle used)."""
    n = len(y)
    coef = np.zeros((n, n))
    coef[:, 0] = y
    for j in range(1, n):
        for i in range(n - j):
            coef[i, j] = ((coef[i+1, j-1] - coef[i, j-1])
                          / (x[i+j] - x[i]))
    return coef

def newton_poly(coef, x_data, x):
    """Evaluate the Newton polynomial by nested multiplication."""
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n-k] + (x - x_data[n-k]) * p
    return p

x = np.array([0.0, 1.0, 2.0])
y = np.array([1.0, 3.0, 2.0])
table = divided_diff(x, y)
a = table[0, :]                 # coefficients a0, a1, a2
print(a)                        # [1.  2.  -1.5]
print(newton_poly(a, x, 1.5))   # 2.875
```

The table is lower triangular in the sense that column j has $n - j$ meaningful entries. The coefficients are the first row `table[0, :]`. The evaluation loop starts from the highest coefficient and works backwards, exactly like Horner's rule.

> **中文要点**
>
> - `divided_diff` 的双重循环：外层 j 是阶数，内层 i 是起点；`x[i+j]-x[i]` 是分母。
> - `table[0, :]` 取第一行，正好是 $a_0, a_1, \ldots, a_n$；表中的其他行是中间量。
> - `newton_poly` 从 a_n 开始，每次乘 $(x - x_{n-k})$ 再加 a_{n-k}，这是嵌套乘法，不是展开求和。
> - 若新增一个点 x_{n+1}，只需在表中增加一行/一列，原有系数不变；这是 Newton 相对 Lagrange 的核心优势。

23.5 Adding a point

加点：只多算一条对角线

Suppose the interpolant through $x_0, \ldots, x_n$ is known. Add x_{n+1}. The new coefficient is

$$a_{n+1} = f[x_0, \ldots, x_{n+1}],$$

and the new polynomial is

$$N_{\text{new}}(x) = N(x) + a_{n+1} \prod_{j=0}^{n} (x - x_j).$$

The old terms are untouched. To compute a_{n+1}, extend the difference table by one row and one column; the work is $O(n)$, not $O(n^2)$.

中文要点

加点公式说明：新多项式 = 旧多项式 + 一个新基函数乘以新系数。旧系数 $a_0, \ldots, a_n$ 完全不变，因为它们只依赖前 $n+1$ 个点。实际计算时，把新点放在表的最后，补算最后一列即可；这就是“在线”插值比 Lagrange 方便的原因。

What to rehearse

Write the Newton form and the recursive definition of divided differences. Fill the three-point table and obtain $1 + 3.5x - 1.5x^2$. Evaluate at 1.5. Explain why adding a node only adds one term. Compare the cost of evaluating Lagrange versus Newton.

本章考点清单

1. 写出 Newton 多项式与系数 $a_k = f[x_0, \ldots, x_k]$。
2. 用递归定义计算一阶、二阶差商（分母是两端节点差）。
3. 填三点差商表，取第一行作为系数。
4. 用嵌套乘法求 $N(1.5) = 2.875$。
5. 说明加点只增加一项、旧系数不变。
6. 解释 Newton 与 Lagrange 是同一个多项式。

24 Comparing Interpolation Methods

插值方法比较：什么时候用哪一种

All polynomial interpolants through the same data are the same polynomial; the differences are in *representation*, *cost* and *stability*. Splines are a different family altogether: piecewise cubics, not a single high-degree polynomial. The practical question is therefore not "which formula is correct?" but "which curve should I trust between the data points?"

> **中文精讲** Chinese companion
> 四种方法，三个维度：
> - **线性**：最简单，$O(n)$ 构造，折线，导数不连续；适合表格查询、绘图占位。
> - **三次样条**：分段三次，C^2 光滑，$O(n)$ 求解，局部性好；适合大量光滑数据。
> - **Lagrange**：单一高次多项式，公式漂亮，$O(n^2)$ 求值，加点要重算；适合小 n、教学推导。
> - **Newton**：同一个多项式，差商系数，加点只多一项，嵌套求值 $O(n)$；适合逐步加点。
>
> 考试常考：给数据与场景选方法；解释为什么高次等距节点的多项式会振荡；说明样条为什么更稳。

24.1 The comparison table

四种方法总表

Method	Form	Smoothness	Add a point	Cost	Best for
Linear	degree 1 pieces	C^0	local	$O(n)$	tables, quick estimates
Cubic spline	cubic pieces	C^2	local	$O(n)$ solve	many smooth samples
Lagrange	degree n	C^∞	rebuild all	$O(n^2)$ eval	small n, theory
Newton	degree n	C^∞	add one term	$O(n^2)$ build, $O(n)$ eval	incremental data

Table 24.1 The four methods of Chapter 17; "local" means a change affects only nearby intervals.

> **中文要点**
> "光滑"指曲线有几阶连续导数：线性是 C^0（连续但折角），样条是 C^2（曲率也连续），高次多项式是 C^∞（无限可导），但"光滑"不等于"稳定"。Lagrange/Newton 的高次多项式在节点之间可能剧烈振荡，样条则不会。选方法时先看数据量、光滑性要求和是否需要逐步加点，而不是看公式是否好看。

24.2 The Runge phenomenon

Runge 现象：为什么高次等距插值会失控

The classic counterexample is

$$f(x) = \frac{1}{1 + 25x^2}, \qquad x \in [-1, 1].$$

With 11 equidistant nodes, the degree-10 interpolant passes through every node but oscillates vi-

olently near the endpoints. The maximum error on the interval is about 1.92, far larger than the function's own range on most of the interval.

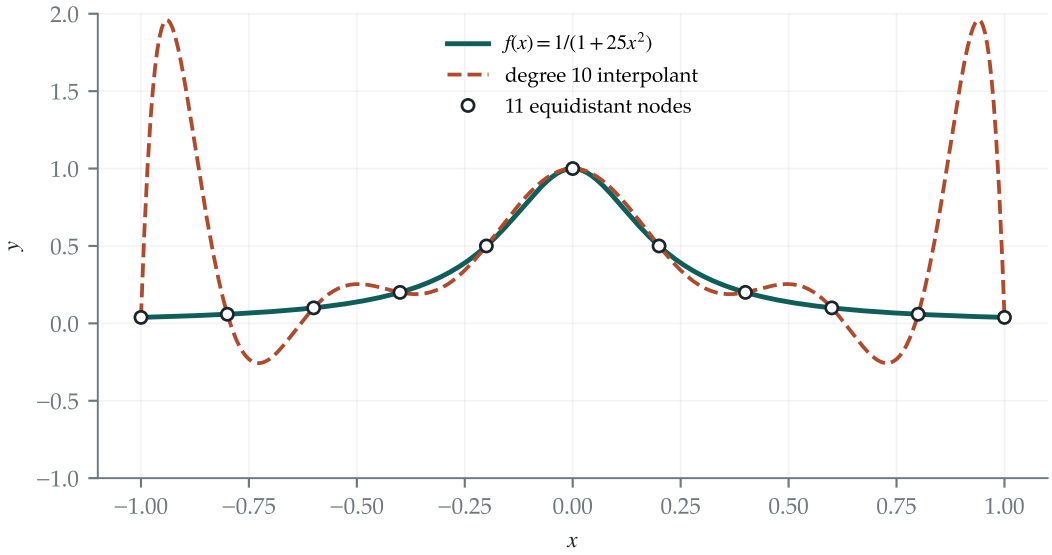


Figure 24.1 Runge's example: the degree-10 interpolant through 11 equidistant nodes overshoots near $x = \pm 1$.

图 24.1 Runge 反例：11 个等距节点上的 10 次插值多项式在两端剧烈过冲。

The error formula explains why:

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

Increasing n makes the factorial grow, but it also makes the $(n+1)$-st derivative and the product of distances grow. For equidistant nodes the product near the ends can overwhelm the factorial, so the error grows rather than shrinks.

Two cures for Runge

1. **Change the nodes.** Cluster them near the ends using Chebyshev points

$$x_k = \cos\left(\frac{2k+1}{2(n+1)}\pi\right), \qquad k = 0, \dots, n.$$

   The product $\prod |x - x_i|$ becomes small across the whole interval, and the interpolation error decays rapidly for smooth f.
2. **Change the method.** Use a cubic spline, which is local and does not suffer from the global oscillation.

中文要点

Runge 现象不是"程序错了"，而是等距节点 + 高次多项式的必然结果。直觉上，高次多项式为了同时穿过所有点，必须在端点附近急剧转弯，形成大过冲。两种解法：一是把节点换成 Chebyshev 点（两端密、中间疏），二是直接用分段样条。注意：增加等距节点反而可能让最大误差更大，所以"点越多越准"是错的。

24.3 Choosing a method

选型指南

1. **Is the data noisy?** If yes, do not interpolate; use least squares or a smoothing spline. Interpolation reproduces noise.
2. **How many points?** For $n \leq 5$ and a need for a closed-form polynomial, Lagrange or Newton is fine. For $n \gg 10$, use a spline.
3. **Do you need derivatives?** A cubic spline gives continuous first and second derivatives; linear interpolation gives discontinuous slopes; a high-degree polynomial gives derivatives but may oscillate.
4. **Will the data grow?** If points arrive one at a time, Newton's form updates cheaply.
5. **Is the function periodic?** Use a periodic spline or trigonometric interpolation, not a generic polynomial.
6. **Do you need to extrapolate?** Prefer a physical model; all interpolants are unreliable outside the data range.

> **中文要点**
> 选型顺序可以记成"先看噪声，再看点数，再看导数，再看是否增量"。有噪声 → 回归/平滑；点少且要公式 → Lagrange/Newton；点多且要光滑 → 样条；逐步加点 → Newton；周期 → 周期样条；要外推 → 物理模型。不要用"最高次多项式"作为默认答案。

24.4 A worked comparison

同一组数据的三种结果

For the three points $(0, 1), (1, 3), (2, 2)$ at $x = 1.5$:

Method	Value at 1.5	Slope at 1.5	Character
Linear	2.500	$2 \to -1$ (jump at nodes)	corner
Natural cubic spline	2.781	-1.1875	smooth, local
Lagrange / Newton	2.875	$3.5 - 3x = -1.0$	single parabola, global

The three answers differ by up to 0.375, which is 15% of the linear value. This is not a rounding error; it is the ambiguity of interpolation between sparse data. With only three points, no method is "right" without additional assumptions about the underlying function.

> **中文要点**
> 同一组数据、同一个 $x = 1.5$，三种方法给出 2.500、2.781、2.875。差异来自"节点之间用什么形状连接"的假设：线性假设直线，样条假设曲率连续，二次多项式假设整体是抛物线。数据越稀疏，方法选择的影响越大；这也是为什么插值必须报告方法，而不能只给一个数。

> What to rehearse
> Compare the four methods in one table: form, smoothness, cost, update behaviour, best use. Explain the Runge phenomenon and two cures. Given a scenario (noisy data, 200 smooth points, incremental data, periodic signal), choose a method and justify it. Compute the three values at 1.5 and explain why they differ.

PART

07

Exam Room

公式、API 与模拟题

Everything condensed: formula sheet, Python API, 30 drill questions, and a last-minute checklist.

25 Formula Sheet

公式速查表

Number bases ·进制

$$(d_k \cdots d_0 . f_1 f_2 \cdots)_N = \sum_{i=0}^{k} d_i N^i + \sum_{j\geq 1} f_j N^{-j}$$

Integer $\to$ base N: divide, read remainders upward.
b bits $\Rightarrow 2^b$ patterns; unsigned $0 \ldots 2^b - 1$.

Binary ·二进制

$$37 = 100101_2, \qquad 17 = 10001_2$$

$$37 + 17 = 54 = 110110_2$$

$$37 \times 17 = 629 = 1001110101_2$$

IEEE 754 double ·双精度

$$n = (-1)^s \, 2^{e-1023} \, (1 + f)$$

$$\text{subnormal: } (-1)^s \, 2^{-1022} \, (0 + f)$$

Fields $1 + 11 + 52$; bias 1023.
$e = 0$: zero / subnormal;
$e = 2047$: $\pm\infty$ or NaN.

$$\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$$

$$\text{spacing}(x) \approx |x| \, \varepsilon$$

Largest 1.7977×10^{308}.
Smallest normal 2.225×10^{-308}.
Smallest subnormal 4.94×10^{-324}.

Error ·误差

$$\text{abs} = |\hat{x} - x|, \qquad \text{rel} = \frac{|\hat{x} - x|}{|x|}$$

$$0.1 + 0.2 + 0.3 - 0.6 \approx 1.11 \times 10^{-16}$$

$$4.9 - 4.845 \approx 0.0550000000000006$$

Stable quadratic:

$$q = -\tfrac{1}{2}\left(b + \text{sgn}(b)\sqrt{b^2 - 4ac}\right)$$

$$x_1 = q/a, \qquad x_2 = c/q$$

Use `np.isclose`, `math.fsum`.

Text and CSV ·文本

Modes: `r, w, a, r+, w+, a+`.
Use `with open(...) as f:` so the file closes.
`savetxt(f, A, fmt, header);`
`loadtxt(f, delimiter, skiprows, unpack);`
`genfromtxt` tolerates missing values.

Pickle and JSON ·序列化

`pickle.dump(obj, f)` / `load(f)`; binary `wb/rb`.
`json.dump(obj, f, indent=2)` / `load(f)`.
Pickle: Python-only, unsafe if untrusted.
JSON: six types, cross-language.

HDF5 ·层次数据

`h5py.File('f.h5', 'w')`; groups, datasets, `attrs`.
`hf['/path'] = array`; slice with `[:]`.
`list(hf.keys())`, `visititems`.
Chunking + compression for large arrays.

2D plot ·二维绘图

`plot(x, y, 'ro')`: red circles.
Labels, title, legend, grid; `figsize`; `dpi`.
Save before `show`: `savefig('f.png')`.

3D plot ·三维绘图

```
fig.add_subplot(projection='3d')
X, Y = np.meshgrid(x, y)
ax.plot_surface(X, Y, Z, cmap='cividis')
ax.scatter(x, y, z); view_init(elev, azim).
```

Map ·地图

$lon \in [-180, 180]$, $lat \in [-90, 90]$.

```
projection=ccrs.PlateCarree()
set_extent([lon0,lon1,lat0,lat1])
coastlines(resolution='10m')
```

Data points need `transform=ccrs.PlateCarree()`.

Animation ·动画

```
FuncAnimation(fig, update, frames=n,
    init_func=init, interval=15)
ani.save('out.mp4', writer=writer, dpi=100)
```

Writer: `writers['ffmpeg']`; fallback Pillow.

Least squares ·最小二乘

$$\hat{y}(x) = \sum_{j=1}^{n} \alpha_j f_j(x)$$

$$E = \sum_{i=1}^{m} \big(y_i - \hat{y}(x_i)\big)^2 = \|Y - A\alpha\|_2^2$$

$$A_{ij} = f_j(x_i), \qquad \hat{Y} = A\alpha$$

$$A^\top A \, \hat{\alpha} = A^\top Y$$

$$\hat{\alpha} = (A^\top A)^{-1} A^\top Y$$

$A^\top A$ invertible iff columns independent.
Projection: $A^\top (Y - A\hat{\alpha}) = 0$.

Spring through origin:

$$\hat{k} = \frac{\sum x_i F_i}{\sum x_i^2}$$

Line $y = mx + b$:

$$m = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}$$

$$b = \frac{\sum y_i - m\sum x_i}{n}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$\mathrm{Cov}(\hat{\alpha}) = s^2 (A^\top A)^{-1}, \quad s^2 = \frac{\mathrm{SS}_{\mathrm{res}}}{m - n}$$

Nonlinear ·非线性

$$y = ae^{bx} \implies \ln y = \ln a + bx$$

$$y = bx^m \implies \ln y = m \ln x + \ln b$$

```
curve_fit(model, x, y, p0=[...])
perr = sqrt(diag(pcov)).
```

Interpolation ·插值

$\hat{f}(x_i) = y_i$, $i = 0, \ldots, n$.

Linear:

$$\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$$

Error:

$$|f - \hat{f}| \le \frac{h^2}{8} \max |f''$$

General:

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

Cubic spline:

$$S_i = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

$4(n-1)$ unknowns, $4(n-1) - 2$ conditions.
Natural: $S''(x_0) = S''(x_{n-1}) = 0$.
Lagrange:

$$P_i(x) = \prod_{j \ne i} \frac{x - x_j}{x_i - x_j}, \quad L(x) = \sum y_i P_i(x)$$

$$P_i(x_j) = \delta_{ij}$$

Newton:

$$N(x) = a_0 + \sum_{k=1}^{n} a_k \prod_{j=0}^{k-1} (x - x_j)$$

$$a_k = f[x_0, \ldots, x_k]$$

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Example: $L(x) = -\frac{3}{2}x^2 + \frac{7}{2}x + 1$.
$L(1.5) = 2.875$; spline 2.78125; linear 2.5.
Runge $1/(1 + 25x^2)$: 11 nodes, max error ≈ 1.92.
Chebyshev:

$$x_k = \cos\left(\frac{2k+1}{2(n+1)}\pi\right)$$

Big-O ·复杂度

$$O(1) < O(\log n) < O(n)$$
$$< O(n \log n) < O(n^2) < O(2^n)$$

$\log_2 10^6 \approx 20$; $n \log_2 n \approx 2 \times 10^7$;
$n^2 = 10^{12}$.

26 Python API Cheat Sheet

Python API 速查表

Core Python ·基础

Call	What it does / returns
`range(start, stop, step)`	half-open integer sequence; stop excluded
`enumerate(seq)`	yields `(index, value)`
`zip(a, b)`	yields pairs, stops at the shorter
`sorted(data, key=lambda x: x[1], reverse=True)`	new sorted list
`[f(x) for x in xs if cond]`	list comprehension
`{k: v for k, v in d.items()}`	dictionary comprehension
`def f(a, b=1): ...`	default parameter evaluated once at definition
`lambda x: x**2`	anonymous one-expression function
`f"{x:.2f}"`	formatted string, two decimals

NumPy ·数组与线性代数

Call	What it does / returns
`np.array([1,2,3])`	1-D array; dtype inferred
`np.linspace(a,b,n)`	`n` evenly spaced points, endpoints included
`np.arange(a,b,h)`	step `h`, stop excluded
`np.meshgrid(x,y)`	2-D coordinate arrays, shape `(len(y),len(x))`
`np.column_stack((a,b))`	columns stacked into a 2-D array
`np.savetxt(f, A, fmt, header)`	write a 1-D/2-D array as text
`np.loadtxt(f, delimiter, skiprows, unpack)`	read a rectangular table
`np.genfromtxt(f, names=True)`	tolerant reader, handles missing values
`np.linalg.lstsq(A,y,rcond=None)`	`(coef, residuals, rank, singular_values)`
`np.linalg.pinv(A)`	Moore–Penrose pseudo-inverse
`np.polyfit(x,y,d)`	polynomial coefficients, highest power first
`np.polyval(p,x)`	evaluate polynomial
`np.spacing(x)`	gap to the next representable float
`np.finfo(float)`	`eps`, `max`, `tiny`, `bits`
`np.isclose(a,b,rtol,atol)`	element-wise tolerance comparison
`np.sum(a)`	sum (pairwise summation)

SciPy ·拟合与插值

Call	What it does / returns
`scipy.optimize.curve_fit(f,x,y,p0)`	`(popt, pcov)` for nonlinear least squares
`scipy.interpolate.interp1d(x,y)`	callable linear interpolant (legacy interface)
`scipy.interpolate.CubicSpline(x,y,bc_type)`	C^2 piecewise cubic; `cs(x,1)` derivative
`scipy.interpolate.lagrange(x,y)`	`numpy.poly1d` (deprecated in new SciPy)
`scipy.interpolate.BarycentricInterpolator(x,y)`	stable evaluation of the same polynomial
`np.interp(x_new, x, y)`	simple 1-D linear interpolation

Matplotlib ·绘图

Call	What it does / returns
`fig, ax = plt.subplots(figsize=(w,h))`	figure and axes objects
`ax.plot(x,y,'r--',label=...)`	line/marker plot; format string
`ax.scatter(x,y,s=...,c=...)`	scatter plot
`ax.set_xlabel/ylabel/title`	axis and figure labels

`ax.legend(loc='upper right')`	legend from `label=` arguments
`ax.grid(True, alpha=0.3)`	reference grid
`fig.savefig('f.png', dpi=300, bbox_inches='tight')`	export before `show`
`fig.add_subplot(projection='3d')`	3-D axes
`ax.plot_surface(X,Y,Z,cmap='cividis')`	surface from meshgrid
`ax.view_init(elev=30, azim=-125)`	camera angles
`fig.colorbar(surf)`	colour scale for a mapped artist
`animation.FuncAnimation(fig, update, frames=n)`	animation object
`ani.save('out.mp4', writer=writer, dpi=100)`	encode frames

Files and data ·文件与数据

Call	What it does / returns
`open(path, mode)`	file object; use `with`
`f.read() / readline() / readlines()`	whole file / one line / list of lines
`csv.writer(f).writerows(rows)`	write rows with quoting
`csv.DictReader(f)`	rows as dictionaries
`pd.read_csv(path)`	DataFrame with header and dtypes
`pd.DataFrame.to_csv(path, index=False)`	write a table
`pickle.dump(obj,f) / load(f)`	binary Python object snapshot
`json.dump(obj,f,indent=2) / load(f)`	text key–value format
`h5py.File(path,'w')`	HDF5 file; groups, datasets, attrs
`hf.create_dataset(path, shape, compression='gzip')`	chunked/compressed array
`list(hf.keys())`	children of a group

Cartopy ·地图

Call	What it does / returns
`plt.axes(projection=ccrs.PlateCarree())`	map axes with a projection
`ax.set_extent([lon0,lon1,lat0,lat1])`	zoom window
`ax.coastlines(resolution='10m')`	coastlines at 110/50/10 m
`ax.add_feature(cfeature.LAND)`	filled land, ocean, borders, lakes, rivers
`ax.gridlines(draw_labels=True)`	latitude/longitude graticule
`ax.plot(lon,lat,transform=ccrs.PlateCarree())`	data in geographic coordinates

Three code skeletons worth memorising

三个必须能默写的代码骨架

Linear least squares + plot.

```
A = np.column_stack([x, np.ones_like(x)])
coef, *_ = np.linalg.lstsq(A, y, rcond=None)
y_hat = A @ coef
plt.plot(x, y, "o", label="data")
plt.plot(x, y_hat, "-", label="fit")
plt.xlabel("x"); plt.ylabel("y"); plt.legend(); plt.show()
```

Interpolation + plot.

```
from scipy.interpolate import CubicSpline
cs = CubicSpline(x, y, bc_type="natural")
x_new = np.linspace(x.min(), x.max(), 200)
plt.plot(x, y, "o", label="data")
plt.plot(x_new, cs(x_new), "-", label="spline")
plt.legend(); plt.show()
```

Write and read a table.

```
data = np.column_stack((t, y1, y2))
```

```
np.savetxt("out.txt", data, fmt="%.3f",
           header="t y1 y2")
t2, a2, b2 = np.loadtxt("out.txt", unpack=True)
```

> **中文提示**
> 考试写代码时，优先保证“骨架正确”：导入、数据、模型、求解、绘图、标签。不要背复杂的参数；把 `lstsq`、`CubicSpline`、`savetxt/loadtxt`、`plot` 这四组写熟，足以覆盖大部分编程题。若题目要求“不使用库函数”，再手写正规方程或差商表。

27 30 Exam-Style Questions

30 道期末风格练习题（含答案）

Work each question before reading the answer. The questions are deliberately short; most are worth 2–5 marks in a written exam. If you can answer 25 of 30 without notes, the material is in good shape.

Part A · Python and complexity

1. **Trace the loop.** What is printed?

```
data = [3, -1, 7, 0, 5]
for i, v in enumerate(data):
    if v < 0:
        continue
    if v == 0:
        break
    print(i, v)
```

A. `(0, 3)` then `(2, 7)`. The negative value is skipped; the zero triggers `break` before printing, so index 4 is never reached.

2. **Comprehension.** Rewrite as a list comprehension: all squares of even integers from 1 to 10.

A. `[x*x for x in range(1, 11) if x % 2 == 0]` → `[4, 16, 36, 64, 100]`.

3. **Return versus print.** A function computes a value and calls `print(value)` but has no `return`. What does `x = f()` store?

A. `None`. `print` writes to the screen; only `return` hands a value back.

4. **Closure.** What does this print, and why?

```
def make_adder(n):
    def add(x):
        return x + n
    return add
add5 = make_adder(5)
print(add5(3))
```

A. 8. The inner function captures `n` from the enclosing scope; the value 5 survives after `make_adder` returns.

5. **Complexity.** Give the Big-O of each snippet.

```
# (a)
for i in range(n):
    for j in range(i):
        ops += 1
# (b)
while n > 1:
    n = n // 2
```

A. (a) $n(n-1)/2 = O(n^2)$. (b) the size halves each step, so $O(\log n)$.

Part B · Numbers, floating point and error

6. **Bases.** Convert 45_{10} to binary and evaluate 121_3.

   A. $45 = 32 + 8 + 4 + 1 = 101101_2$. $121_3 = 9 + 6 + 1 = 16_{10}$.

7. **Decode a double.** The pattern is `0 10000000000 1000...0`. What number is it?

   A. $s = 0$, $e = 1024$, real exponent 1, $f = 0.5$, so $n = +2^1(1.5) = 3.0$.

8. **Why `0.1 + 0.2 != 0.3`.** Explain in two sentences and give the correct comparison.

   A. 0.1, 0.2 and 0.3 have infinite repeating binary expansions and are stored as nearby doubles; their sum rounds to 0.30000000000000004. Compare with `np.isclose(0.1+0.2, 0.3)` or a tolerance.

9. **Spacing.** What is `np.spacing(1e6)` approximately, and why does `1e16 + 1 == 1e16`?

   A. About 1.16×10^{-10}. At 10^{16} the spacing is 2, so adding 1 rounds back to 10^{16}.

10. **Error types.** Distinguish round-off error from truncation error, and explain why decreasing a step size h is not always better.

    A. Round-off comes from finite precision; truncation comes from replacing an infinite process by a finite one. Smaller h reduces truncation but amplifies cancellation/round-off, so total error often has a U-shape.

Part C · Files and formats

11. **Modes.** Which mode erases an existing file, which appends, and which requires the file to exist?

    A. `'w'` erases; `'a'` appends; `'r'` (and `'r+'`) requires an existing file.

12. **The `with` statement.** Why is `with open(...) as f:` preferred over manual `open`/`close`?

    A. It closes the file automatically, even if an exception occurs, flushing buffered data and releasing the handle.

13. **Large file.** A text file has 10^7 lines. Which reading method should you use and why?

    A. Iterate directly: `for line in f:`. It holds one line at a time; `read()` or `readlines()` would load the whole file into memory.

14. **NumPy I/O.** Write the calls to save a 2-D array with a header and read it back with columns unpacked.

    A. `np.savetxt("f.txt", A, fmt="%.3f", header="t y1 y2")` then `t, y1, y2 = np.loadtxt("f.txt", unpack=True)`. The `#` header is skipped automatically.

15. **Format choice.** A project has 50 GB of multi-channel sensor data, must be readable by MATLAB, and needs per-channel metadata. Which format, and why not CSV or Pickle?

    A. HDF5: binary, hierarchical, attributes, compression, cross-language. CSV is too large and loses structure; Pickle is Python-only and unsafe as an archive.

Part D · Visualization

16. **Format strings.** Describe the plots produced by `'ro'`, `'g--'` and `'k.-'`.

A. Red circles with no line; green dashed line; black dash-dot line with point markers.

17. **A self-explanatory figure.** List five elements a scientific plot should normally have.

    A. A descriptive title; x and y labels with units; a legend for multiple datasets; a grid or scale cue when values must be read; a caption stating conditions and take-away.

18. **Save versus show.** Why can `plt.savefig` after `plt.show` produce a blank image?

    A. `show` may clear the current figure when the interactive window closes; save before showing, or keep the `fig` object and call `fig.savefig`.

19. **Meshgrid.** For `x` of length 4 and `y` of length 3, what is the shape of `X, Y = np.meshgrid(x, y)`? What is `Z[i,j]`?

    A. Shape $(3, 4)$. `Z[i,j]` corresponds to (x_j, y_i): rows follow y, columns follow x.

20. **Maps.** What does `ax.set_extent([-123.5, -121.5, 37.0, 38.5])` select, and why is `transform=ccrs.PlateCarree()` needed for data?

    A. The San Francisco Bay Area window: longitude from -123.5 to -121.5, latitude from 37 to 38.5. The transform declares the data coordinates as lon/lat so they can be projected onto the map axes.

Part E · Least squares

21. **Design matrix.** Write A for fitting $y = \alpha_0 + \alpha_1 x + \alpha_2 x^2$ to m points, and state its shape.

    A. A is $m \times 3$ with rows $[1, x_i, x_i^2]$. Then $\hat{Y} = A\alpha$ with $\alpha = (\alpha_0, \alpha_1, \alpha_2)^\top$.

22. **Normal equations.** Starting from $E = \|Y - A\alpha\|^2$, derive $A^\top A\alpha = A^\top Y$.

    A. Expand or differentiate: $\partial E/\partial\alpha = -2A^\top(Y - A\alpha) = 0$, hence $A^\top(Y - A\alpha) = 0$, i.e. $A^\top A\alpha = A^\top Y$.

23. **Rank deficiency.** A has columns x and $2x$. What goes wrong, and what does `lstsq` return?

    A. The columns are linearly dependent, so $A^\top A$ is singular and the parameters are not unique. `lstsq` returns a minimum-norm solution with `rank < n`.

24. **Spring.** For $x = [0.1, 0.2, 0.3]$ m and $F = [2.1, 3.9, 6.2]$ N, fit $F = kx$ through the origin.

    A. $\hat{k} = \sum x_i F_i / \sum x_i^2 = (0.21 + 0.78 + 1.86)/(0.01 + 0.04 + 0.09) = 2.85/0.14 \approx 20.36$ N/m.

25. **Nonlinear fit.** How do you fit $y = ae^{bx}$ by linear least squares, and what assumption does the transformation make about the noise?

    A. Fit $\ln y = \ln a + bx$, then $a = e^{\text{intercept}}$, $b =$ slope. It minimises squared errors in $\ln y$, which weights small y more; it is appropriate for multiplicative (constant relative) noise, not additive noise.

Part F · Interpolation

26. **Linear interpolation.** With $(0, 1)$ and $(2, 5)$, estimate $y(1.5)$.

    A. $1 + \frac{5-1}{2-0}(1.5 - 0) = 1 + 4(0.75) = 4.0$.

27. **Uniqueness and Lagrange.** Why is the interpolating polynomial through 5 distinct points unique? For $(0, 1), (1, 3), (2, 2)$, write the middle Lagrange basis P_1.

    A. The 5×5 Vandermonde matrix has determinant $\prod_{i<j}(x_j - x_i) \neq 0$. $P_1(x) = \frac{x(x-2)}{1(1-2)} = 2x - x^2$.

28. **Cubic spline count.** For 6 points, how many unknowns and conditions? Why two extra conditions, and what is the natural choice?

    A. 5 intervals $\times 4 = 20$ unknowns; interpolation $2 \times 5 = 10$, continuity of S' and S'' at 4 interior nodes $= 2 \times 4 = 8$, total 18 conditions, two short. Natural: $S''(x_0) = S''(x_5) = 0$.

29. **Newton table.** For $(0, 1), (1, 3), (2, 2)$, compute the divided differences and write $N(x)$.

    A. $f[x_0, x_1] = 2$, $f[x_1, x_2] = -1$, $f[x_0, x_1, x_2] = (-1 - 2)/(2 - 0) = -1.5$.
    $N(x) = 1 + 2x - 1.5x(x - 1) = 1 + 3.5x - 1.5x^2$.

30. **Runge.** Why can a high-degree polynomial through many equidistant points be worse than a spline? Give two remedies.

    A. Equidistant nodes make $\prod |x - x_i|$ large near the ends; the interpolant oscillates (Runge). Remedies: Chebyshev-clustered nodes, or a cubic spline / lower-degree local method.

> **评分提示**
> 期末编程/计算题通常按步骤给分：写出模型或公式（1 分）→ 代入数据（1 分）→ 结果与单位（1 分）→ 解释/检查（1 分）。即使算错，写清方法和单位也能拿到大部分分数。概念题则要给出“定义 + 一个例子 + 一个反例/限制”。

28 The Last Ten Minutes

考前最后十分钟：只看这一页

Ten equations to write from memory

1. $n = (-1)^s 2^{e-1023}(1+f)$
2. $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$
3. $E = \sum_i (y_i - \hat{y}(x_i))^2 = \|Y - A\alpha\|^2$
4. $A^\top A\hat{\alpha} = A^\top Y$
5. $\hat{\alpha} = (A^\top A)^{-1} A^\top Y$
6. $\hat{k} = \sum x_i F_i / \sum x_i^2$
7. $\hat{f}(x) = y_i + \frac{(y_{i+1}-y_i)(x-x_i)}{x_{i+1}-x_i}$
8. $P_i(x) = \prod_{j\neq i} \frac{x-x_j}{x_i-x_j}$
9. $a_k = f[x_0,\ldots,x_k]$, $f[x_i,x_{i+1}] = \frac{y_{i+1}-y_i}{x_{i+1}-x_i}$
10. $|f - \hat{f}| \le \frac{h^2}{8}\max|f''|$

Ten lines of code

1. `A = np.column_stack([x, np.ones_like(x)])`
2. `coef, *_ = np.linalg.lstsq(A, y, rcond=None)`
3. `np.savetxt("f.txt", A, fmt="%.3f", header="...")`
4. `x, y = np.loadtxt("f.txt", unpack=True)`
5. `cs = CubicSpline(x, y, bc_type="natural")`
6. `popt, pcov = curve_fit(model, x, y, p0=[1, 0.1])`
7. `X, Y = np.meshgrid(x, y); Z = f(X, Y)`
8. `ax.plot_surface(X, Y, Z, cmap="cividis")`
9. `ax.set_extent([lon0, lon1, lat0, lat1])`
10. `ani = FuncAnimation(fig, update, frames=n, interval=15)`

Ten traps

1. `'w'` erases the file.
2. `0.1+0.2 != 0.3`; never compare floats with `==`.
3. `label=` without `legend()` shows nothing.
4. `savefig` after `show` can be blank.
5. `meshgrid` returns shape `(len(y),len(x))`.
6. `set_extent` order is lon, lon, lat, lat.
7. Interpolation is not regression; do not use it on noise.
8. $A^\top A$ singular if columns are dependent.
9. `polyfit` returns highest power first.
10. Spline needs two boundary conditions; natural sets $S'' = 0$.

Exam strategy

1. Read the whole paper first; do the short definitions before the long derivations.
2. Write the formula before substituting numbers; marks are given for the method.
3. Carry units through every calculation; a missing unit loses a mark and catches errors.
4. Sanity-check orders of magnitude: a spring constant of 10^9 N/m or a probability above 1 signals a slip.
5. For code questions, include imports, labels and a comment on complexity or shape.
6. For “explain” questions, use: definition → reason → example/limitation.
7. Leave the last five minutes to check signs, powers of ten and the question’s actual ask.

If you remember only one sentence per part

Numbers: finite bits mean every float is a nearby rational. **Files:** the format is a promise about who can read the data and for how long. **Plots:** axes, units and legends are part of the result. **Fitting:** least squares makes the residual orthogonal to the model space. **Interpolation:** a curve through every point is a strong assumption, not a free lunch.

最后的话

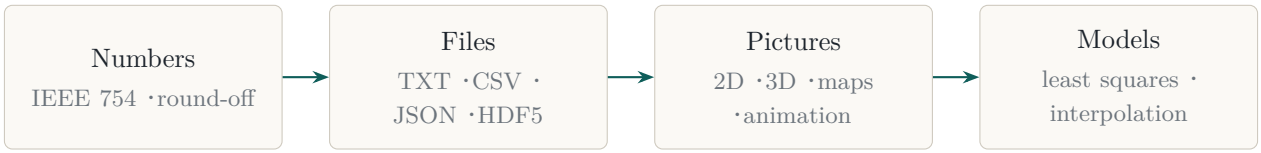
这份讲义覆盖了六章课件与三次辅导课的核心，但考试不会只考“记住”，而会考“迁移”：给你一组新数据、一个新场景，让你选方法、写代码、解释结果。复习时优先做到三件事：

1. 能不看笔记写出每章的核心公式，并说明每个符号的含义；
2. 能默写四条代码骨架（最小二乘、样条、文件读写、绘图）；
3. 能对每个方法说出一个适用场景和一个失败场景。

做到这三点，剩下的就是细心：单位、符号、边界条件、以及题目到底在问什么。祝考试顺利。

The last 24 hours　**考前 24 小时**

- Re-read only the formula sheet and the ten traps; do not start a new topic.
- Reproduce three figures from memory: the damped oscillator, the least-squares fit with residuals, and the Runge overshoot.
- Write the four code skeletons once by hand: `lstsq`, `CubicSpline`, `savetxt/loadtxt`, `plot`.
- Check the exam logistics: calculator, student card, pens, water, and the room.
- Sleep. A rested brain recalls formulas better than a tired one that stayed up for one more chapter.



From bits to models: the whole course in one line.　从比特到模型：整门课的一条主线。

“The purpose of computing is insight, not numbers.”

— Richard Hamming